

ZOMBI2

A Simulator of Genome and Species-Tree Evolution — Concepts & Tutorial

Adrián A. Davín

July 2026

Contents

1	Introduction	1
1.1	The levels of evolution	1
1.2	For the impatient	2
1.3	How this manual is organised	3
2	Installation	4
2.1	From source	4
2.2	Building the compiled engine	4
2.3	Development extras	4
2.4	Optional dependencies	4
2.5	Verifying the installation	4
3	A first simulation	6
3.1	Simulating a species tree	6
3.2	Evolving gene families along the tree	7
3.3	The output folder	7
3.4	Reading the results back in Python	9
4	Species trees (basic models)	11
4.1	The birth–death model	11
4.2	Simulating a tree	11
4.3	Backward versus forward simulation	12
4.4	The <code>Tree</code> object	13
5	Species trees (advanced models)	14
5.1	Episodic (skyline) birth–death	14
5.2	Mass extinctions	16
5.3	Per-lineage rates	17
5.4	Diversity-dependent diversification	18
5.5	Clade-specific rate shifts	19
5.6	Fossilized birth–death	20
6	Ghost lineages	22
6.1	Un-pruning versus simulating forward	22
6.2	Basic use	23
6.3	When do ghosts appear?	23
6.4	The conditional law, briefly	24
7	Genome evolution	25
7.1	Two orthogonal choices	26
7.2	Events, by level	26
7.3	Rate models	28

8	Unordered genomes	29
8.1	The four unordered events	29
8.2	Rates	30
8.3	Transfers	32
8.4	Bounding growth	33
9	Gene trees and output	34
9.1	The event log is a full genealogy	34
9.2	Reconstructing the gene trees	34
9.3	The profile matrix	35
9.4	The output folder	36
10	Gene-family coupling	38
10.1	The model	38
10.2	Building a coupling	40
10.3	Running a coupled simulation	41
10.4	Two caveats worth knowing	42
10.5	Coupling a trait instead of another family	43
11	Ordered genomes	44
11.1	Using an ordered genome	44
11.2	Segment events	45
11.3	How events reach the genome	47
12	Nucleotide genomes	48
12.1	The model	48
12.2	Blocks: units of shared ancestry	49
12.3	Reading a leaf genome	50
12.4	Per-block gene trees and reconciliation	51
12.5	Genes and intergenes	51
12.6	Intergenic indels	53
12.7	The Rust fast path	53
13	Trait evolution	54
13.1	The shared overlay engine	54
13.2	Trait models at a glance	55
13.3	Continuous traits	56
13.4	Discrete traits	60
13.5	Adaptation to different regimes (multi-optimum OU)	64
13.6	Pagel's tree transforms	65
13.7	Historical biogeography (DEC)	65
13.8	From the command line	67
13.9	Output	68
14	Coevolution	69
14.1	The idea	69
14.2	Species and traits	71
14.3	Species and genes	75
14.4	Traits and genes	78

14.5 A note on inference	81
15 Sequence evolution	82
15.1 The problem	82
15.2 The model	82
15.3 Usage from Python	83
15.4 Usage from the CLI	84
15.5 Simulating sequence alignments	85
16 Relaxed molecular clocks	87
16.1 The strict clock and what relaxing it means	87
16.2 Drawing a phylogram	87
16.3 The uncorrelated clocks	88
16.4 The autocorrelated clocks	89
16.5 Feeding a sequence-evolution run	90
16.6 Usage from the CLI	90
References	92

Chapter 1

Introduction

Evolutionary studies rely on statistical analyses of data observed in the present to make inferences about what happened in the past. This creates a fundamental problem: we rarely have access to the real history, so we cannot easily confirm that our methods are recovering it correctly.

Simulation offers a way out. By generating artificial datasets under an explicit model, we build worlds in which we know *everything* about how the data came to be — a solid ground truth against which reconstruction methods can be tested, calibrated, and compared.

ZOMBI2 is a phylogenetic simulator — a ground-up redesign of ZOMBI (Davín et al. 2020) built around a fast Rust engine, a simple command-line interface, and a composable Python library. It offers a collection of models for simulating species trees, gene trees, and traits, independently or jointly, and records the true history behind every dataset it produces. For example, ZOMBI2 can:

- simulate very large species trees under a coalescent point-process model, and decorate them *a posteriori* with extinct (“ghost”) lineages;
- evolve gene families along any tree by duplication, transfer, loss and origination, recovering their gene trees, reconciliations, and copy-number profiles;
- descend to nucleotide resolution — structural rearrangements from which genes emerge as conserved “blocks”, and real DNA sequences under standard substitution models;
- evolve phenotypic traits (Brownian motion, Ornstein–Uhlenbeck, Mk, threshold, biogeographic ranges) along the same tree;
- couple these processes into models of *coevolution* — gene content that shapes traits, traits that shape gene content, or gene families that evolve non-independently;
- run the inverse problem — fitting gene-family rates back to an empirical presence/absence profile by Approximate Bayesian Computation.

1.1 The levels of evolution

Evolution is a single process, but it leaves its trace at several levels at once: a lineage diversifies into a species tree; along that tree, genomes gain and lose gene families; the organisms carry traits that drift and adapt; and inside every gene, molecular sequences accumulate substitutions. Reality draws no line between these levels. We do — because a tractable model usually captures only one of them at a time.

ZOMBI2 is organised around that separation. It simulates four levels: **species** (the dated tree), **genomes** (the gene families along it), **traits** (phenotypes on the tree), and **sequences** (the nucleotides or amino acids in each gene). There are two ways to put them together.

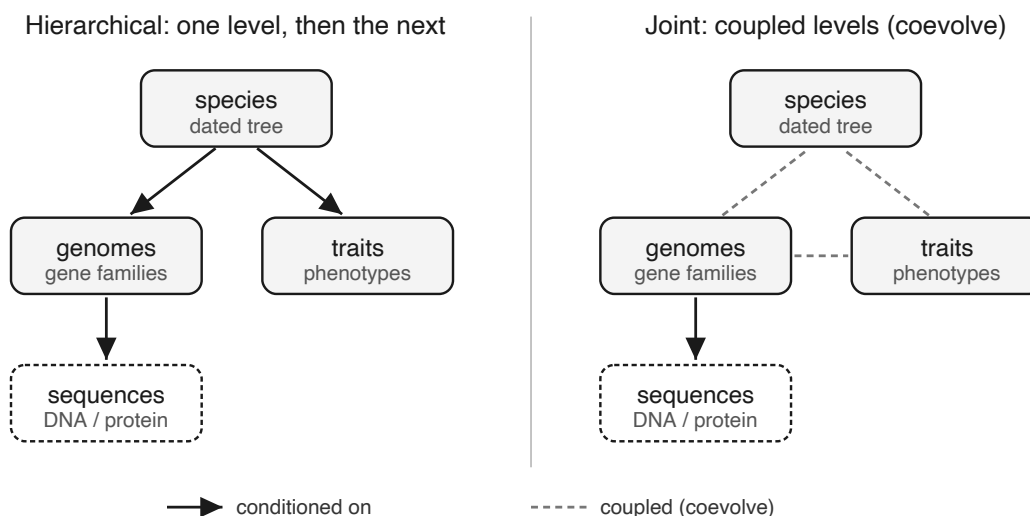


Figure 1.1. The four levels of evolution and the two ways ZOMBI2 composes them. In the hierarchical pipeline (left), each level is simulated conditioned on the one above; in a joint *coevolve* run (right), species, genomes and traits are mutually coupled. Sequences are a downstream layer in both. You always choose which levels to simulate — you need not run them all.

The usual way is **hierarchical**: simulate one level, then the next *conditioned on* it — a species tree, then gene families along its branches, then sequences along the resulting gene trees. Each level treats the one above as a fixed backbone. This keeps the levels independent and easy to reason about, and lets a single species tree seed many genome, trait, or sequence runs, so one backbone underlies a whole benchmark. It is the design ZOMBI has always had: species, then genomes, then sequences, in that order.

Sometimes the levels are *not* independent — a trait may change the rate at which genes are gained or lost; gene content may decide which lineages survive and diversify. When influence runs *between* levels, a one-directional pipeline is no longer enough and the levels must be simulated **jointly**. ZOMBI2’s *coevolve* mode does this, coupling species, genomes, and traits along directed links you choose. Sequences remain a downstream layer for now, though one can imagine them feeding back on the tree too.

1.2 For the impatient

A minimal run is just two commands: build a species tree, then evolve gene families along it.

```
# 1. a species tree: 20 extant species from a birth-death process
zombi2 species --tips 20 --age 5 --birth 1.0 --death 0.3 --seed 1 -o my_tree

# 2. gene families evolving along it (duplication, transfer, loss, origination)
zombi2 genomes -t my_tree/species_tree.nwk \
  --dup 0.2 --trans 0.1 --loss 0.25 --orig 0.5 --seed 42 -o my_genomes
```

The first writes `my_tree/species_tree.nwk`; the second writes gene trees and the presence/absence **profile matrix** (gene families × species) into `my_genomes/`. The same run from Python, where

`initial_families` sets how many gene families seed the root genome:

```
import zombi2 as z

tree = z.simulate_species_tree(z.BirthDeath(1.0, 0.3), n_tips=20, age=5.0, seed=1)
genomes = z.simulate_genomes(tree, duplication=0.2, transfer=0.1, loss=0.25,
                             origination=0.5, initial_families=20, seed=42)

genomes.profiles.matrix      # gene families × extant species (copy numbers)
genomes.write("out/")       # trees, event tables, transfers, profiles
```

1.3 How this manual is organised

The manual is a **concepts-and-tutorial** companion to the software. Part I gets you installed and running end to end. Parts II–VI then cover the models in the order you meet them in a simulation: species trees, gene families, traits, coevolution, and sequence evolution — each with worked examples and figures. The exhaustive command-line and Python API reference lives in the online documentation rather than here, so this book stays readable front to back.

Chapter 2

Installation

ZOMBI2 needs Python 3.10 or newer. Its only runtime dependency is **numpy**; the built-in gene-family engine additionally requires a compiled Rust extension, built at install time (see below).

2.1 From source

Clone the repository and install the package in editable mode:

```
git clone https://github.com/AADavin/zombi2.git
cd zombi2
pip install -e . maturin
```

The editable install exposes the `zombi2` command-line tool and the Python package.

2.2 Building the compiled engine

ZOMBI2's gene-family simulator is backed by a Rust engine, compiled to a native Python extension with `maturin`. From the repository root:

```
cd rust && maturin build --release -i python3 && pip install --force-reinstall target/wheels/*
```

This compiles the engine in release mode and installs the resulting wheel over the editable package, wiring the compiled backend into `zombi2`.

2.3 Development extras

To run the test suite and the statistical checks, install the optional `dev` dependencies:

```
pip install -e ".[dev]" # adds pytest and scipy
pytest
```

2.4 Optional dependencies

- **scipy** — needed only when passing a `scipy.stats` frozen distribution to `FamilySampledRates`; the built-in distributions (`Gamma`, `Exponential`, and so on) require nothing beyond `numpy`.
- **matplotlib** / **pandas** — used only by the demo notebook, not by the library itself.

2.5 Verifying the installation

Confirm that the command-line tool is on your path:


```
zombi2 --help
```

This lists the available subcommands (`species`, `genomes`, `trait`, and others). Then confirm the Python package imports cleanly:

```
import zombi2
```

If both succeed, the core installation is complete. To exercise the compiled gene-family engine end to end, simulate a small species tree and evolve genomes along it:

```
zombi2 species --birth 1 --death 0.3 --tips 50 --age 5 --seed 1 -o out/  
zombi2 genomes --tree out/species_tree.nwk \  
    --dup 0.2 --trans 0.1 --loss 0.25 --orig 0.5 --seed 42 -o out/
```

A successful `genomes` run confirms that the compiled backend is correctly installed.

Chapter 3

A first simulation

This chapter walks you through your first complete ZOMBI2 run, start to finish. You will build a species tree, evolve gene families along it, look at everything the simulator writes to disk, and finally load two of those results back into Python. By the end you will have a full simulated dataset in a folder called `out/` and a clear picture of what each file means.

This is the simplest use of ZOMBI2: a *hierarchical* run over the first two levels from the previous chapter — a species tree, then gene families evolved along it. Each stage is seeded, so the same `seed` always gives byte-identical results, and your run will match the numbers you see here.

3.1 Simulating a species tree

Start with the species tree. We condition a birth–death process on 20 extant tips and a crown age of 5.

```
import zombi2 as z

tree = z.simulate_species_tree(z.BirthDeath(birth=1.0, death=0.3),
                               n_tips=20, age=5.0, seed=1)

print(tree.to_newick())    # timed Newick
```

The same run from the command line writes the tree straight to a folder:

```
zombi2 species --birth 1 --death 0.3 --tips 20 --age 5 --seed 1 -o out/
```

The result is a timed tree with 20 surviving tips. `BirthDeath` is the model; `birth` and `death` are the per-lineage speciation and extinction rates.



Figure 3.1. A simulated species tree with the duplication, loss and transfer events of one gene family marked on its branches.

3.2 Evolving gene families along the tree

Now evolve gene families forward along that tree. Each family starts as a single copy at its point of origination and then duplicates, transfers between branches, and is lost, at the rates you supply.

```
genomes = z.simulate_genomes(tree, duplication=0.2, transfer=0.1, loss=0.25,
                             origination=0.5, initial_families=40, seed=42)

print(genomes.profiles.matrix.shape)    # (n_families, n_species) copy numbers
```

Equivalently, from the command line, using the tree you just wrote:

```
zombi2 genomes --tree out/species_tree.nwk --dup 0.2 --trans 0.1 \
             --loss 0.25 --orig 0.5 -o out/
```

Here `initial_families=40` seeds the root genome with 40 families; `origination=0.5` is the rate at which new families appear along branches. The four core rates — duplication, transfer, loss, origination — drive the whole forward process. The built-in model runs on the Rust engine automatically.

3.3 The output folder

Write everything to disk with a single call:

```
genomes.write("out/")
```

The command line produces the same layout under `-o out/`. Here is what you will find:

File / folder	Contents
<code>species_tree.nwk</code>	the timed species tree (Newick)
<code>species_nodes.tsv</code>	node name, time, <code>is_leaf</code> , <code>is_extant</code>
<code>gene_family_events/<fid>_events.tsv</code>	per-family event log (O/D/T/S/L with lineage ids)
<code>gene_trees/<fid>_complete.nwk,</code> <code>_extant.nwk</code>	reconstructed gene trees (with / without losses)
<code>Transfers.tsv</code>	one row per transfer: time, family, donor, recipient, ids
<code>Gene_family_summary.tsv</code>	per family: origin, event counts, extant copies, species present
<code>Profiles.tsv, Presence.tsv</code>	families $\times$ species copy-number and presence matrices

Two things are worth understanding before you open these files.

The **event log** is a full genealogy. Every event re-mints the gene-lineage ids it touches, and speciations are logged too, so each family's `_events.tsv` records a complete parent-to-children history. Reconstructing a gene tree from it is then pure post-processing.

The **gene trees** come in two flavours. The *complete* tree keeps every lineage, including losses (leaves labelled `LOSS_<id>`). The *extant* tree is pruned to lineages with a surviving copy, with unifurcations suppressed; its leaves are labelled `<species>_<geneid>`. The number of leaves in an extant tree equals the family's total copy count across all species.

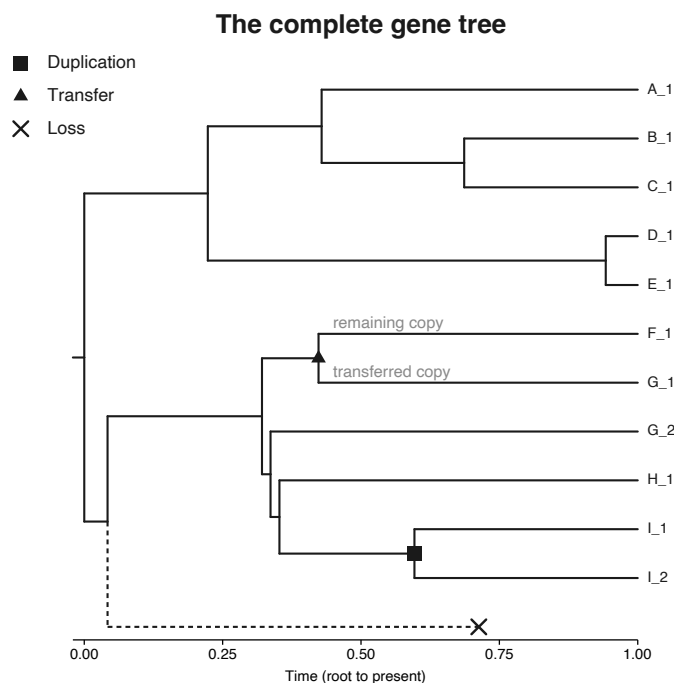


Figure 3.2. A gene family’s *complete* gene tree: every copy implied by its duplication, transfer, and loss events — including a copy that was later lost (the dashed lineage ending in a cross). The *extant* tree is this tree pruned to the surviving copies.

3.4 Reading the results back in Python

3.4.1 A gene tree

`gene_trees()` returns, for each family id, a pair of Newick strings — complete and extant.

```
trees = genomes.gene_trees()           # {fid: (complete, extant) newick}
complete, extant = trees["7"]
```

`extant` is `None` when a family has no survivors. Otherwise it is the tree you would reconcile against the species tree, implied by that family’s duplication, transfer and loss events.

3.4.2 The profile matrix

The profile matrix is the key object for phylogenetic-profiling analyses: gene families as rows, extant species as columns.

```
P = genomes.profiles
P.matrix           # integer copy numbers
P.presence()      # binary presence / absence
P.families        # row labels
P.species         # column labels
P.to_tsv()        # tab-separated text (as written to Profiles.tsv)
```

`P.matrix` is exactly what was written to `Profiles.tsv`, and `P.presence()` matches `Presence.tsv`.

Each entry counts how many copies of a family survive in a species.

If you want the raw events instead of reconstructed trees, iterate the event log directly:

```
from zombi2.events import EventType

for r in genomes.event_log:
    if r.event is EventType.TRANSFER:
        print(r.time, r.donor, "->", r.recipient, [op.gid for op in r.genes])
```

Each `EventRecord` carries a `time`, an `event` (an `EventType`), a `branch`, optional `donor` and `recipient`, and `genes` — the per-gene rows, the first being the source lineage and the rest its descendants.

Note

`Transfers.tsv` gives you the same transfer events in flat tabular form, one row each, if you would rather not walk the log in code.

You now have a complete simulated dataset: a timed species tree, gene families evolved along it, reconstructed gene trees, and the profile matrix — all reproducible from a pair of seeds. The following chapters unpack each stage: species-tree models and conditioning, shared versus per-family rates, transfers, and bounding genome growth — before the later parts add the remaining levels (traits and sequences) and the joint coevolution models.

Chapter 4

Species trees (basic models)

Every ZOMBI2 simulation has a species tree: the branching history of the species onto which gene families, traits, and sequences are later layered. This chapter covers the constant-rate birth–death process and its pure-birth (Yule) special case, how to condition a simulation on the number of tips and on the tree’s age, the two ways ZOMBI2 can generate a tree — backward (the reconstructed process) and forward (the complete process, keeping extinct lineages) — and the `Tree` object the simulator returns.

4.1 The birth–death model

Under the constant-rate birth–death process, each lineage independently speciates at rate λ (birth) and goes extinct at rate μ (death). A pure-birth process, in which lineages never die ($\mu = 0$), is the **Yule** model. Because extinction removes lineages, a tree grown forward in time contains two kinds of tips: *extant* leaves that survive to the present, and *extinct* leaves that died along the way. The tree obtained by pruning away the extinct lineages — keeping only the branches ancestral to the survivors — is the **reconstructed tree** (Nee et al. 1994). It is what one recovers from present-day data, and it is what ZOMBI2 produces by default.

The reconstructed birth–death tree is a *coalescent point process*: conditional on the number of extant tips and the tree age, its internal-node ages are independent and identically distributed draws from a known distribution (Hartmann et al. 2010). ZOMBI2 exploits this directly. It draws the internal-node ages i.i.d. from the reconstructed-process CDF and assembles a ranked tree by uniform coalescence (Hartmann et al. 2010). No lineages are simulated and discarded, so producing a tree with a prescribed number of tips is exact and fast, however small the survival probability.

The two models are constructed by naming their rates:

```
from zombi2.species import BirthDeath, Yule, simulate_species_tree

BirthDeath(birth=1.0, death=0.3)    # speciation lambda, extinction mu
Yule(birth=1.0)                     # pure birth == BirthDeath(birth, death=0)
```

4.2 Simulating a tree

`simulate_species_tree` takes a model and the conditioning that pins down its size and timescale:

```
tree = simulate_species_tree(
    BirthDeath(1.0, 0.3),
    n_tips=20,           # condition on the number of extant species (>= 2)
    age=5.0,             # tree age
    age_type="crown",    # "crown": age of the root; "stem": time of origin
```

```
seed=1,          # or rng=<numpy Generator>
)
```

The two knobs fix the two quantities the reconstructed process needs:

- **n_tips** — the tree has exactly this many extant leaves. The conditioning is exact, not the result of rejection sampling, so it is cheap even when extinction is high.
- **age** and **age_type** — with **age_type**="crown" the root sits at time 0 and every extant leaf sits at **age**; the tree is ultrametric. With **age_type**="stem" the age is instead the *origin* time, and a stem branch precedes the crown (the first speciation) ([Stadler 2009](#)).

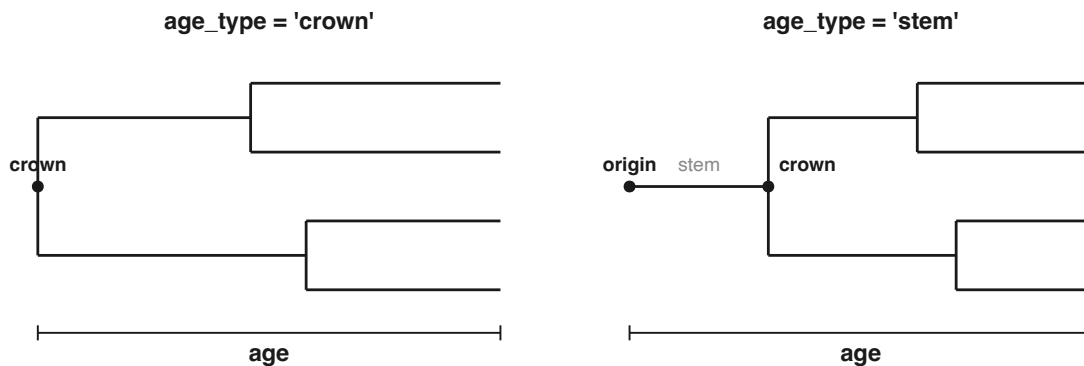


Figure 4.1. What **age** measures. With **age_type**="crown" (left) the age is the depth from the crown — the root of the reconstructed tree — to the present. With **age_type**="stem" (right) it is measured from the origin instead, so a stem branch precedes the crown and the crown subtree is correspondingly shorter.

4.3 Backward versus forward simulation

By default ZOMBI2 simulates the *reconstructed* tree backward in time, sampling node ages from the coalescent point process as described above. This is the right object when you want a tree that looks like an inferred phylogeny of extant species: it contains no extinct lineages, because none survive to be observed.

Sometimes, though, the extinct lineages matter. Gene families evolving on the tree can be transferred *from* a since-extinct donor, so the dead branches leave a genomic signature even though they contribute no tips. For this ZOMBI2 can instead grow the **complete** tree forward in time, retaining every lineage — extant and extinct alike:

```
# grow the full tree forward, keeping extinct lineages
tree = simulate_species_tree(
    BirthDeath(1.0, 0.3),
    age=5.0,
    direction="forward",
    seed=1,
)
```

A forward run realizes the process by an exact event-by-event (Gillespie) loop: starting from the root it draws waiting times to the next birth or death and applies them until it reaches **age**. The result keeps the extinct leaves — conventionally named **e*** — so a lineage that died before the

present is still a tip of the complete tree. The run is conditioned to leave at least two survivors; a realization in which every lineage dies is rejected and redrawn.

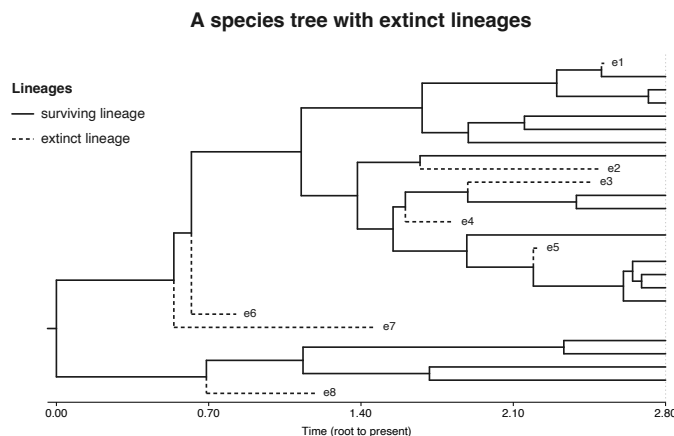


Figure 4.2. A complete forward tree that keeps its extinct lineages, drawn dashed and named **e1**, **e2**, ...

The complete-versus-reconstructed distinction is the same one drawn analytically for these processes: the reconstructed tree is the complete tree with its extinct subtrees pruned away (Nee et al. 1994; Lambert and Stadler 2013).

4.4 The Tree object

Both modes return the same **Tree** object. Its nodes carry times, and it exposes the traversals the rest of ZOMBI2 needs:

```
tree.to_newick()           # timed Newick (branch lengths from node times)
tree.leaves()              # extant leaves
tree.internal_nodes()
tree.branches_alive_at(t) # lineages crossing time t (used by the gene-family loop)
tree.total_age
```

Node times increase forward from the root at time 0 to the extant leaves at **total_age**. A branch is identified by its child node and spans the half-open interval (**parent.time**, **node.time**]. In a backward (reconstructed) tree every leaf sits at **total_age** and the tree is ultrametric; in a forward tree the extinct leaves sit at earlier times, wherever their lineages died.

The **branches_alive_at(t)** method returns the set of lineages crossing a given time, which is exactly what the forward gene-family simulator iterates over as it walks the species tree. Because the timed Newick encodes node times as branch lengths, the tree round-trips through standard phylogenetics tooling.

Beyond the constant-rate birth–death and Yule models, ZOMBI2 offers a family of richer diversification models — episodic (skyline) rates, instantaneous mass extinctions, per-lineage ClaDS rates, diversity-dependent diversification, and scheduled clade-specific shifts. Those are the subject of the next chapter.

Chapter 5

Species trees (advanced models)

ZOMBI2’s default species-tree sampler runs **backward in time**: it draws internal-node ages i.i.d. from the reconstructed birth–death CDF and assembles a ranked ultrametric tree by uniform coalescence. That machinery keeps the constant-rate `BirthDeath` and `Yule` models cheap, and it extends cleanly to piecewise-constant rates. But a whole family of macroevolutionary models cannot be written as “draw i.i.d. node ages”: rates that depend on the standing diversity, rates that drift lineage-by-lineage, scheduled shifts in one clade, instantaneous kills, and serially sampled fossils all need the process to be simulated **forward in time**, retaining extinct lineages. This chapter covers the diversification models ZOMBI2 ships today, flagging which run backward and which are forward-only.

Every forward model is invoked through the same entry point, differing only in the model object passed and the conditioning:

```
from zombi2.species import (
    simulate_species_tree, BirthDeath, EpisodicBirthDeath,
    ClaDS, DiversityDependent, CladeShiftBirthDeath, prune,
)

tree = simulate_species_tree(
    model,                # a diversification model object
    age=5.0,              # crown age (a fixed present); or n_tips=...
    direction="forward",  # forward growth, extinct lineages retained
    seed=1,
)
```

A forward run returns the **complete** tree (extinct leaves carry `is_extant=False` at their death times). `prune(tree)` extracts the survivors-only reconstructed counterpart. Any complete tree feeds `simulate_genomes` unchanged, and extinct lineages become ghost transfer partners automatically.

Note

The forward growth loop is exact: because every model below has rates that are constant *between* events, the next event time is drawn from the summed rate by an exact-Gillespie step — no thinning bound needed. Mass-extinction pulses and incomplete sampling layer on top of any of them.

5.1 Episodic (skyline) birth–death

`EpisodicBirthDeath` lets speciation and extinction be **piecewise-constant through time** — the model behind shifting diversification regimes and *gradual* extinction pulses (Stadler et al. 2013;

Höhna et al. 2016). Rates are given one value per epoch, ordered from the present backward, with the epoch boundaries supplied as strictly increasing **ages** before the present:

```
# a gradual extinction spike: normal extinction recently, high extinction older than age 1
epi = EpisodicBirthDeath(birth=[1.0, 1.0], death=[0.2, 3.0], shifts=[1.0])
tree = simulate_species_tree(epi, n_tips=30, age=4.0, seed=1)
```

`birth[i]` and `death[i]` apply to epoch *i*; `shifts` has one fewer entry than `birth` (the boundaries between epochs). A single epoch (`shifts=[]`) reproduces the constant-rate BirthDeath. Because the reconstructed tree is still a coalescent point process, ZOMBI2 samples i.i.d. node ages from the numerically inverted piecewise CDF and assembles exactly as for the constant-rate case — the tree stays ultrametric. This is the one time-varying model that still runs through the **backward** sampler; it also runs forward in age mode.

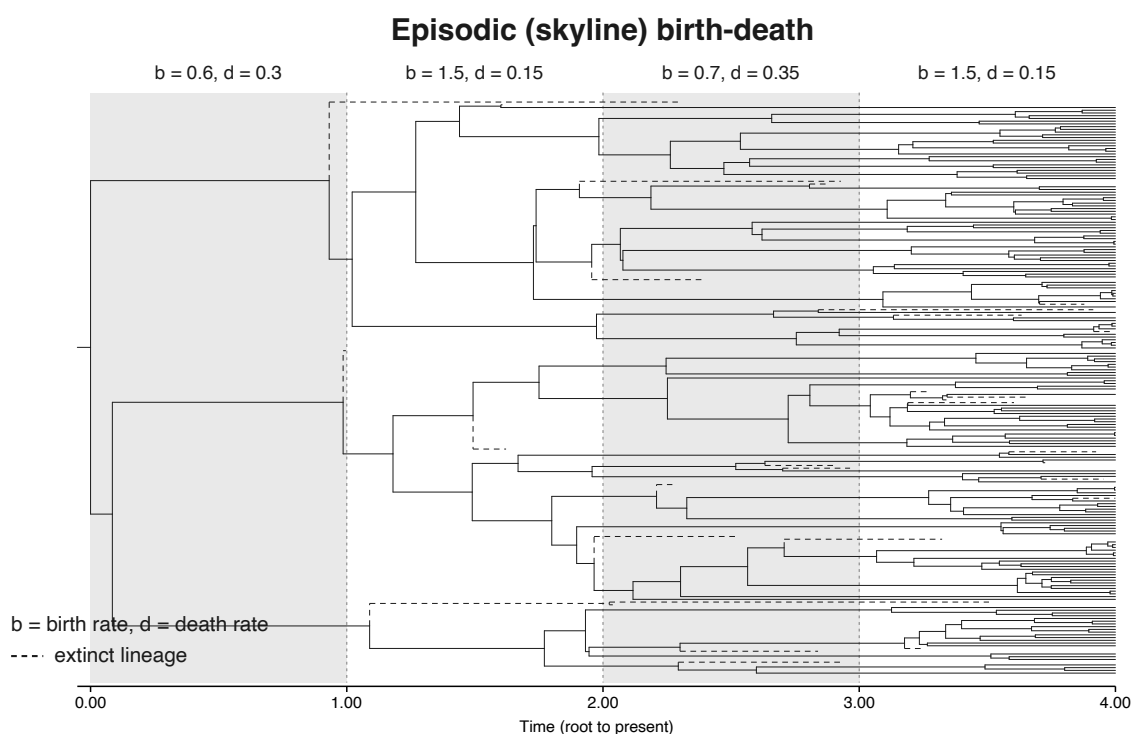


Figure 5.1. Piecewise-constant speciation and extinction rates across skyline epochs, with boundaries at fixed ages before the present.

5.1.1 Incomplete extant sampling

Real phylogenies rarely include every living species. Pass `sampling_fraction`, the probability ρ that an extant species is sampled:

```
EpisodicBirthDeath(birth=[1.0], death=[0.3], shifts=[], sampling_fraction=0.25)
```

In a forward run, unsampled extant tips are marked `is_extant=False`. Incomplete sampling keeps the tree ultrametric, and is accepted as an overlay by every forward diversification model in this chapter.

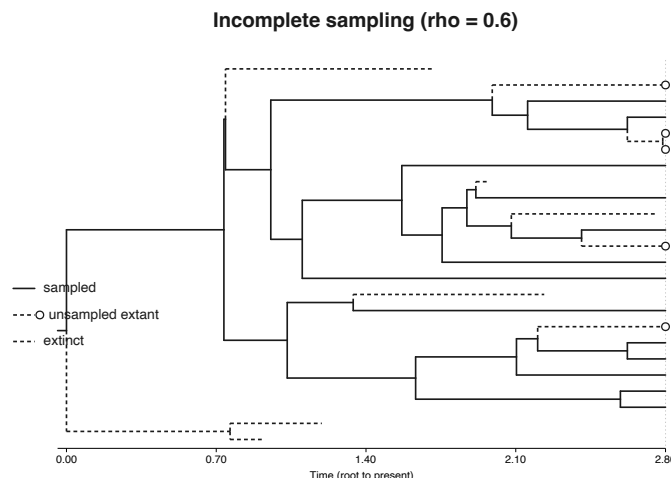


Figure 5.2. Incomplete extant sampling: only a fraction of the living tips (filled) are retained; the rest (open) are dropped.

Warning

Serial sampling *through time* — dated fossil tips, as in the fossilized birth–death process below — is a different mechanism from incomplete extant sampling. It requires forward simulation with retained extinct lineages, and is not available through the backward sampler.

5.2 Mass extinctions

Raising `death` over an epoch spreads extra extinction *smoothly* through a time window. A **mass extinction** in the palaeobiological sense is instead an *instantaneous, tree-wide pulse*: at a single instant a large fraction of the standing diversity is wiped out at once. Give any forward model a `mass_extinctions` list of (age, fraction) pulses:

```
# a radiation punctuated by two cataclysms (75% then 50% die), grown forward:
m = BirthDeath(1.0, 0.3, mass_extinctions=[(1.0, 0.75), (2.5, 0.5)])
tree = simulate_species_tree(m, age=5.0, direction="forward", seed=1)

# pulses compose with an episodic background, too:
m = EpisodicBirthDeath(birth=[1.0, 1.4], death=[0.2, 0.3], shifts=[2.0],
                      mass_extinctions=[(1.0, 0.8)])
```

At each `age` before the present, every lineage then alive **independently** dies with probability `fraction` (survives with `1 - fraction`) — the standard survival-pulse formulation of TreeSim and TESS (Stadler 2011; Höhna et al. 2016). Killing is per-lineage Bernoulli, so the realized fraction fluctuates around `fraction`. The victims become ordinary extinct leaves at the pulse instant, so `simulate_genomes` treats them as ghost transfer partners — a mass extinction leaves a *genomic* signature (families lost with the dead clades, transfers *from* the dead) with no extra wiring.

Mass extinctions are a **forward** feature and need **age mode** (their times are ages before a fixed present); the backward reconstructed sampler and `n_tips` mode reject them. A pulse of

`fraction=1.0` wipes the whole tree out, and the requirement of at least two surviving lineages then rejects the run.

```
zombi2 species --mode forward --age 5 \
  --mass-extinction 1.0 0.75 --mass-extinction 2.5 0.5 -o out/
```

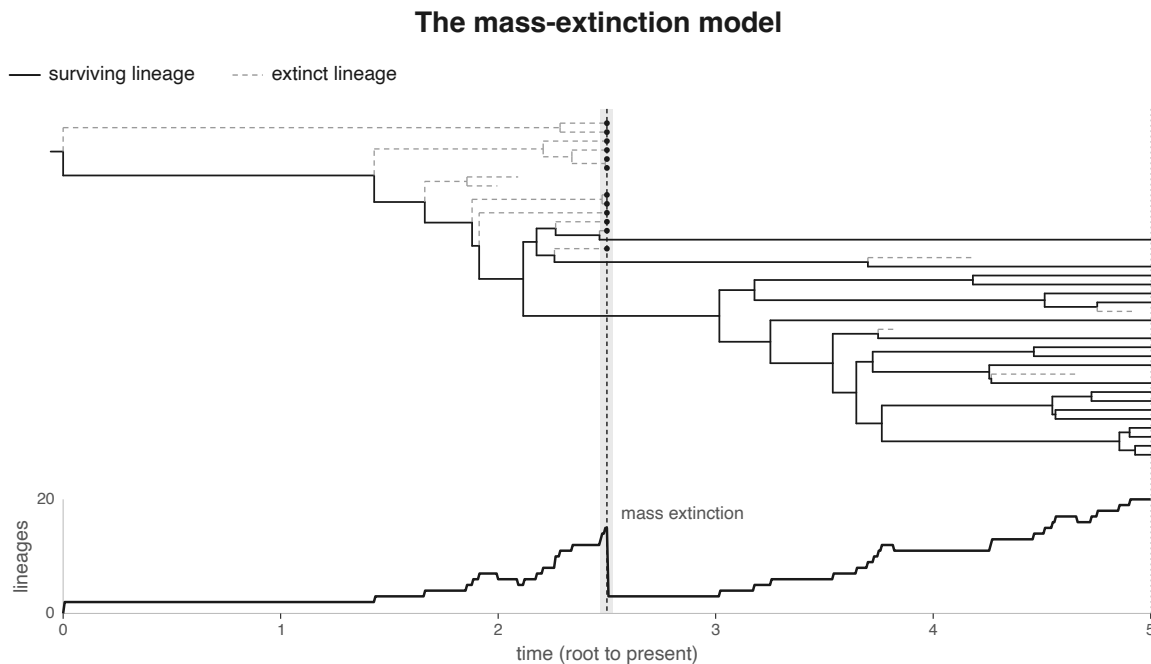


Figure 5.3. A mass extinction: at one instant a fraction of the standing diversity is pruned, driving a sharp drop in the lineages-through-time curve.

5.3 Per-lineage rates

In the models so far, every lineage shares the same rates at any instant. **ClaDS** (Maliot et al. 2019) instead gives each lineage its *own* speciation rate: at each speciation the two daughters inherit the parent’s rate times an independent lognormal jump,

$$\lambda_{\text{child}} = \lambda_{\text{parent}} \cdot \exp(\mathcal{N}(\log \alpha, \sigma^2)),$$

so rates drift lineage-by-lineage down the tree. It is the diversification counterpart of a relaxed molecular clock, and captures the heavy among-clade rate variation real phylogenies show.

```
# alpha<1 = speciation slows toward the present; sigma = jump spread; turnover = mu/lambda
m = ClaDS(lambda_0=1.0, alpha=0.9, sigma=0.2, turnover=0.1)
tree = simulate_species_tree(m, age=5.0, direction="forward", seed=1) # or n_tips=...
```

`alpha` is the trend (`alpha<1` reproduces the empirically typical slow-down of speciation toward the present), `sigma` the jump spread, and extinction is set by a constant turnover $\varepsilon = \mu/\lambda$. `turnover=0` is ClaDS0 (pure birth with shifts); `turnover>0` adds proportional extinction. ClaDS is **forward-only** — per-lineage rates have no closed-form reconstructed CDF — and runs in either `age` or `n_tips` mode. It accepts `sampling_fraction` and `mass_extinctions` overlays like BirthDeath.

```
zombi2 species --mode forward --diversification clads --birth 1.0 --age 5 -o out/
```

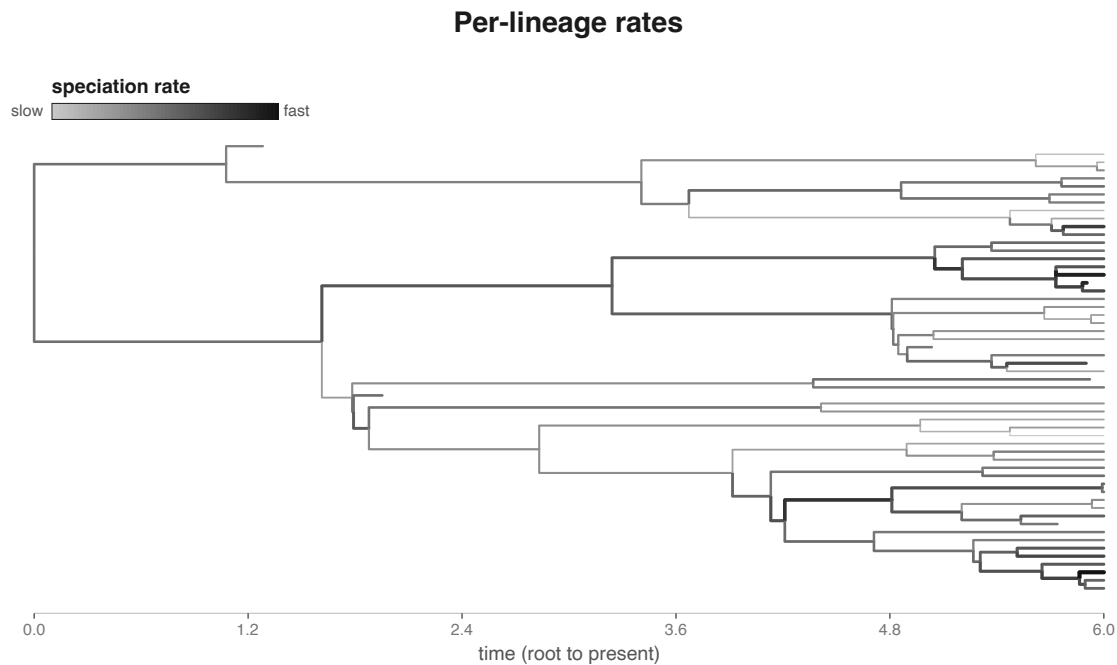


Figure 5.4. A ClaDS tree, branches shaded by their per-lineage speciation rate: some clades radiate, others stall.

5.4 Diversity-dependent diversification

DiversityDependent (Rabosky and Lovette 2008; Etienne et al. 2012) makes speciation slow as the tree fills an ecological carrying capacity K ,

$$\lambda(n) = \max(0, \lambda_0 (1 - n/K)),$$

with constant extinction μ . The tree radiates fast when small and saturates near K — a diversity brake, and the macroevolutionary analogue of the per-family carrying_capacity ZOMBI2 already offers for genes.

```
m = DiversityDependent(lambda_0=2.0, death=0.2, carrying_capacity=50)
tree = simulate_species_tree(m, age=15.0, direction="forward", seed=1) # or n_tips <= K
```

With $\mu = 0$ the tree saturates at exactly K ; with $\mu > 0$ it settles near the equilibrium $n^* = K(1 - \mu/\lambda_0)$. The model is **forward-only** but supports both `age` and `n_tips` mode — in `n_tips` mode the target must be $\leq K$. It accepts `sampling_fraction` and `mass_extinctions` overlays.

```
zombi2 species --mode forward --diversification diversity-dependent \
  --birth 2 --death 0.2 -K 50 --age 15 -o out/
```

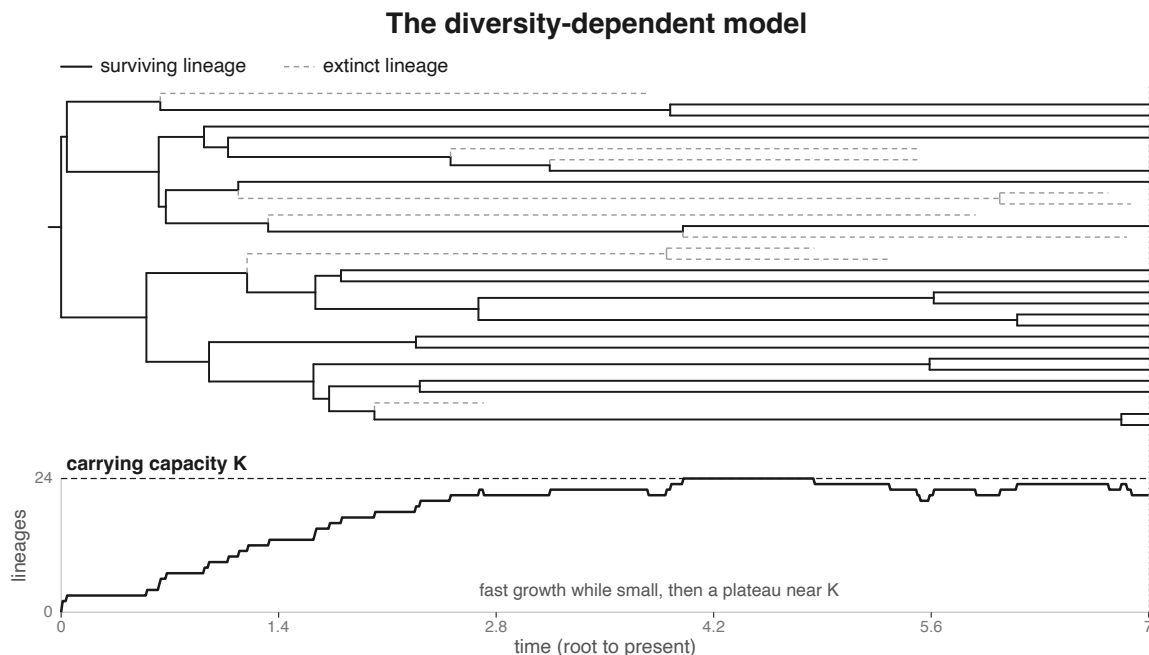


Figure 5.5. Diversity-dependent diversification: a fast early radiation flattening into a plateau as the lineage count approaches the carrying capacity K .

5.5 Clade-specific rate shifts

Where ClaDS shifts *every* lineage a little at *every* speciation, `CladeShiftBirthDeath` shifts *one* clade a lot at a *scheduled* time — the discrete, hand-specified version of rate heterogeneity. The tree runs at the background (`birth`, `death`) until, at each scheduled age before the present, a uniformly chosen lineage then alive (and all its descendants) adopts a new (`birth`, `death`) regime. It is the model for “a key innovation sparks a radiation in one clade.”

```
# a slow background; at age 3 one clade starts diversifying fast
m = CladeShiftBirthDeath(0.6, 0.4, clade_shifts=[(3.0, 2.0, 0.1)])
tree = simulate_species_tree(m, age=5.0, direction="forward", seed=1)
```

The shifted lineage is drawn at random: you cannot name an unborn clade in a forward run, and contemporaneous lineages are exchangeable. Supply several (`age`, `birth`, `death`) shifts for several clades. This model is **forward-only** and **age mode only** (the shifts are scheduled as ages before a fixed present); it accepts `sampling_fraction` and `mass_extinctions` overlays.

```
zombi2 species --mode forward --age 5 --birth 0.6 --death 0.4 \
  --clade-shift 3.0 2.0 0.1 -o out/
```

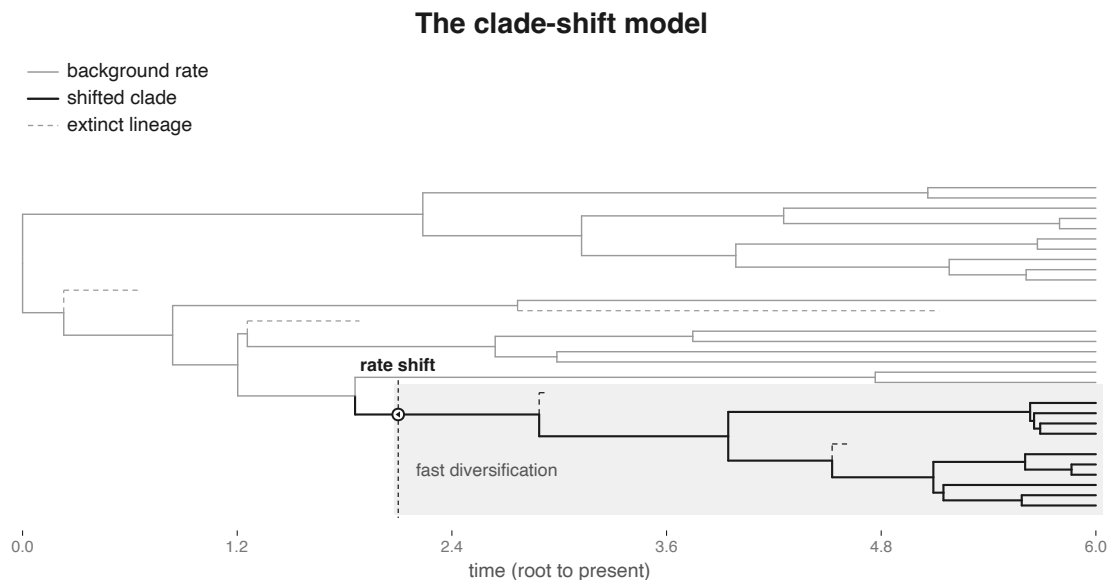


Figure 5.6. A clade-specific shift: at a scheduled age one lineage and all its descendants adopt a new speciation/extinction regime, sparking a fast-diversifying clade.

5.6 Fossilized birth–death

The same BirthDeath (or EpisodicBirthDeath) model gains **serial sampling through time** via optional keywords (Gavryushkina et al. 2014; Heath et al. 2014). Beyond speciation (λ) and extinction (μ), lineages are sampled through time at rate ψ (**fossilization**) — each a **dated fossil tip** — and extant lineages are sampled at the present with probability ρ (**sampling_fraction**). Because fossils are sampled points on extinct or surviving lineages, this is a **forward** feature that requires retaining the extinct lineages.

```
m = BirthDeath(birth=1.0, death=0.5, fossilization=0.5, sampling_fraction=0.9)
# the complete tree, with dated fossils along the branches
tree = simulate_species_tree(m, age=6.0, direction="forward", seed=1)
```

Fossil tips carry `sampled=True`, `is_extant=False` at their (past) sampling times; sampled extant tips carry `sampled=True`, `is_extant=True`. With `removal` set below 1 (`--removal r`), a sampled lineage *continues* with probability $1 - r$ instead of being removed — a **sampled ancestor**, represented as a degree-two node that the gene simulator passes genomes straight through, so DTL simulation runs unchanged on such trees.

```
zombi2 species --mode forward --age 6 --fossilization 0.5 \
  --sampling-fraction 0.9 --removal 0.5 -o out/
```

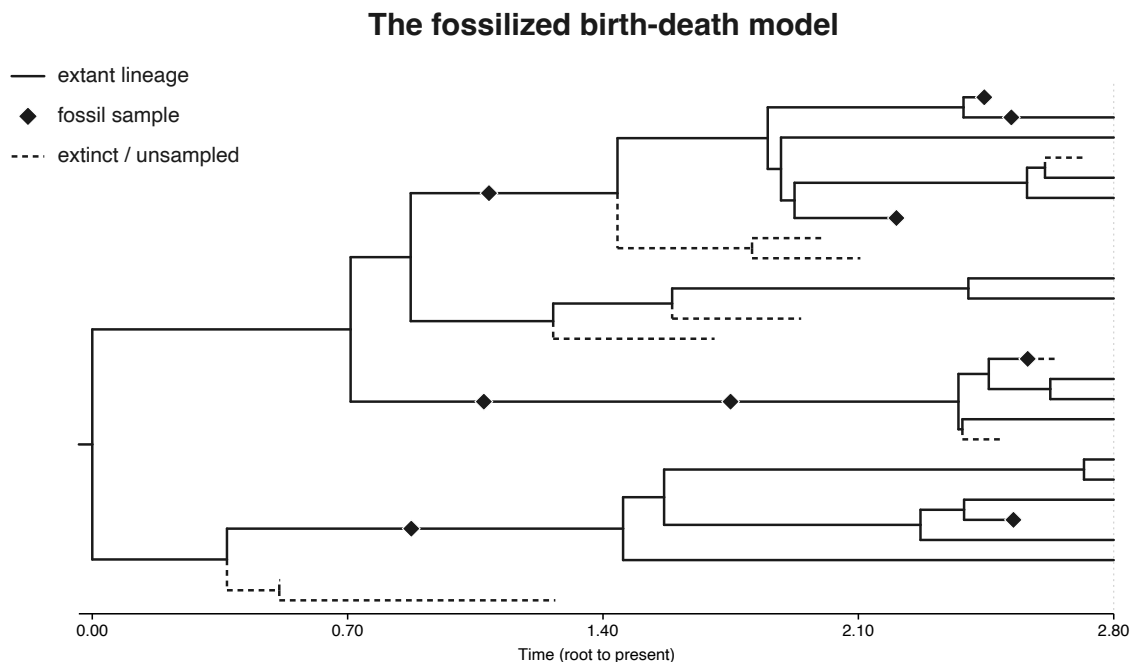



Figure 5.7. Fossilized birth–death: dated fossil tips (sampled through time along extinct and surviving lineages, as diamonds) alongside sampled extant tips.

Warning

The occurrence birth–death process ([Andréoletti et al. 2022](#)), which mixes fossil, occurrence, and extant data types, builds on this same forward machinery but is not yet shipped.

Note

Every time-varying diversification model shares an identifiability caveat: distinct histories of $\lambda(t)$ and $\mu(t)$ can produce the same distribution of reconstructed (extant-only) trees ([Louca and Pennell 2020](#)). Serial sampling — fossils — is one way to break that degeneracy, since it constrains the extinct part of the process directly.

Chapter 6

Ghost lineages

A reconstructed species tree contains only the sampled, extant lineages — that is the whole point of the backward, conditioned birth–death process behind `simulate_species_tree`. But the real diversification process also produced lineages that went extinct before the present, or that are alive today yet were never sampled. These *dead* lineages are invisible in the reconstructed tree, and dropping them is not harmless. A gene can be laterally transferred from a lineage that later vanished — a transfer from the dead (Szöllősi et al. 2013). If the simulated tree carries only survivors, every transfer is forced to run between sampled, co-existing lineages, which biases any benchmark of transfer inference.

`add_ghost_lineages` puts the dead lineages back. It *un-prunes* the reconstructed tree: it grafts the extinct and unsampled (“ghost”) lineages back on, so the tree again reflects the full process rather than only its survivors. The result is an ordinary `Tree`; the extra tips are just leaves marked `is_extant=False`.

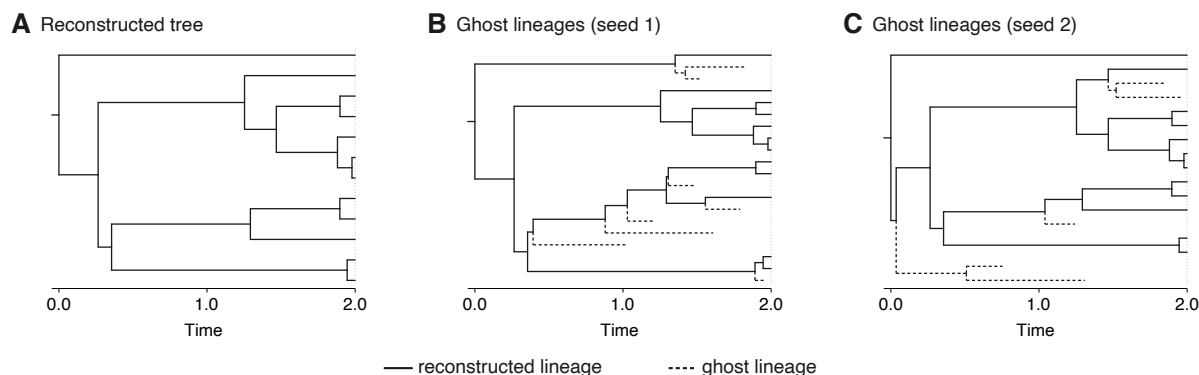


Figure 6.1. The reconstructed tree of survivors (solid) with the extinct and unsampled ghost lineages grafted back on (dashed), recovering the full diversification history.

6.1 Un-pruning versus simulating forward

There are two ways to obtain a tree that includes dead lineages, and they serve different starting points.

Note

Un-pruning is for when you already have — or want — a reconstructed tree conditioned on N extant tips, and need the dead lineages added back. If instead you want the complete tree from the start, `simulate_species_tree(..., direction="forward")` grows it forward with extinct lineages included natively, and no un-pruning is needed. The two are distributionally

equivalent for the reconstructed part; pick by which tree you have in hand.

6.2 Basic use

Pass the *same model* you used to build the tree, so the ghost process runs at the matching rates. The function mutates the tree in place and also returns it.

```
model = z.BirthDeath(birth=1.0, death=0.5)
tree = z.simulate_species_tree(model, n_tips=50, age=5.0, seed=1)

z.add_ghost_lineages(tree, model, seed=7)    # grafts ghosts in place
```

Each grafted ghost attaches along an edge and roots a birth–death subtree that leaves no sampled descendant. Every new node is marked `is_extant=False`; the dead leaves are named `e*` and the new internal nodes `i*`, following ZOMBI2’s convention (extant leaves stay `n*`). The sampled leaves are left untouched, so pruning back to the extant tips recovers the original reconstructed tree exactly.

```
extant = [n for n in tree.leaves() if n.is_extant]    # sampled tips (n*)
ghosts = [n for n in tree.leaves() if not n.is_extant] # dead tips (e*)
print(len(extant), "extant +", len(ghosts), "ghost leaves")
```

Ghosts flow straight into the forward gene simulation with no extra step: they are just branches, so speciation gives each a genome and transfers can now draw ghost donors and recipients automatically. Because gene trees still prune to sampled tips, a gene from a ghost donor surfaces as a transfer from a lineage absent from the species tree — exactly the transfer-from-the-dead case.

On the command line, add ghosts with the `--ghosts` flag, which un-prunes the tree before the gene layer runs (`--ghost-method` picks the sampler):

```
zombi2 species --birth 1.0 --death 0.5 --tips 50 --age 5.0 \
  --ghosts --seed 7 -o run/
```

6.3 When do ghosts appear?

Ghosts appear only where lineages could actually have been lost — the process must allow extinction or incomplete sampling.

Model	Ghosts?
Yule / BirthDeath(death=0), full sampling	none — nothing goes extinct
BirthDeath(death>0)	yes — extinct lineages grafted back
EpisodicBirthDeath(..., sampling_fraction<1)	yes — extinct <i>and</i> unsampled lineages

Extinct lineages need $\mu > 0$; unsampled-extant lineages need $\rho < 1$. Both reduce to “leaves no sampled descendant,” so a tree with no possible loss returns from `add_ghost_lineages` unchanged. The supported models are BirthDeath / Yule and EpisodicBirthDeath, the latter with time-varying rates and incomplete sampling.

6.4 The conditional law, briefly

Un-pruning is exact because it uses the known conditional law of the complete birth–death tree given its reconstructed version (Nee et al. 1994; Stadler 2009; Lambert and Stadler 2013). Along each edge of the reconstructed tree, dead lineages attach as an inhomogeneous Poisson process with attachment intensity proportional to $\lambda(t) E(t)$, where $E(t)$ is the probability that a lineage present at time t leaves no sampled descendant at the present.

The intuition is short. A surviving lineage still speciates at rate λ ; each side-branch independently leaves a sampled descendant (probability $1 - E$) or nothing (probability E). The reconstructed tree kept only the branchings where both sides survived — those are its nodes. So *between* its nodes, every branching had a dying sibling, and those attach at rate $\lambda E(t)$. The quantity $E(t)$ is the same survival probability ZOMBI2 already solves for the birth–death process, with the boundary condition $E(0) = 1 - \rho$ — which is why the extinct and unsampled cases fall out of one equation.

Each attachment then roots a subtree grown forward and conditioned on leaving no sampled descendant. Two equivalent samplers are available through `method=`:

- "rejection" (default) — grow an ordinary birth–death subtree from the attachment time and reject any that leaves a survivor. Simple and provably correct; ghosts are small because they are conditioned to die out, so the work stays cheap.
- "htransform" — rejection-free, via Doob's h-transform. The conditioned process is itself a birth–death with per-lineage birth λE and death μ/E , so each subtree is drawn in a single pass. It is statistically equivalent to rejection and faster when heavy extinction or incomplete sampling would otherwise force many retries.

```
z.add_ghost_lineages(tree, model, method="htransform", seed=7)
```

Tip

Pruning the ghosts back off returns the original reconstructed tree unchanged, since ghosts have no sampled descendants. This is an exact augmentation, not an approximation: the sampled-tree statistics — the set and times of the extant tips — are untouched.

Chapter 7

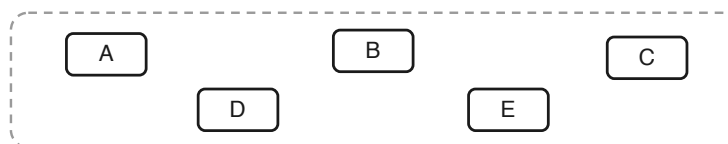
Genome evolution

Once ZOMBI2 has a species tree, it populates the branches with **genomes**. ZOMBI2 offers three models of genome evolution, differing in how much structure a genome carries — from an unordered *set* of gene families to a real nucleotide sequence.

- **Unordered gene families** — a genome is an unordered *set* of gene copies. Families have no position, no neighbours, no length; each evolves on its own. This is the fastest model, and the right one when what you care about is gene content and copy number: presence/absence profiles, reconciliations, family sizes.
- **Ordered gene families** — genes sit on an *ordered chromosome*. Position now matters: rearrangements shuffle neighbourhoods, and a transfer can land beside a syntenic locus. But distance is counted in *genes*, not nucleotides — length stays abstract.
- **Nucleotide genomes** — the chromosome is a real sequence of base pairs. Events have real lengths, genes and intergenes have real coordinates, and structural events act at nucleotide resolution.

Unordered gene families

a set; no order



Ordered gene families

order, not length



Nucleotide genomes

real distances

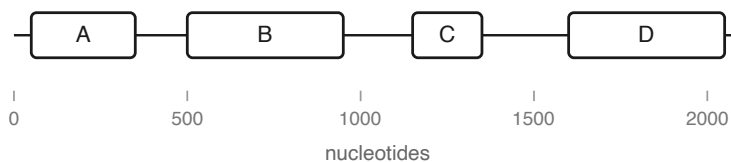


Figure 7.1. The three models of genome evolution, in increasing structure: an unordered set of gene families; genes on an ordered chromosome (order matters, length does not); and a real nucleotide sequence with genes and intergenes at true coordinates.

This chapter is a map of that space. The chapters that follow work through each level in turn; here we lay out the two choices that define any genome run, and — most importantly — which events act at which level.

7.1 Two orthogonal choices

Two independent choices define a genome run.

The first is the **level** — unordered, ordered, or nucleotide. The level is what a genome *is*, and therefore which events can act on it (see *Events, by level* below); each level *adds* events to the one beneath it. On the command line, `--genome-model` selects the level: **unordered** (the default), **ordered**, or **nucleotide**.

The second choice, orthogonal to the first and meaningful only within the unordered level, is how the four gene-family rates *vary across families* — the **rate model** (`--rate-model`; see *Rate models* below).

Level (<code>--genome-model</code>)	A genome is...	Coupling?	Chapter
unordered	a <i>set</i> of gene families (presence/absence)	yes	Chapter 8 (coupling: Chapter 10)
ordered	genes on a chromosome (order matters, length does not)	no	Chapter 11
nucleotide	a real sequence (genes + intergenes at true coordinates)	no	Chapter 12

7.2 Events, by level

A genome model is defined as much by *what can happen to a genome* as by what a genome is. The three levels form a hierarchy: each one inherits every event of the level below and adds its own, acting on the extra structure that level introduces.

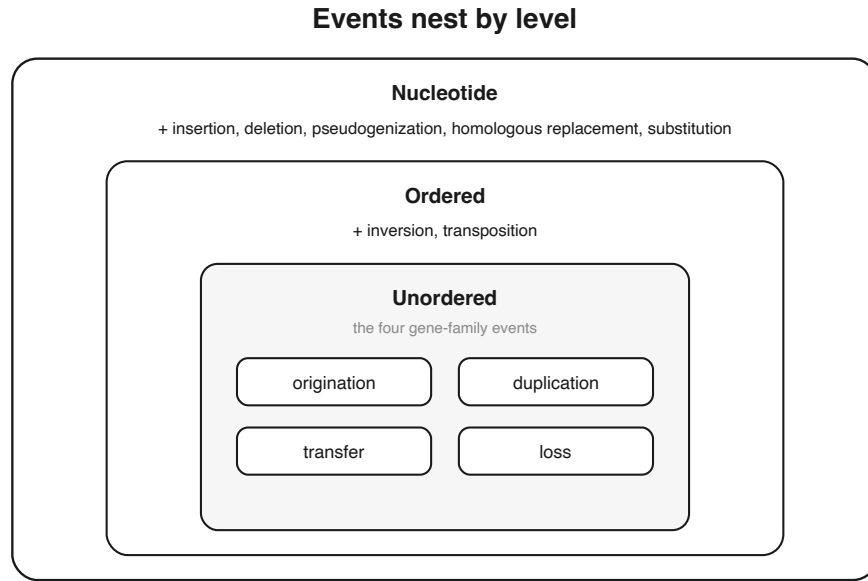


Figure 7.2. The event sets nest — $\text{unordered} \subset \text{ordered} \subset \text{nucleotide}$. The four gene-family events act on any genome; a richer level only ever *adds* events, never removes them: gene order brings the two rearrangements, and real sequence brings the base-pair events.

- **Unordered** — the four gene-family events. **Origination (O)** seeds a brand-new family; **duplication (D)** copies a gene within a genome; **transfer (T)** copies a gene into another lineage alive at the same time; **loss (L)** removes a copy. These four act on gene *content* alone — which families are present, and in how many copies.
- **Ordered** — the four events above (now acting on contiguous *segments* of the chromosome) plus two **rearrangements**: **inversion** reverses a segment and flips its strands, and **transposition** cuts a segment and pastes it elsewhere. These change gene *order*, not content.
- **Nucleotide** — everything above, now at base-pair resolution, plus the events that exist only once a genome has sequence: **intergenic insertion and deletion** (indels that lengthen or shorten intergenic DNA), **pseudogenization** (a loss that demotes a gene to intergenic DNA rather than deleting it), **homologous replacement** (a transfer that overwrites the recipient’s syntenic copy), and **substitution** (point mutation of the sequence itself).

Level	Events it adds	These events act on...
Unordered	origination, duplication, transfer, loss	gene <i>content</i> (which families, how many copies)
Ordered	inversion, transposition	gene <i>order</i> (segments of the chromosome)
Nucleotide	insertion, deletion, pseudogenization, homologous replacement, substitution	the <i>sequence</i> (base pairs; gene/intergene coordinates)

Each level is a strict superset: choosing a richer level never removes an event, it only adds the ones its extra structure makes possible. A nucleotide genome still undergoes duplication, transfer, loss and rearrangement — but now they have real lengths and can split or fuse at any base.

7.3 Rate models

Within the unordered level, a second axis controls **how the four rates vary across gene families**. This is the `--rate-model` flag (a `RateModel` object in Python). Three are available on the command line — **shared** (the default), **per-genome**, and **coupled** (Chapter 10); per-family rates are reached through the Python API.

<code>--rate-model</code>	The rate is...	Family size grows	Object
shared (default)	one per-copy rate, the same for every family	exponentially	SharedRates
per-genome	one per-genome rate, size-independent	linearly	PerGenomeRates
<i>per-family</i>	each family draws its own rates	exponentially, per family	FamilySampledRates
<i>coupled</i>	loss depends on which partner families are present	—	Chapter 10 (PottsRates)

The level and the rate model are independent: any rate model can drive any unordered run, and the choice never affects *which* events fire, only *how often* each family fires them. Chapter 8 works through the four rate models in detail; the coupled model, where families gain and lose non-independently, has its own chapter (Chapter 10).

Chapter 8

Unordered genomes

The **unordered** level is the simplest and fastest of the three genome models (Chapter 7): a genome is an unordered *set* of gene families, each present in some copy number, with no position, no neighbours and no length. It is the right model whenever what you care about is gene *content* — presence/absence profiles, copy number, reconciliations, family sizes — and it is the only level that supports coupling between families (Chapter 10).

Once the species tree is fixed, ZOMBI2 populates it with genes: a single forward continuous-time (Gillespie) process runs over every branch alive at a given moment, firing discrete events that create, move, and remove gene copies. The result is a set of gene families, each with its own history, threaded through the species tree.

8.1 The four unordered events

The genome of a lineage is a collection of gene copies. As simulated time advances, four kinds of event can fire on any living branch — the four events of the unordered level (the ordered and nucleotide levels add more; see Chapter 7 and their own chapters):

Event	Effect
Origination (O)	a brand-new family appears (one copy) on a branch
Duplication (D)	a gene copy splits into two
Transfer (T)	a copy is gained by another lineage alive at that time
Loss (L)	a copy is removed

Speciation is implicit: at each species-tree node the branch’s genome is inherited, intact, by both children. No gene event is attached to the node itself; the four events fire only along the branches between nodes.

The process is a Gillespie simulation over all branches alive at once. Duplication, transfer, and loss act **per gene copy**, so a family’s total event rate scales with its current copy number and its size follows an exponential birth–death. Origination acts **per branch**, seeding new families over time independently of what is already present.

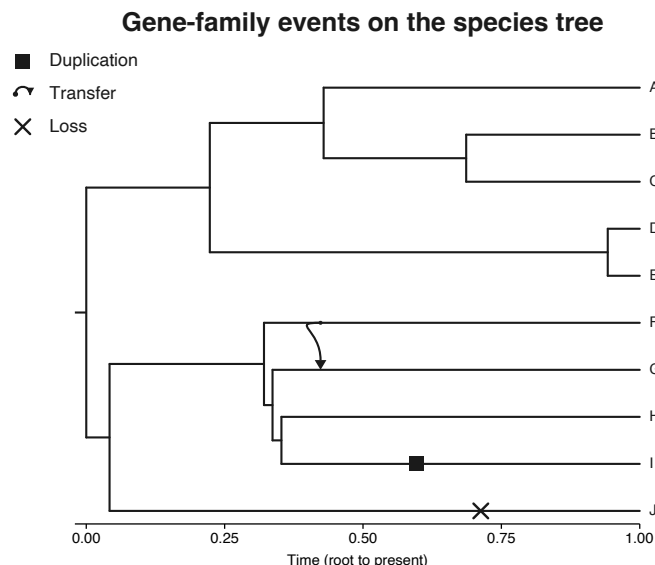


Figure 8.1. One gene family evolving along a species tree: a duplication, a loss, and a transfer, each placed on the branch where the Gillespie process fired it.

8.2 Rates

Within the unordered level, `--rate-model` — or a `RateModel` object in Python — chooses **how the four rates vary across gene families**. Chapter 7 summarises the four rate models in a table; this section works through them. Two of them are available on the command line, `shared` (the default) and `per-genome`, as is the coupled model (`--rate-model coupled`, Chapter 10); per-family rates are Python-API for now.

8.2.1 Shared rates

`SharedRates` gives every family the same per-copy D/T/L rates and a shared per-branch origination rate:

```
from zombi2.genomes import (
    SharedRates, PerGenomeRates, FamilySampledRates, TransferModel, simulate_genomes,
)
from zombi2.distributions import Gamma, Exponential, LogNormal, Uniform, Fixed

rates = SharedRates(duplication=0.2, transfer=0.1, loss=0.25, origination=0.5)
genomes = simulate_genomes(tree, rates, initial_families=40, seed=42)
```

Because the family-level rate scales with copy number, this is a gene-wise model: bigger families experience more events. There is a shorthand that builds `SharedRates` for you from the same keywords:

```
genomes = simulate_genomes(tree, duplication=0.2, transfer=0.1, loss=0.25,
                           origination=0.5, initial_families=40, seed=42)
```

8.2.2 Per-genome rates

`PerGenomeRates` fires each event at a **constant per-genome rate**, independent of genome size; a target copy is then chosen uniformly among the copies present:

```
genomes = simulate_genomes(tree, PerGenomeRates(duplication=1.0, transfer=0.3,
                                                loss=0.5, origination=0.4),
                           initial_families=20, seed=1)
```

Because the rate no longer scales with copy number, family sizes grow *linearly* rather than exponentially, which makes per-genome models far less prone to runaway growth than gene-wise ones.

8.2.3 Per-family sampled rates

`FamilySampledRates` gives each family its **own** D/T/L rates, drawn from distributions the first time the family appears and kept fixed for its lifetime:

```
rates = FamilySampledRates(
    duplication=Gamma(2, 0.06),          # built-in distribution
    transfer=Exponential(0.08),
    loss=Gamma(2, 0.07),
    origination=0.5,                    # per-branch, a single rate
)
genomes = simulate_genomes(tree, rates, initial_families=40, seed=42)
```

Each distribution argument accepts a built-in (`Gamma(shape, scale)`, `Exponential(mean)`, `LogNormal(mu, sigma)`, `Uniform(low, high)`, `Fixed(value)`), any `scipy.stats` frozen distribution, or a callable `rng -> float`. Negative draws are clipped to 0.

Note

`initial_families` sets how many families the root genome starts with, each originated at time 0. Additional families continue to appear over the run at the origination rate.

8.2.4 From the command line

The same four rates drive the `genomes` command:

```
zombi2 genomes --tree species_tree.nwk \
  --dup 0.2 --trans 0.1 --loss 0.25 --orig 0.5 \
  --initial-families 40 --seed 42 -o out/
```

`simulate_genomes` returns a `Genomes` object exposing the input `species_tree`, the profiles matrix (families $\times$ extant species), the chronological `event_log`, the per-family `gene_families` records, and `gene_trees()`; `genomes.write("out/")` serialises them.

All the rate models take the same four rates (duplication, transfer, loss per copy or per genome; origination per branch) and compose with the growth caps below.

8.3 Transfers

The rate model decides *how often* a transfer fires; a `TransferModel` decides *what a transfer does* — who receives the copy, and whether it adds to or overwrites the recipient’s genome:

```
genomes = simulate_genomes(
    tree, transfer=0.3,
    transfers=TransferModel(
        replacement=0.2,      # additive vs replacement
        distance_decay=2.0,   # recipient choice by phylogenetic distance
        allow_self=False,     # self-transfer would act as a duplication
    ),
)
```

The default `TransferModel()` is additive, uniform-recipient, and forbids self-transfer.

8.3.1 Recipient choice

By default the recipient is drawn **uniformly** among the lineages alive at the transfer time. Setting `distance_decay`= λ favours phylogenetically close recipients: a candidate r is weighted by $\exp(-\lambda d)$, where $d = 2(t - t_{\text{MRCA}})$ is the patristic distance between donor and candidate at the transfer time t . Larger λ means more local transfers; distant transfers are damped but never forbidden.

8.3.2 Additive versus replacement

- **Additive** (`replacement=0`): the recipient simply gains a copy, a net +1.
- **Replacement** (`replacement=p`): with probability p the transfer also removes one pre-existing copy of that family in the recipient — a net-zero swap. Replacement is only possible when the recipient already carries the family; otherwise the transfer is additive. `replacement=1` makes every eligible transfer a replacement. In the event log a replacement appears as a transfer plus a compensating loss.

8.3.3 Self-transfer

With `allow_self=True` the donor is itself an eligible recipient. A self-transfer creates a second copy in the same genome, which is mechanically a duplication. This lets you drop explicit duplications and run a transfer/loss-only model:

```
simulate_genomes(tree, transfer=1.0, duplication=0.0,
                 transfers=TransferModel(allow_self=True),
                 max_family_size=0.5, seed=1)
```

Warning

Self-transfers grow families exactly as duplications do, so always pair `allow_self=True` with a growth cap.

8.4 Bounding growth

A family’s copy number is a birth–death process in disguise: with duplication rate above loss rate, its expected size grows like $e^{(d-l)t}$ without bound. Both duplication and transfer create copies, so both must be reined in. ZOMBI2 offers a hard cap and a soft cap, which compose.

8.4.1 Hard cap: `max_family_size`

A single ceiling on family size, enforced across **all** copy-creating events:

```
simulate_genomes(tree, duplication=0.5, transfer=0.2, loss=0.1, origination=0.3,
                 max_family_size=0.5)      # cap = round(0.5 * N_species)
```

An **integer** is an absolute cap (`max_family_size=20`); a **float** is a fraction or multiple of the number of species (0.5 gives half the tips, 2.0 twice the tips). At the cap, duplication is rate-suppressed and an additive transfer that would overflow is turned into a replacement, so family size never exceeds the ceiling.

8.4.2 Soft cap: `carrying_capacity`

A logistic, duplication-only alternative that lives on the rate model: the per-copy duplication rate is scaled by $\max(0, 1 - n/K)$, so family size settles *around* K with a proper stationary distribution:

```
SharedRates(duplication=0.5, loss=0.1, origination=0.3, carrying_capacity=20)
```

Note

`carrying_capacity` shapes duplication smoothly but does **not** bound transfers; use `max_family_size` when you need a firm ceiling that also accounts for transfer-driven growth. You can set both.

If a run still diverges — for example `allow_self=True` with no cap — the simulator raises a clear error pointing you to `max_family_size` rather than hanging.

Chapter 9

Gene trees and output

A gene-family simulation produces one artefact that everything else is derived from: the *event log*. This chapter treats that log as the ground truth it is, shows how the complete and extant gene trees are read back out of it, documents the profile matrix and its sparse form, and specifies every file the output folder can contain — including the compact event trace that lets large runs skip the per-family reconstruction entirely.

9.1 The event log is a full genealogy

Every event re-mints the gene-lineage ids it touches, and speciations are recorded alongside the origination, duplication, transfer and loss events, so each family’s slice of the log is a complete parent-to-children genealogy. Nothing is inferred after the fact: the simulator writes the true species branch of every event, so gene-tree reconstruction and reconciliation are pure post-processing (as in ZOMBI 1), not likelihood or parsimony inference.

You can walk the log directly. Each record is an `EventRecord` carrying `time`, `event` (an `EventType`), `branch`, optional `donor` and `recipient`, and `genes` — the per-gene rows, the first of which is the source lineage and the rest its descendants. Each gene row exposes `.gid` (the gene-lineage id) and `.role`.

```
from zombi2.events import EventType

for r in genomes.event_log:
    if r.event is EventType.TRANSFER:
        print(r.time, r.donor, ">", r.recipient, [op.gid for op in r.genes])
```

`genomes.event_log.by_family()` (also reachable as `genomes.gene_families`) groups the flat log into `{family_id: [EventRecord, ...]}`, the per-family lists that every reconstruction consumes.

9.2 Reconstructing the gene trees

`gene_trees()` turns the per-family records into Newick strings, one pair per family:

```
trees = genomes.gene_trees() # {family_id: (complete, extant)}
complete, extant = trees["7"]
```

- **complete** — every lineage the family ever had, including the ones that were lost. A lost tip is labelled `LOSS_<gid>`.
- **extant** — pruned to the lineages with a surviving copy, with the resulting degree-two nodes suppressed. An extant tip is labelled `<species>_<gid>`. None when the family has no survivors.

The complete gene tree

■ Duplication
 ▲ Transfer
 ✕ Loss

Time (root to present)

A_1
 B_1
 C_1
 D_1
 E_1
 F_1
 G_1
 G_2
 H_1
 I_1
 I_2

A fully annotated reconciliation is available separately. `reconciliations()` returns `{family: Reconciliation(complete, extant, events)}`, where `complete` and `extant` are the two trees annotated with their species mapping and `events` is the S/D/T/L event list. Reconciliation here is exact annotation, not LCA inference, because the generating species branch of every event is already known.

The `ProfileMatrix` is the key object for phylogenetic-profiling analyses: gene families (rows) by extant species (columns), read straight off the final genome at each leaf. It is exposed as `genomes.profiles`.

```
P = genomes.profiles
P.families      # row labels (family ids)
P.species       # column labels (species names)
P.matrix        # dense integer copy-number array
P.presence()    # dense binary presence/absence (int8)
P.to_tsv()      # dense TSV text (to_tsv(presence=True) for the 0/1 form)
```

A copy-number profile is overwhelmingly zero — most families live in a handful of species — so the dense `families × species` array is $O(N^2)$ in tip count and becomes the memory wall long before the simulation itself does. `ProfileMatrix` therefore stores its data in **COO** form: three parallel arrays holding, one entry per *present* cell, the family index, the species index and the copy number. This is $O(\text{non-zeros})$, roughly $O(N)$.

The dense `matrix` and `presence()` views densify only when you touch them. The summaries below never do — they run straight off the COO arrays and stay cheap at a million tips:

```
P.nnz           # number of present (non-zero) cells
P.coo           # the raw (rows, cols, data) arrays
P.presence_per_family() # species count per family
P.copies_per_family()  # total copies per family
P.copies_per_species() # genome size per species
```

For on-disk use, `to_coo_tsv()` writes the sparse long table — one `family<TAB>species<TAB>copies` row per present cell — with two leading `#species` and `#families` header lines that record the full label sets in order, so the round trip through `from_coo_tsv()` is lossless even for all-absent rows or columns. `from_tsv()` / `from_coo_tsv()` load either format back, and `from_leaf_genomes()` builds a matrix directly from a `{node: Genome}` map without ever allocating the dense array.

9.4 The output folder

`genomes.write("out/")` writes the ZOMBI-1-style output folder; the CLI's `--write` flag (whose parts map onto the same `include=` set) selects the same components by name. `species_tree.nwk` and `species_nodes.tsv` are always written; every other component is opt-in, and omitted components do no work — in particular `trees` is what drives the expensive gene-tree reconstruction.

```
genomes.write("out/", include=["profiles", "trees", "transfers"])
```

```
zombi2 genomes -t species_tree.nwk --dup 0.1 --trans 0.05 --loss 0.1 \
  --write profiles trees transfers summary -o out/
```

The selectable components are exactly `Genomes.WRITE_PARTS`: `profiles`, `trace`, `trees`, `events`, `transfers`, `summary` (the CLI also accepts `all`). Each maps to these paths:

Component	Path(s) written
<i>(always)</i>	<code>species_tree.nwk</code> , <code>species_nodes.tsv</code>
profiles	<code>Profiles.tsv</code> + <code>Presence.tsv</code> (dense), or <code>Profiles_sparse.tsv</code> (with <code>--sparse</code>)
trees	<code>gene_trees/<fid>_complete.nwk</code> , <code>gene_trees/<fid>_extant.nwk</code>
events	<code>gene_family_events/<fid>_events.tsv</code> — one file per family
transfers	<code>Transfers.tsv</code> — one row per transfer
summary	<code>Gene_family_summary.tsv</code> — per-family origin, event counts, copies
trace	<code>Events_trace.tsv</code> — the whole genealogy in one file

`species_nodes.tsv` carries `name`, `time`, `is_leaf`, `is_extant` per node. A per-family events file has columns `time`, `event`, `branch`, `donor`, `recipient`, `nodes`, where `nodes` is a `;`-separated list of `role=gid` pairs. `Transfers.tsv` gives `time`, `family`, `donor_branch`, `recipient_branch`, `parent_id`, `donor_copy_id`, `transfer_id`. `Gene_family_summary.tsv` reports, per family, the origin time and branch, the D / T / L / speciation counts, and the extant copy and species-present totals — the last two taken from the sparse profile so no dense matrix is built.

9.4.1 Sparse profiles

`--sparse` (Python `sparse=True`) replaces the dense `Profiles.tsv` / `Presence.tsv` pair with a single `Profiles_sparse.tsv` in the COO long format described above. The dense pair is the one output that does not scale — it is `families × species` — so at large tip counts the sparse form is the only viable profile output. It only affects the profile component; the CLI rejects `--sparse` unless `profiles` is among the requested parts.

9.4.2 The event trace and its fast path

`events` writes one small file per family; `trace` writes the entire log as a single `Events_trace.tsv`. Its header is:

```
time    event    branch    donor    recipient    family    parent    child1    child2
```

One row per event, with the touched gene-lineage ids as `parent`, `child1`, `child2` (blank when the event has no children). This is the scalable substitute for the `gene_family_events/` directory, and it is a complete record: the gene trees can be rebuilt from it later, on demand, with no need to keep the simulation object around.

The trace also unlocks a genuinely faster run. `simulate_genomes(..., output="trace")` returns a `GenomeTrace` rather than a full `Genomes`: it holds the genealogy in its cheapest form and builds the per-event Python objects and gene trees only if you ask.

```
trace = z.simulate_genomes(tree, duplication=0.1, transfer=0.05, loss=0.1, output="trace")
trace.write("out/", include=["trace", "profiles"])    # the cheap components
genomes = trace.genomes()                            # promote on demand
```

`trace.write(...)` defaults to `include=("trace", "profiles")`, the two components that cost almost nothing; writing `Events_trace.tsv` from the engine's raw columns never materialises a single `EventRecord`. Requesting any heavier component (`trees`, `events`, `transfers`, `summary`) transparently promotes the trace to a full `Genomes` first, so the fast path is exactly the subset `{trace, profiles}`. On the command line the same fast path fires automatically when `--write` is `trace`, or `trace profiles`.

Note

The compact trace carries **no speciation rows** — a lineage keeps its id across speciations, so they are redundant and omitted. When the trace is promoted (via `trace.genomes()`) or reloaded with a species tree, it is *replayed* against that tree to re-insert the implied speciations and re-mint per-instance gene ids, yielding the full O/S/D/T/L genealogy that ordinary reconstruction expects.

Chapter 10

Gene-family coupling

This chapter is the fourth rate model of the unordered genome level (Chapter 8): **coupled** gene families, which gain and lose *non-independently*. Like the other unordered rate models it runs on a presence/absence genome, and only there.

By default every gene family in ZOMBI2 evolves independently, so the phylogenetic profile correlates families *only* through the shared species tree. Real genomes correlate through **function**: families in the same pathway or complex tend to be present or absent together (Pellegrini et al. 1999). ZOMBI2 can inject that structure directly — a prescribed pairwise coupling J (and fields h) that makes families gain and lose non-independently — so the simulated profiles carry a *known ground-truth* coupling. It is the forward, generative counterpart of the inverse-Potts and direct-coupling-analysis methods that infer functional linkage from profiles (Croce et al. 2019; Fukunaga and Iwasaki 2022), and the benchmark those methods can be tested against.

10.1 The model

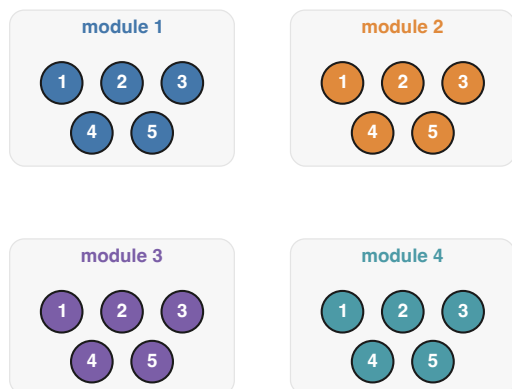
Presence or absence of a fixed panel of N families inside one genome is an Ising vector $\sigma \in \{0, 1\}^N$. Fields h_i and couplings J_{ij} (symmetric, zero diagonal) define the **local field** family i feels — its intrinsic bias plus a contribution from every *present* partner:

$$f_i = h_i + \sum_j J_{ij} \sigma_j$$

where the sum runs over present partners.

Modules are kept or lost together

A the panel: 20 families in 4 modules



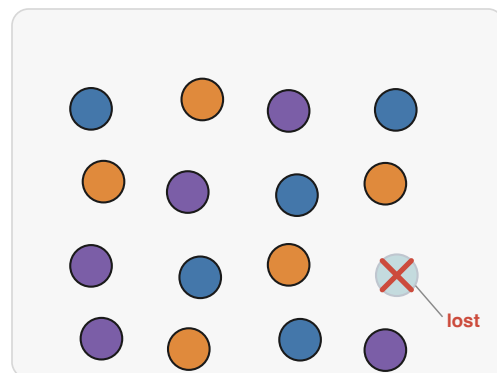
● module 1

● module 2

● module 3

● module 4

B one genome, drawn unordered



module 4 is incomplete (1 of 5): its lone gene has no partners, so a small field, a fast loss -- the module empties out

Figure 10.1. Left: the fixed panel of N families is organised into pathway modules — here four modules of five, coloured by module. Right: one genome is a present/absent vector $\sigma \in \{0, 1\}^N$ over that panel, drawn as an unordered bag of its present families. Three modules are complete and protect their members; module 4 is incomplete (one of five present), so its lone gene feels a weak field, is lost quickly (the loss law below), and the module empties out — coupled families are kept, or lost, together.

Coupling enters through loss. A present family is lost at a rate that is modulated by its local field:

$$\text{loss}_i = \text{base_loss} * \exp(-\text{beta} * f_i)$$

Coupling enters through the loss rate

$$\text{local field } f_i = h_i + \sum_j J_{ij} \sigma_j \text{ (over present partners)}$$

$$\text{loss rate } \text{loss}_i = \text{base_loss} * \exp(-\text{beta} * f_i)$$

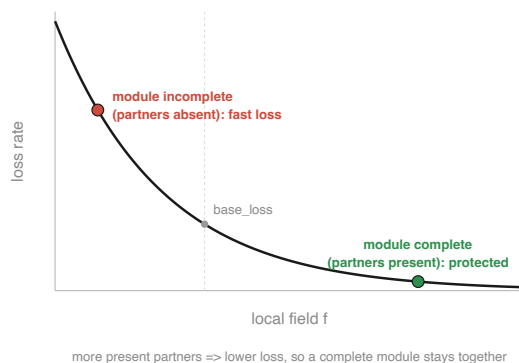


Figure 10.2. Coupling enters through the loss rate: a present family is lost at rate $\text{base_loss} \cdot e^{-\beta f}$, so a high local field (partners present) protects it while a low field lets it go quickly.

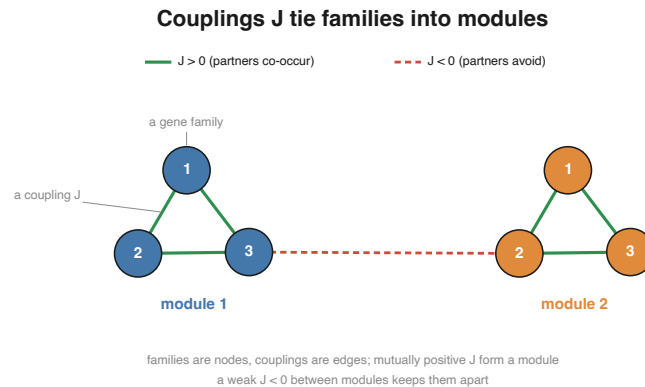


Figure 10.3. The coupling graph: families are nodes and couplings are edges. Positive couplings (solid) tie families into co-occurring modules; a negative coupling (dashed) makes two modules mutually exclusive.

The parameters have direct interpretations:

- **within** and **between** (in `pathway_blocks`) set the coupling *inside* a block — positive values make pathway members co-occur — and *across* blocks — negative values make blocks mutually exclusive “rival” pathways, while 0 leaves the blocks independent.
- **h** is a scalar applied to every family, or a length- N vector of per-family biases.
- **base_loss** is the loss at $f_i = 0$, **transfer** is the per-copy horizontal gain rate, and **beta** is the global coupling strength.
- **origination** is an optional background rate of brand-new *uncoupled* families; leaving it at 0 keeps the panel closed.

Panel families are named `F0` through `F{N-1}`, and `spec.dense_J()` materialises the coupling matrix for inspection.

10.3 Running a coupled simulation

Simulate a species tree, then evolve the coupled panel over it:

```
tree = z.simulate_species_tree(z.BirthDeath(1.0, 0.2), n_tips=100, age=6.0, seed=1)
res = z.simulate_coupled(tree, spec, seed=1)

res.profiles          # ProfileMatrix: N panel rows x extant species
res.profiles.presence()
```

`simulate_coupled` seeds the root with the whole panel present — override this with `initial_presence`, a length- N 0/1 mask — and returns a `CoupledResult` exposing `.profiles`, `.leaf_genomes`, `.event_log`, and `.spec`. Every panel row is kept in the profile matrix, including families absent from all extant species. Transfers default to `TransferModel(replacement=1.0)`; pass `transfers=` to customise them, for example with distance-weighted recipients.

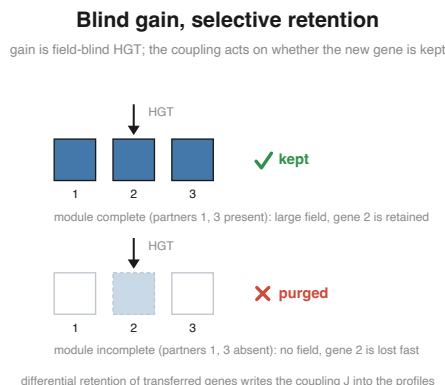


Figure 10.4. Blind gain, selective retention: horizontal transfer re-acquires a family regardless of context, and the coupled loss then keeps it where its partners are present but purges it where they are absent — the differential retention that writes the coupling J into the profiles.

Because `PottsRates` is a custom rate model, a coupled simulation runs on the pure-Python engine: the coupling breaks the per-family independence that the Rust fast path assumes. The cost is $O(N + \text{nnz}(J))$ per event, which is comfortable at benchmark scale.

10.3.1 From the command line

The coupled model is a `--rate-model`, so a pathway-block coupling runs straight from the `genomes` command without building the spec by hand:

```
zombi2 genomes -t species_tree.nwk --rate-model coupled \
  --pathways 4,4 --within 3 --between 0 --field 2 --base-loss 1 \
  --seed 1 --write profiles -o out/
```

`--pathways` gives the block sizes (4,4 = two blocks of four families); `--within` and `--between` set the coupling J inside and across blocks; `--field` is the presence field h ; and `--base-loss` is the baseline loss rate, the channel the coupling acts through ($\text{loss}_i = \text{base-loss} \cdot e^{-\beta f_i}$), with `--beta` scaling it. `--gain-coupling` additionally turns on field-biased transfer establishment (the gain-side coupling). The family panel is fixed by `--pathways` and seeded whole at the root.

10.4 Two caveats worth knowing

Note

Recovered couplings track the injected J in sign and rank, not as a clean multiple. Because the gain channel is field-blind, detailed balance does not hold and the process has no exact Boltzmann stationary distribution. Couplings recovered from the profiles follow the injected J *monotonically* — positive to co-occurrence, negative to avoidance — but not as an affine constant. This is the price of keeping regain mechanistic, since a lost family returns only from a donor that still carries it. An exact-Boltzmann mode, using an explicit Glauber gain, is a documented extension rather than the default.

Warning

Control for the phylogeny. On an ordinary birth–death tree even *uncoupled* families co-occur, because loss is clade-restricted and families share ancestry. The coupling model’s own tests isolate the injected J on a **near-star** tree — all lineages split just below the root and then evolve independently. This is exactly why real inference corrects for the tree (Fukunaga and Iwasaki 2022), and any downstream benchmark should respect the same caveat.

10.5 Coupling a trait instead of another family

The same retention mechanism drives trait-linked gene families (the coevolution chapter). There a family’s loss is modulated by a **trait** value, `loss_i = base_loss * exp(-effect * w_i * s)`, rather than by its partners’ presence — the coupling field with the trait standing in for the coupled state. Use this chapter’s model when you want family-to-family structure, and the trait-linked model when you want gene content to track a phenotype.

Chapter 11

Ordered genomes

Chapter 7 introduced three models of genome evolution. The first — unordered gene families (Chapter 8) — treats a genome as an unordered *set*. This chapter covers the second, **ordered gene families**: genes sit on a chromosome and their order matters, but distance is still counted in genes, not nucleotides. (The third model, nucleotide genomes, is Chapter 12.)

By default a genome in ZOMBI2 is *order-free* (`UnorderedGenome`): a multiset of gene families with copy numbers, which is all you need for phylogenetic profiles. When gene *order* matters — synteny, operons, rearrangements — use `OrderedGenome`, the classic ZOMBI-1 model. An ordered genome is a circular chromosome of genes, each carrying a strand orientation, with no intergenic regions.

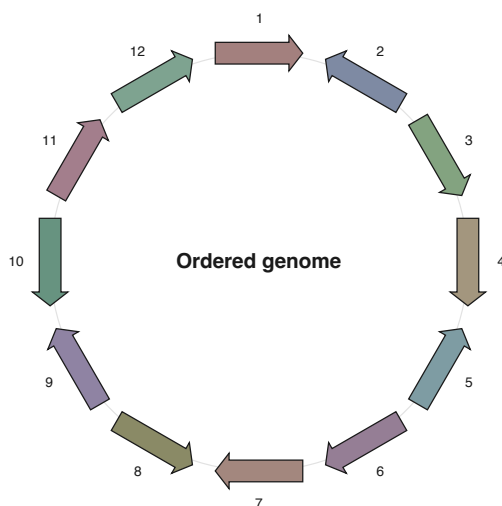


Figure 11.1. An ordered genome is a circular chromosome of genes, each with a strand orientation.

11.1 Using an ordered genome

You select the representation by passing a `genome_factory` to `simulate_genomes`. Because `OrderedGenome` needs an `extension` parameter (the segment-length knob), you pass a small factory that builds one from the gene ids:


```

from zombi2.genomes import SharedRates, OrderedGenome, simulate_genomes

rates = SharedRates(
    duplication=0.2, transfer=0.1, loss=0.2, origination=0.4,
    inversion=0.3, transposition=0.3,      # rearrangement rates (ordered genomes only)
)
genomes = simulate_genomes(
    tree, rates, initial_families=30, seed=1,
    genome_factory=lambda ids: OrderedGenome(ids, extension=0.5),  # continuation prob; mean
)

leaf = next(iter(genomes.leaf_genomes.values()))
leaf.chromosome      # list of OrderedGene(gid, family, orientation=±1), in order

```

Each leaf genome exposes its chromosome as an ordered list of `OrderedGene` records, every one tagged with its family and a strand orientation of +1 or −1.

11.1.1 From the command line

The ordered level is also a `--genome-model`, so the same run is available from the `genomes` command:

```

zombi2 genomes -t species_tree.nwk --genome-model ordered \
    --dup 0.2 --trans 0.1 --loss 0.2 --orig 0.4 \
    --inversion 0.3 --transposition 0.2 --mean-length 2 \
    --initial-families 25 --seed 1 --write profiles trees events -o out/

```

Here `--inversion` and `--transposition` are the rearrangement rates (**per gene copy**, unlike the per-nucleotide rates of the nucleotide model), and `--mean-length` is the segment-length knob: the **mean number of genes** an event spans (2 here; omit it for single-gene events), with the lengths drawn geometrically around that mean. Because rearrangements live on the shared per-copy rate model, they require the default `--rate-model shared`; `--rate-model per-genome` runs an ordered genome with duplication, transfer and loss only.

11.2 Segment events

Events act on a *contiguous segment* of the circular chromosome rather than a single gene. The number of genes in a segment is **geometric**, set by its **mean length** — on the command line the `--mean-length` knob (in genes), and in the Python API the equivalent per-step continuation probability `extension` of `OrderedGenome`, the two related by $\text{extension} = 1 - 1/\text{mean length}$. Every segment starts with one gene and each further gene is added with probability `extension`, so a segment of k genes has probability $(1 - \text{extension})\text{extension}^{k-1}$ and mean $1/(1 - \text{extension})$. A mean of one gene (`extension=None` or 0) gives single-gene events; larger means give longer segments with a heavier tail.

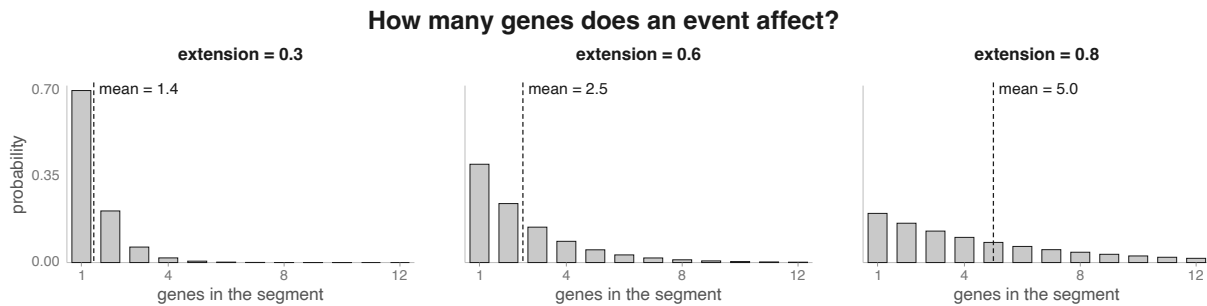


Figure 11.2. The number of genes an event affects is geometric, with the mean set by `--mean-length` (equivalently the continuation probability `extension = 1 - 1/mean`). Small means keep events local (mostly single genes); larger means give longer segments with a heavier tail. The dashed line marks the mean.

Event	Effect on the segment
Duplication	tandem copy inserted after the segment
Loss	segment removed
Transfer	segment copied into a recipient at a chosen position
Inversion	segment reversed and every strand flipped
Transposition	segment cut and pasted elsewhere

The first three events change gene *content* exactly as they do for unordered genomes; the ordered representation simply also tracks where the affected genes sit and which way they point.

11.2.1 Rearrangements: inversion and transposition

On top of duplication, transfer and loss, ordered genomes add two rearrangement events. An *inversion* reverses a segment in place and flips the strand of every gene it contains.

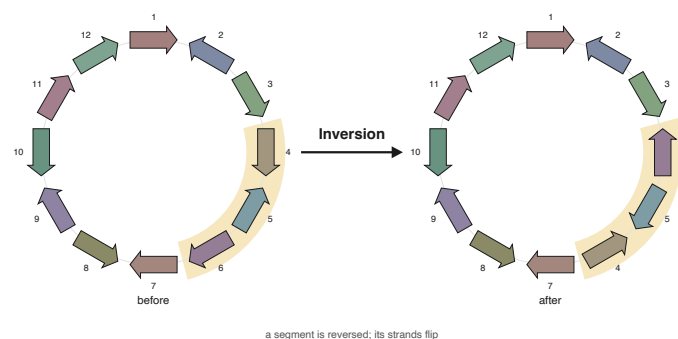


Figure 11.3. An inversion reverses a chromosomal segment and flips the orientation of each gene in it.

A *transposition* cuts a segment out and pastes it elsewhere on the chromosome.

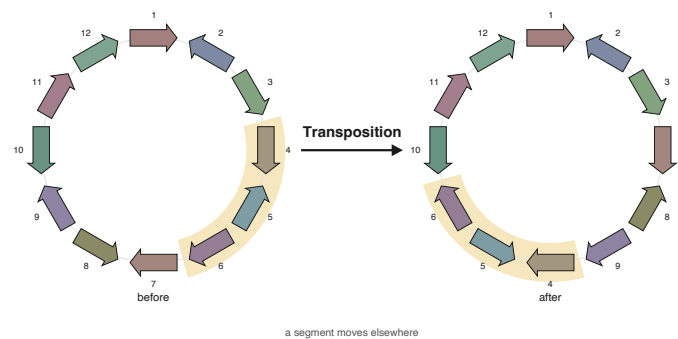


Figure 11.4. A transposition moves a chromosomal segment to a new location.

Inversions and transpositions change gene order and orientation but *not* gene content.

Note

Because rearrangements preserve gene content, they leave the profile matrix and the gene trees unchanged. They appear only in the event log and in the final chromosome order.

11.3 How events reach the genome

Inversion and transposition rates are emitted by `SharedRates` as candidate events, but a genome only undergoes the events it declares in `supported_events()`:

- `UnorderedGenome` supports `{0, D, T, L}`. It silently ignores inversion and transposition rates.
- `OrderedGenome` supports `{0, D, T, L, I, P}`. It acts on them.

The same rate model therefore serves both representations, and the genome decides what applies. Adding gene order required no change to the simulator, the sampler, the rate interface or the output code — you swap the `genome_factory` and nothing else.

Chapter 12

Nucleotide genomes

Chapter 7 introduced three models of genome evolution. The first two — unordered gene families (Chapter 8) and ordered gene families (Chapter 11) — treat each gene as an indivisible token. This chapter covers the third and most detailed: the **nucleotide genome** model, where a genome is a real sequence of base pairs and structural events act on **variable-length segments** of nucleotides. This resolves duplication, lateral gene transfer, and gene order and orientation at nucleotide resolution, and reconstructs a gene tree for every stretch of shared ancestry — the *blocks* of the sections below.

12.1 The model

`simulate_nucleotide_genomes` evolves a genome forward along a fixed species tree. It starts from `initial_chromosomes` chromosome(s) of `root_length` nucleotides at the root, and these events fire:

Event	Effect
<code>duplication</code>	copy a segment elsewhere (a tandem duplicate)
<code>transfer</code>	copy a segment into another lineage (a laterally transferred copy)
<code>loss</code>	delete a segment
<code>inversion</code>	reverse a segment's orientation
<code>transposition</code>	move a segment
<code>origination</code>	insert a brand-new gene under a fresh source
<code>insertion</code>	add a run of novel intergenic nucleotides (an indel; off by default)
<code>deletion</code>	remove a run of intergenic nucleotides (an indel; off by default)

Duplication, transfer, loss, inversion, transposition, and the two indels are **per-nucleotide** rates — the total genome rate is `rate × current_length`, so longer genomes evolve faster — while `origination` is **per branch**. Event lengths follow a geometric model with mean $1/(1 - \text{extension})$ nucleotides (`extension=0.99` gives about 100 nt); indels have their own mean-length knob (see *Intergenic indels* below). The two indels are off by default (rate 0).

```
tree = z.simulate_species_tree(z.BirthDeath(1.0, 0.3), n_tips=20, age=5.0, seed=1)

result = z.simulate_nucleotide_genomes(
    tree, root_length=1000,
```

```

duplication=1e-4, transfer=5e-5, loss=1.5e-4,
inversion=1e-3, transposition=5e-5, origination=0.2, seed=1)

```

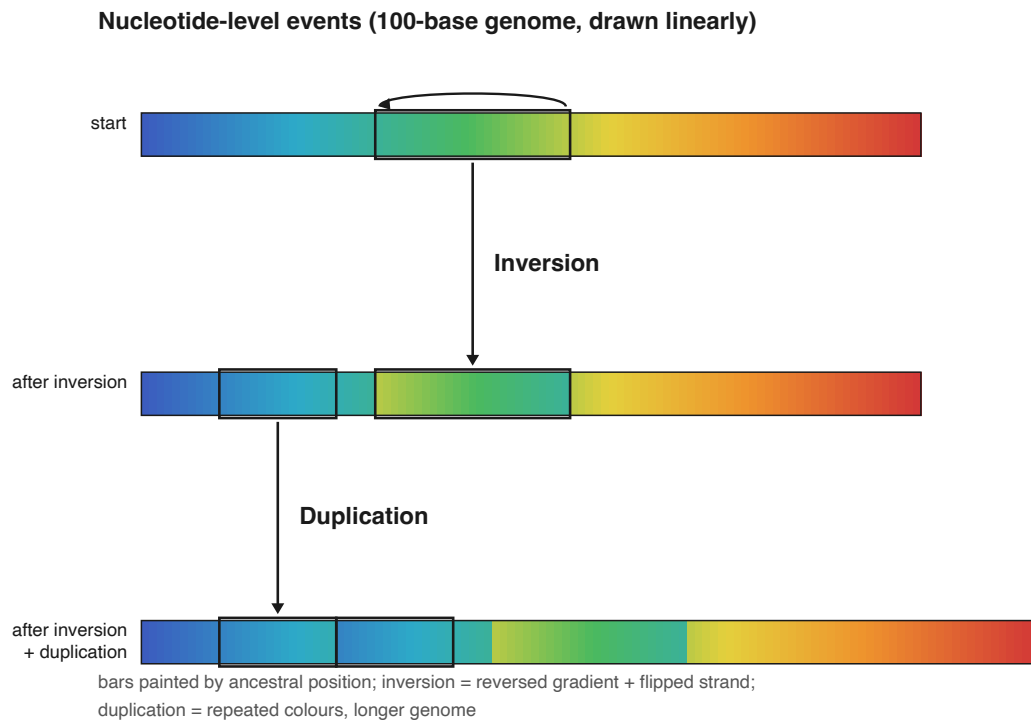


Figure 12.1. An inversion reverses a segment’s orientation and a tandem duplication lengthens it, each acting on a variable-length stretch of nucleotides.

12.2 Blocks: units of shared ancestry

The simulator partitions the surviving material into **blocks** — maximal segments that share one unbroken ancestry. Every event boundary splits blocks, so a block is the finest unit for which a single gene tree is meaningful. Results are expressed over blocks:

```

block_ids, species, matrix = result.profile_matrix()    # copy number of each block per extant

```

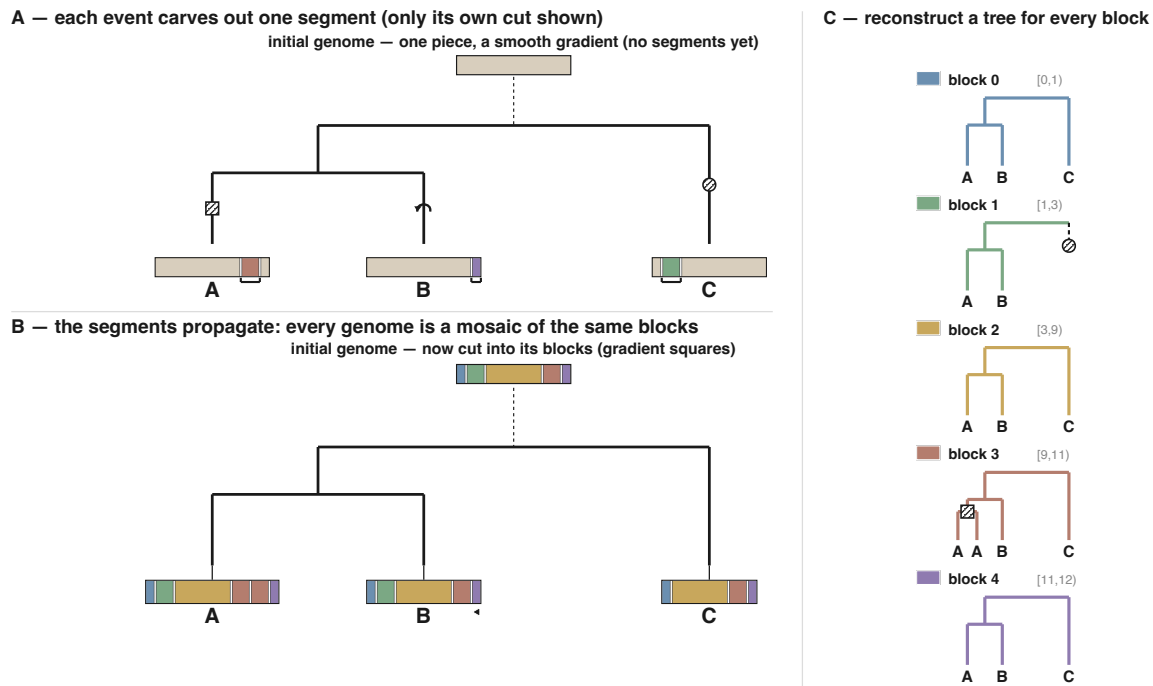


Figure 12.2. Each structural event carves out a segment; the same breakpoints partition every genome into shared blocks (left, panels A and B), and each block has its own reconstructed gene tree (right, panel C).

A duplication adds a tip to the block's tree, a loss prunes one, and an inversion leaves the genealogy unchanged.

12.3 Reading a leaf genome

```
leaf = tree.leaves()[0]
result.leaf_mosaic(leaf)    # the genome as ordered, signed blocks: [(block_id, strand), ...]
result.trace_back(leaf)    # every nucleotide's ancestral origin: [(source, src_pos, strand), ...]
```

`leaf_mosaic` gives the leaf as a sequence of blocks with orientation; `trace_back` resolves each nucleotide to where it came from.

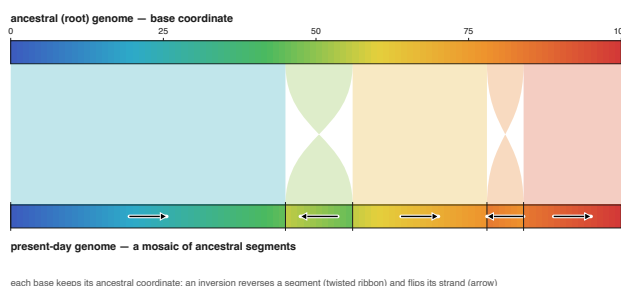


Figure 12.3. A leaf genome (bottom) traced back to its ancestral coordinates (top): collinear stretches keep the gradient, an inversion shows it reversed.

12.4 Per-block gene trees and reconciliation

With the default `output="genomes"` (the pure-Python engine), the result also carries the full event log and a reconstructed gene tree per block:

```
trees = result.block_gene_trees()           # {block_id: (complete_newick, extant_newick)}
result.write_reconciliations("out/")        # reconciled trees + the events table on disk
```

12.5 Genes and intergenes

By default a genome is an unstructured sequence and “genes” are only recovered *post hoc* as blocks. Pass `gene_intervals` — non-overlapping (`start`, `end`) (or (`start`, `end`, `name`)) intervals on the root chromosome — to declare **genes** up front. Everything else is **intergene**. In this genic mode:

- **Genes are never split.** Event breakpoints fall only in intergene positions, so every event moves, copies, inverts, or deletes a gene *as a whole*. Each gene is therefore exactly one block (one genealogy) wherever it survives; intergene stretches still fragment into many intergene blocks. A short event drawn entirely inside a gene is promoted to the whole gene.
- **Pseudogenization.** With probability `pseudogenization`, a loss that hits a gene *demotes* it to intergene — the sequence is retained, but the gene loses function. It is a state change on the continuing lineage (a **G** node in that gene’s tree), not a deletion, and it is lineage-specific: the gene stays functional in sister lineages.
- **Homologous replacement transfer.** With probability `replacement`, a transfer replaces the recipient’s syntenic copy instead of adding a new one. The homologous locus is found by the genes flanking the transferred segment — the recipient must carry the same flank genes in the **same orientation** (an inverted flank breaks synteny) — and the recipient material between those flank genes is replaced (and logged as recipient losses). When the recipient has no such homolog, the transfer falls back to additive insertion.
- **Origination** mints a brand-new gene (its own gene tree), as in the base model.

```
genes = [(100, 180, "dnaA"), (300, 360, "gyrB"), (500, 620, "rpoB")]
result = z.simulate_nucleotide_genomes(
    tree, inversion=1e-3, loss=8e-4, duplication=5e-4, transfer=5e-4,
    root_length=1000, extension=0.97, gene_intervals=genes,
```

```

    pseudogenization=0.3, replacement=0.4, seed=1)

result.gene_trees()          # {block_id: (complete, extant)} for the gene blocks
result.intergene_trees()     # ...and for the intergene blocks
result.pseudogenizations()   # [(block_id, gene_id, species_branch, time, gene_lineage), ...]

```

Blocks carry their classification (`block.kind` is "gene" or "intergene", plus `block.gene_id`), so `gene_blocks()` / `intergene_blocks()` partition the block set. Genic mode runs on the Python engine only (the Rust profiles path does not model genes). On the CLI:

```

zombi2 genomes -t species_tree.nwk --genome-model nucleotide \
  --genes genes.tsv --pseudogenization 0.3 --replacement 0.4 \
  --inversion 0.001 --loss 0.0008 --write profiles trees -o out/

```

where `genes.tsv` is a BED/TSV of `start end [name]` lines. The run writes `genes.tsv` (the annotation, including originated genes), gene and intergene trees under `Gene_trees/` and `Intergene_trees/`, a `kind/gene_id` column in `blocks.tsv`, and `Pseudogenizations.tsv`.

12.5.1 Starting from a real genome (GFF)

Instead of writing intervals by hand, point the model at a real annotation — e.g. a RefSeq bacterial chromosome — and it copies the genome's **length** and **gene coordinates** (the intergenes are the gaps). `read_gff` returns both; because bacterial genes sometimes overlap (shared start/stop codons, nested ORFs) and the genic model forbids breakpoints inside a gene, overlaps are removed by trimming — each gene's start is clipped to the previous gene's end, and a gene swallowed whole is dropped:

```

g = z.read_gff("GCF_000005845.2_ASM584v2_genomic.gff")    # E. coli K-12 MG1655
g.length, len(g.genes), g.n_trimmed, g.n_dropped          # 4641652, 4480, 768, 26

result = z.simulate_nucleotide_genomes(
    tree, root_length=g.length, gene_intervals=g.genes,
    inversion=2e-6, loss=1.5e-6, extension=0.999, pseudogenization=0.3, seed=1)

```

On the CLI, `--gff` sets the length and genes in one step (superseding `--genes/--root-length`):

```

zombi2 genomes -t species_tree.nwk --genome-model nucleotide \
  --gff ecoli.gff --inversion 2e-6 --loss 1.5e-6 --pseudogenization 0.3 \
  --write profiles trees -o out/

```

The GFF may be gzipped. For a multi-sequence file (chromosome plus plasmids), the most-annotated sequence is used by default; `--gff-seqid ID` (or `read_gff(..., seqid=...)`) picks another. The genes keep their annotation names (locus tag / Name), so `genes.tsv` and the trees are labelled with real gene ids.

Tip

Real chromosomes are millions of nucleotides long, and every rate is per-nucleotide. Scale the per-nucleotide rates down accordingly (the *E. coli* example uses `inversion=2e-6`, `loss=1.5e-6`) and raise `extension` toward 0.999 for realistically long segments.

12.6 Intergenic indels

Two optional events change the *length* of the chromosome by editing intergenic DNA:

- **Insertion** lays down a run of novel nucleotides — a fresh source, its own block — inside an intergene stretch, lengthening it. It is an intergenic origination of new sequence, with its own genealogy rooted at the event.
- **Deletion** removes a run of nucleotides from *within a single* intergene stretch. It is clamped so it never reaches into, spans, or removes a gene, and never shrinks the chromosome below a small floor (`MIN_GENOME_LENGTH`, one nucleotide) — a deletion can never empty the genome.

Both are **per-nucleotide** rates, off by default (0). Unlike the structural events, an indel run's length is geometric with mean `indel_mean_length` (default 10) — a knob *independent* of **extension**, since indels and rearrangements have different length scales. In genic mode indels act only in intergenes (a gene is never split, spanned, or deleted); with no genes declared they may act anywhere.

```
result = z.simulate_nucleotide_genomes(
    tree, root_length=3000, gene_intervals=genes,
    insertion=1e-3, deletion=1e-3, indel_mean_length=12, seed=5)
```

From the command line:

```
zombi2 genomes -t species_tree.nwk --genome-model nucleotide --root-length 3000 \
    --insertion 0.001 --deletion 0.001 --indel-mean-length 12 \
    --seed 5 --write profiles trees -o out/
```

Because an insertion roots a new block and a deletion terminates one, indels enter the block genealogy exactly like an origination and a loss respectively. They require the Python engine (`output="genomes"`); setting either rate routes the run there automatically.

12.7 The Rust fast path

`output="profiles"` runs the compiled `zombi2_core` Rust engine over leaf segments only — much faster, and enough for `profile_matrix()`, `leaf_mosaic()`, and `trace_back()`. It emits **no event log**, so `block_gene_trees()` is unavailable, it does **not** model genes/intergenes or indels, and it **requires** the built extension:

```
result = z.simulate_nucleotide_genomes(tree, duplication=1e-4, loss=1.5e-4,
                                         inversion=1e-3, seed=1, output="profiles")
```

Chapter 13

Trait evolution

Once you have a tree, you can evolve *traits* on it: a body size, an expression level, a discrete character such as habitat or the presence of a structure. ZOMBI2 simulates the classic phylogenetic comparative models with a single driver, `simulate_traits`, and a family of model objects that plug into it.

```
from zombi2.species import BirthDeath, simulate_species_tree
from zombi2.traits import (
    simulate_traits, BrownianMotion, OrnsteinUhlenbeck, EarlyBurst,
    MultivariateBrownian, MultivariateOU, Mk, CorrelatedBinary, HiddenStateMk,
    ThresholdModel, MultiOptimumOU, DEC, simulate_biogeography,
    pagel_lambda, pagel_delta, pagel_kappa,
)

tree = simulate_species_tree(BirthDeath(1.0, 0.3), n_tips=30, age=5.0, seed=1)
result = simulate_traits(tree, BrownianMotion(sigma2=0.5), seed=1)

result.values          # {extant leaf: value} - the observable tip data
result.ancestral_states() # {internal node: value} - exact, not inferred
```

The trait works on any `Tree`: a simulated species tree, or a gene tree loaded from disk. Because ZOMBI2 simulates the process forward, the ancestral states it returns are the true history, not a reconstruction.

13.1 The shared overlay engine

Every model, continuous or discrete, runs on one engine. A value is dropped at the root and evolved branch by branch in pre-order traversal, each node inheriting its parent's end state. The two model families differ only in how a branch is traversed:

- *Continuous* models draw the exact branch endpoint. Along a branch of length t , the change is a single normal draw whose mean and variance the model prescribes; nothing between the endpoints is simulated because nothing needs to be.
- *Discrete* models simulate the Markov jumps exactly. The engine steps through the continuous-time chain, drawing waiting times and transitions, so the realized history along each branch, a *stochastic character map*, comes for free rather than being sampled after the fact.

Two ways a branch is traversed

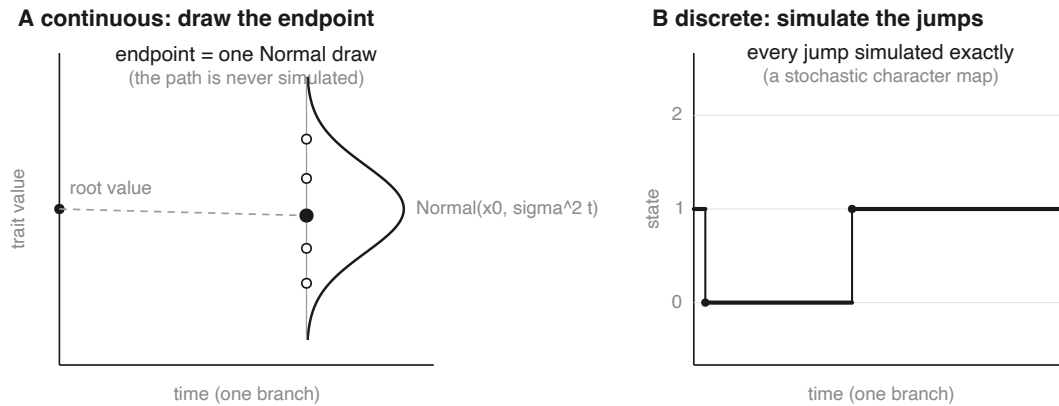


Figure 13.1. Two ways a branch is traversed. **A**, continuous models draw the branch endpoint in one step — the change over a branch of length t is a single Normal draw, and the path in between is never simulated. **B**, discrete models simulate the Markov jumps exactly, so the realized history along the branch (a stochastic character map) comes for free.

`simulate_traits` returns a `TraitResult`. Its accessors expose the process at whatever granularity you need:

Access	Meaning
<code>result.values</code>	Values at the extant tips (the observable comparative data)
<code>result.labeled_values()</code>	Same, with discrete state indices decoded to their labels
<code>result.ancestral_states()</code>	Values at every internal node (exact ancestral states)
<code>result.node_values</code>	Every node: root, internal, and tips
<code>result.history</code>	Per-branch (<code>state</code> , <code>duration</code>) segments, the stochastic map (discrete only)
<code>result.changes()</code>	Realized transitions (<code>node</code> , <code>time</code> , <code>from</code> , <code>to</code>) (discrete only)
<code>result.to_tsv()</code> / <code>result.to_newick()</code>	Tip table / annotated Newick (<code>[&trait=...]</code>)

13.2 Trait models at a glance

Model	Kind	What it captures
BrownianMotion	continuous	neutral random walk
OrnsteinUhlenbeck	continuous	stabilizing selection toward an optimum

Model	Kind	What it captures
EarlyBurst	continuous	a rate that changes through time (adaptive radiation)
MultivariateBrownian / MultivariateOU	continuous	correlated multi-trait evolution
Mk	discrete	a k -state Markov character
CorrelatedBinary	discrete	two binary characters evolving jointly (Pagel)
HiddenStateMk	discrete	hidden rate classes (corHMM)
ThresholdModel	discrete	a discrete state from a latent continuous liability
MultiOptimumOU	continuous	OU with a different optimum per regime
DEC	range	geographic-range evolution (dispersal / extinction / cladogenesis)

13.3 Continuous traits

13.3.1 Brownian motion

Brownian motion is the reference model of quantitative-character evolution ([Felsenstein 1985](#)): a random walk, $dX = \text{trend} \cdot dt + \sigma dW$. The tips are jointly multivariate-normal with mean $x_0 + \text{trend} \cdot \text{depth}$ and covariance $\text{sigma2} \cdot C$, where C is the shared-path-length matrix, so more closely related tips are more strongly correlated.

```
simulate_traits(tree, BrownianMotion(sigma2=0.5, x0=0.0, trend=0.0), seed=1)
```

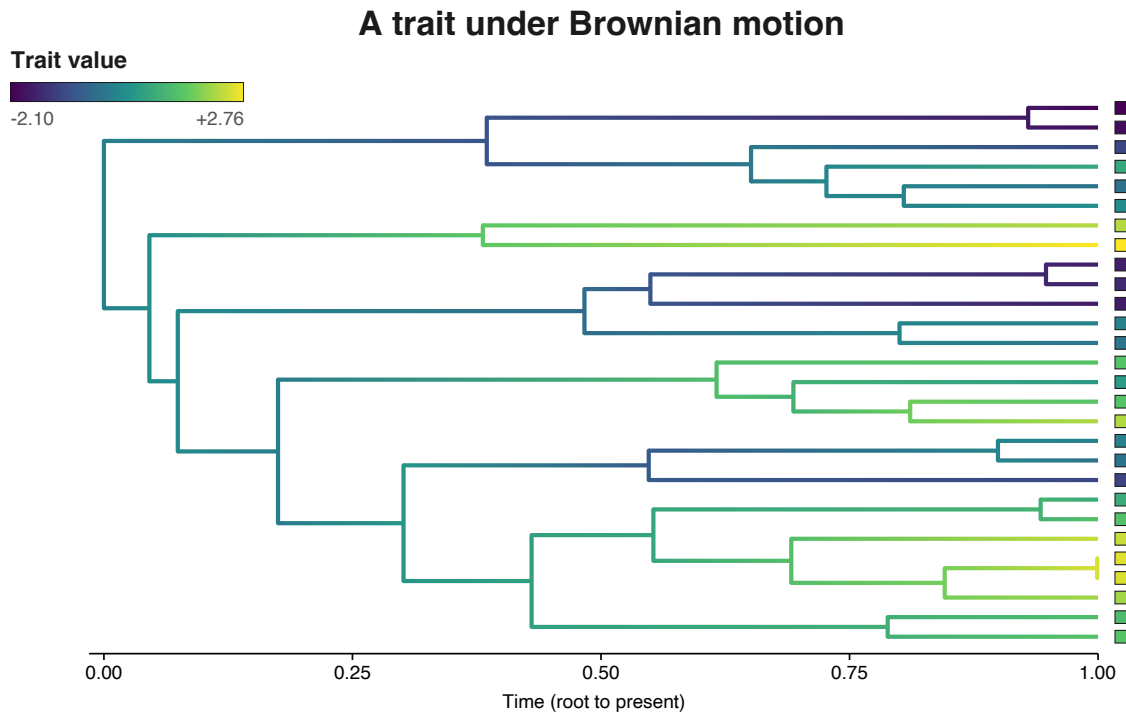


Figure 13.2. A continuous trait wanders down the branches by Brownian motion; each tip value is the endpoint of its root-to-tip random walk. Branches are painted by the trait value (viridis).

`sigma2` is the diffusion rate, `x0` the root value, and `trend` a directional drift added per unit time.

13.3.2 Ornstein–Uhlenbeck

The Ornstein–Uhlenbeck process adds stabilizing selection (Hansen 1997; Butler and King 2004): the trait diffuses but is pulled toward an optimum `theta` with strength `alpha`, $dX = \alpha(\theta - X)dt + \sigma dW$. It starts at the optimum unless you set `x0`. The pull bounds the variance, so lineages cluster around `theta` instead of wandering freely.

```
simulate_traits(tree, OrnsteinUhlenbeck(sigma2=0.4, alpha=2.0, theta=10.0), seed=1)
```

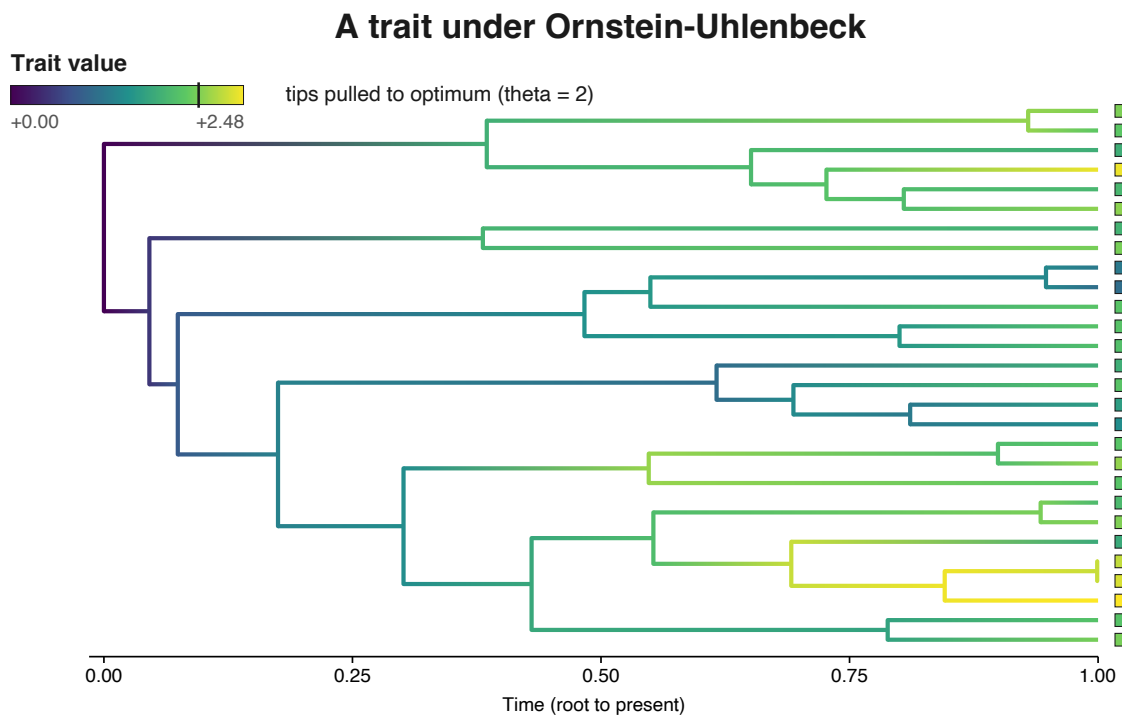


Figure 13.3. Ornstein–Uhlenbeck: the trait diffuses but is pulled back toward an optimum, so lineages cluster around it.

13.3.3 Early burst / ACDC

EarlyBurst is Brownian motion whose rate changes exponentially through time, $\sigma^2(t) = \text{sigma2} \cdot e^{\text{rate} \cdot t}$ (Harmon et al. 2010). A negative **rate** is an *early burst*, in which most divergence happens early, the signature of an adaptive radiation; a positive **rate** accelerates through time; **rate** = 0 recovers plain Brownian motion.

```
simulate_traits(tree, EarlyBurst(sigma2=1.0, rate=-0.8), seed=1)
```

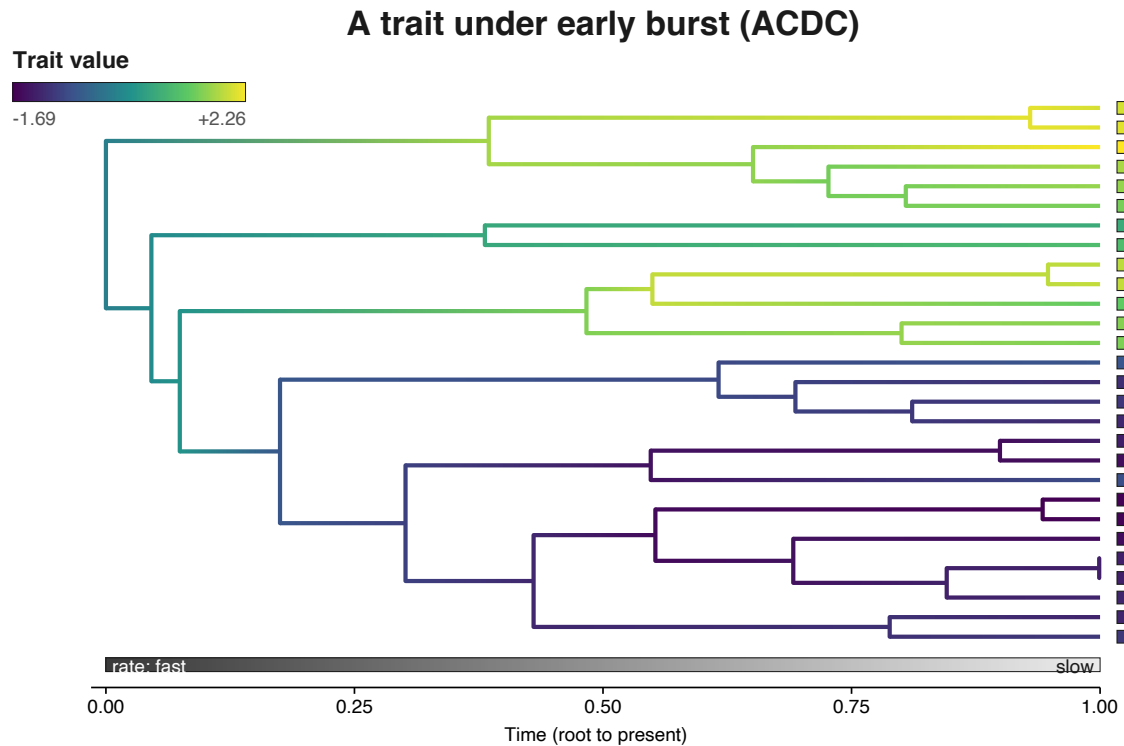


Figure 13.4. A trait under early burst (ACDC). The rate decays through time (grey strip: fast early, slow late), so most change happens early — the deep branches carry big divergences (clades split apart) while young clades stay similar. Branches are painted by the trait value (viridis).

13.3.4 Correlated continuous traits

A vector-valued trait with a rate (covariance) matrix R couples its dimensions, the standard model of correlated multivariate evolution (Clavel et al. 2015). Each node's value is a length- k array, and off-diagonal entries of R make the dimensions evolve together.

```
R = [[1.0, 0.9],
      [0.9, 1.0]]      # strong positive correlation between the dimensions
simulate_traits(tree, MultivariateBrownian(R), seed=1)

# multivariate OU: pull each dimension toward an optimum
# (alpha may be a scalar, vector, or matrix)
simulate_traits(tree, MultivariateOU(R, alpha=1.5, theta=[0.0, 5.0]), seed=1)
```

Multivariate Brownian motion

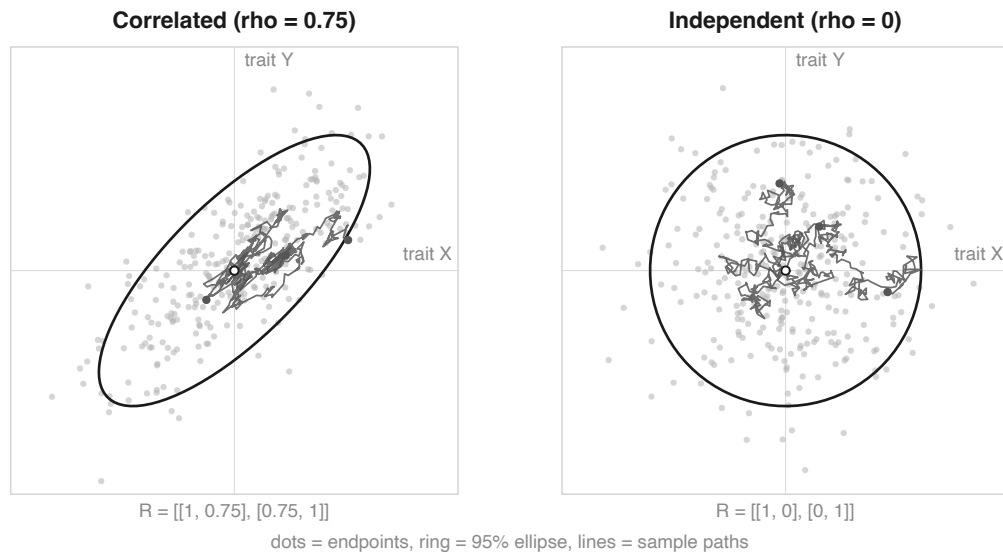


Figure 13.5. Multivariate Brownian motion. Two traits diffuse jointly under a rate matrix R ; each dot is one simulated endpoint, the ring is the 95% covariance ellipse, and the lines are sample paths. **Left**, a correlated R tilts the cloud so the traits move together; **right**, an independent R gives an upright circle.

13.4 Discrete traits

13.4.1 The Mk model

A discrete character evolves as a continuous-time Markov chain over k states with a rate matrix Q (Lewis 2001). Convenience constructors cover the standard sub-models, and the raw constructor takes any Q , so you can specify an all-rates-different or an ordered/meristic character.

```
# equal-rates Mk over 3 labeled states (one shared rate)
mk = simulate_traits(
    tree, Mk.equal_rates(3, 0.4, states=["marine", "brackish", "fresh"]), seed=2)

mk.labeled_values()    # {extant leaf: "marine" | "brackish" | "fresh"}
mk.history[node]       # [(state, duration), ...] - the stochastic map
mk.changes()           # the transition events

Mk.symmetric([[0, 2, 1], [2, 0, 3], [1, 3, 0]])    # SYM: symmetric Q
Mk.ordered(4, 0.5)    # ordered: adjacent-only chain
Mk([[0, 1, 2], [3, 0, 1], [1, 1, 0]])    # ARD: any user-supplied Q
```

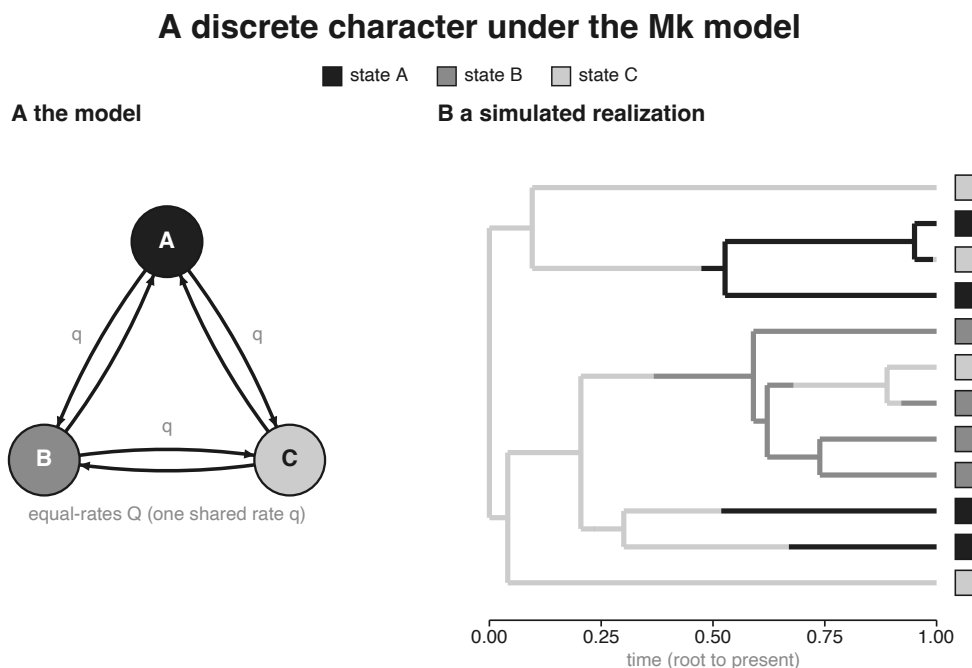



Figure 13.6. A discrete character under the Mk model. **A**, the model — an equal-rates chain over three states (all the transition structure lives in the rate matrix Q). **B**, a realization: the exact stochastic character map, each branch painted in the state it is in and switching at the instant of a transition; the tip chips give each lineage’s observed state.

The transition structure lives entirely in the Q you pass. `equal_rates` is all-to-all at one rate, `symmetric` makes $i \rightarrow j$ and $j \rightarrow i$ equal, `ordered` is the tridiagonal nearest-neighbour chain, and the raw constructor takes an arbitrary Markov chain. Every Mk also exposes its analytic quantities:

```
m = Mk.equal_rates(3, 0.4)
m.transition_matrix(1.0)           #  $P(t) = \exp(Q \cdot t)$ 
m.stationary_distribution()         #  $\pi$  with  $\pi Q = 0$ 
```

The root state is "uniform" by default; pass `root="stationary"`, an index, or a probability vector.

13.4.2 Correlated binary characters (Pagel)

`CorrelatedBinary` evolves two binary characters **X** and **Y** jointly, one changing at a time (Pagel 1994). Each trait’s gain and loss rate may depend on the *other* trait’s state, and that dependence is exactly what correlated evolution means. The `independent` constructor builds the null model in which the two evolve separately, giving the comparison that tests for correlation.

```
# Y tracks X: Y is gained quickly when X = 1 and lost quickly when X = 0
m = CorrelatedBinary(x_gain_y0=0.5, x_gain_y1=0.5, x_loss_y0=0.5, x_loss_y1=0.5,
                    y_gain_x0=0.05, y_gain_x1=2.0, y_loss_x0=2.0, y_loss_x1=0.05)
res = simulate_traits(tree, m, seed=1)
res.labeled_values()                # {extant leaf: (X, Y)}

null = CorrelatedBinary.independent(x_gain=0.5, x_loss=0.5, y_gain=0.5, y_loss=0.5)
```

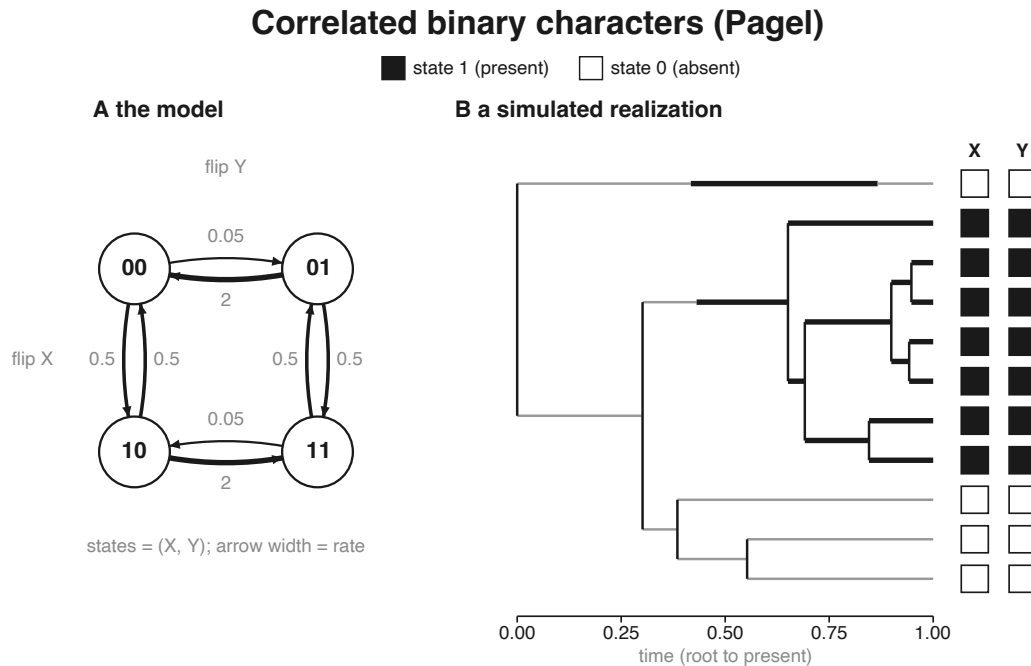


Figure 13.7. Correlated binary characters. **A**, the joint state space $\{00, 01, 10, 11\}$: horizontal edges flip Y, vertical edges flip X, and each arrow's width is its rate. With these rates Y is gained fast only when X = 1 and lost fast when X = 0, so Y is driven toward X. **B**, one realization on a tree: branches are drawn heavy where X = 1 and light where X = 0, and the two tip columns give each leaf's (X, Y) — the clades where X switches on are exactly the clades where Y is present.

13.4.3 Hidden rate classes (corHMM)

HiddenStateMk gives the observed character *hidden rate classes*: its transition rates depend on an unobserved class that itself switches along the tree, capturing rate heterogeneity that a plain Mk cannot represent (Beaulieu et al. 2013). Tips report the observed state; `full_label` and `changes()` expose the hidden dimension.

```
slow = [[0, 0.1], [0.1, 0]]
fast = [[0, 3.0], [3.0, 0]]
hmm = HiddenStateMk(observed_rates=[slow, fast], hidden_rate=0.5,
                    observed_states=[0, 1], hidden_states=["slow", "fast"])
res = simulate_traits(tree, hmm, seed=1)
res.labeled_values()                                # observed 0/1 (hidden class collapsed)
res.full_label(res.node_values[tree.extant_leaves()[0]]) # (observed, hidden)
```

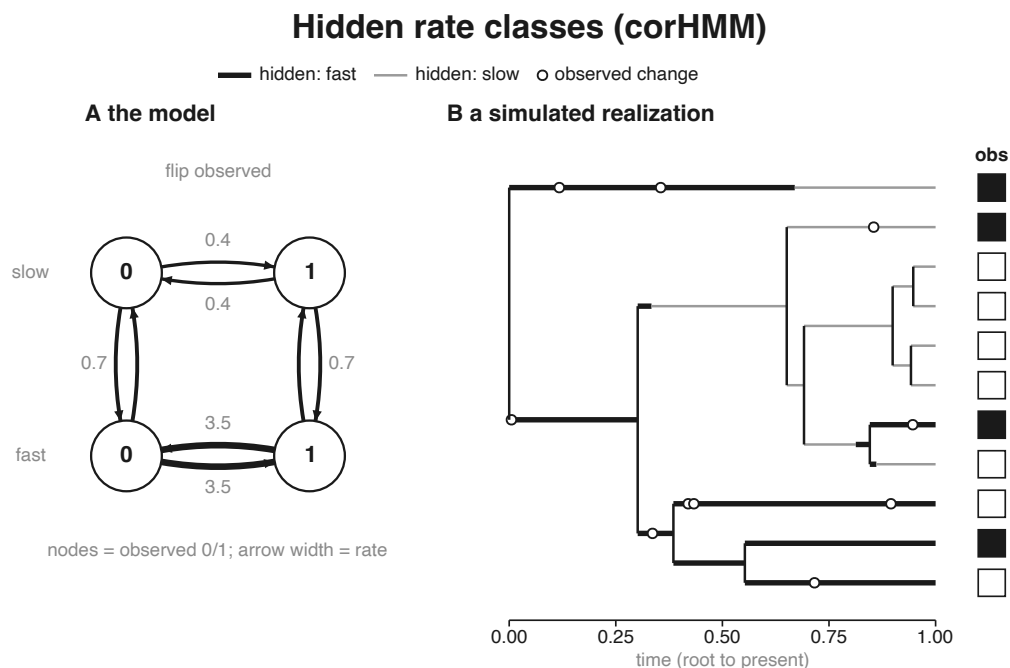


Figure 13.8. Hidden rate classes. **A**, the four joint states, (observed 0/1) $\times$ (hidden slow/fast): the observed character flips slowly in the slow class and quickly in the fast class (arrow width = rate), while the hidden class itself switches along the tree. **B**, a realization — branches are heavy where the hidden class is fast and light where it is slow, small circles mark observed-state changes, and the tip chips give the observed state. The observed changes cluster on the fast (heavy) branches, the signature of hidden rate classes.

13.4.4 The threshold model

In the threshold model an unobserved continuous *liability* evolves by Brownian motion, and the observed discrete state is the interval the liability falls in, cut by an ordered set of thresholds (Felsenstein 2012). The evolving value is the liability; the observed state comes from `labeled_values()`. Because the underlying process is continuous, the model links quantitative and discrete evolution.

```
th = simulate_traits(tree, ThresholdModel(thresholds=[0.0]), seed=1) # binary
th.values                      # liabilities (continuous, latent)
th.labeled_values()            # observed 0/1 states
```

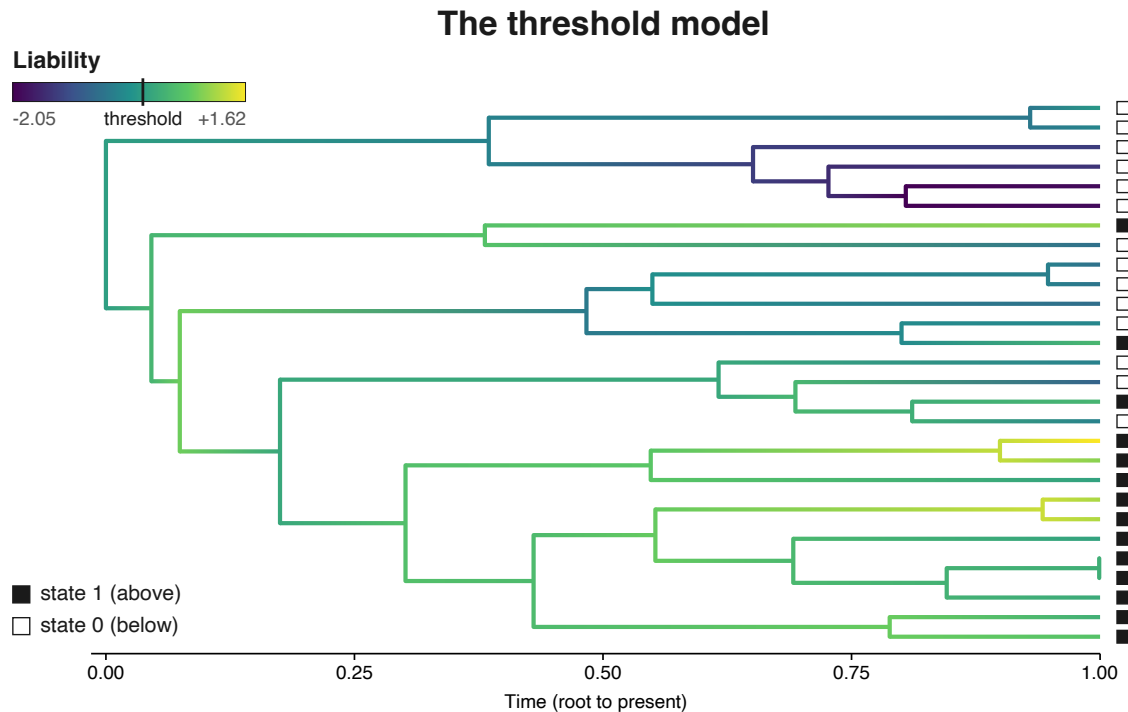


Figure 13.9. The threshold model. A latent liability evolves by Brownian motion — the branches are painted by its value (viridis) — and the observed discrete state at each tip is which side of the threshold the liability ends on: open square = below (state 0), filled = above (state 1). The threshold is marked on the colour bar, so colours to its left are below and to its right above. The continuous gradient flowing into a binary tip pattern is the model’s point.

Discrete simulation, whether Mk, correlated-binary, hidden-rate, or the DEC ranges below, always yields a full stochastic character map (Nielsen 2002; Huelsenbeck et al. 2003); `history` and `changes()` read it back.

13.5 Adaptation to different regimes (multi-optimum OU)

Different parts of the tree can adapt toward different optima, the multi-optimum OU of OUwie (Beaulieu et al. 2012). The *regimes* come from a discrete stochastic map: simulate them with Mk on the *same* tree, then run an OU with one `theta` per regime. `alpha` and `sigma2` may also vary by regime.

```
regimes = simulate_traits(tree, Mk.equal_rates(2, 0.4), seed=1)           # paint 2 regimes
mou = MultiOptimumOU(regimes, theta=[-5.0, 5.0], alpha=4.0, sigma2=0.4)
simulate_traits(tree, mou, seed=2)                                       # tips track their op
```

This is the natural way to model a shift in selective regime: a discrete character paints where the optimum changes, and the continuous trait chases whichever optimum its lineage currently sits under.

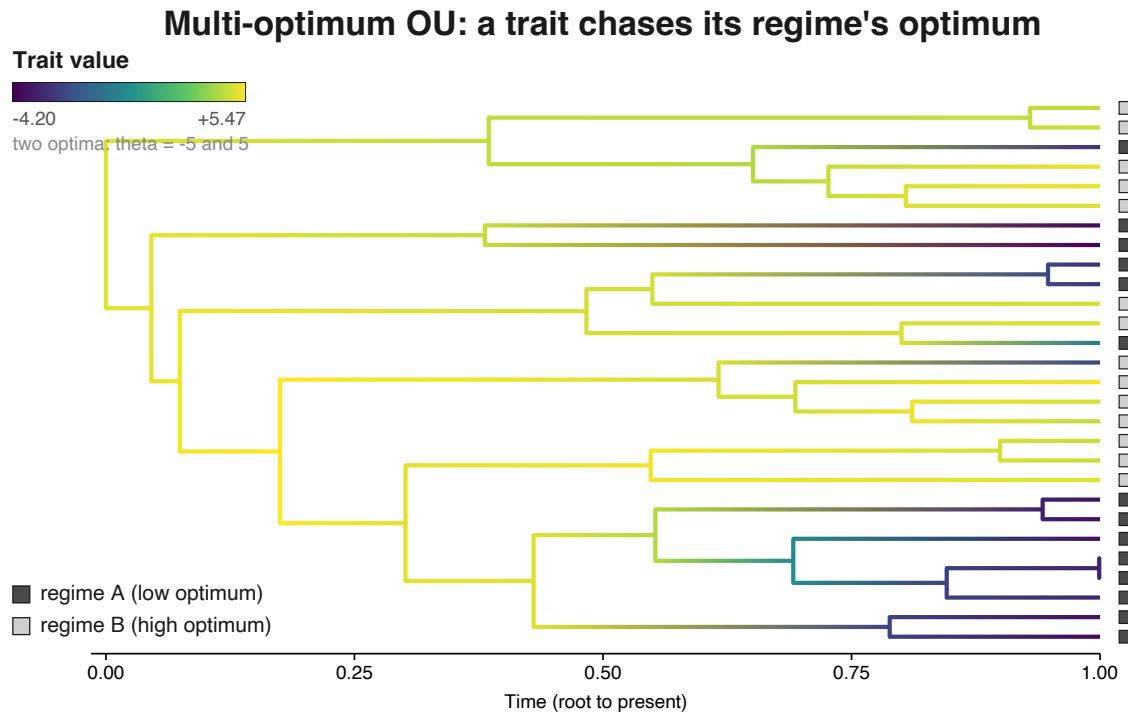


Figure 13.10. Multi-optimum OU. A discrete character paints two regimes on the tree (the tip bars: regime A dark, regime B light); the continuous trait then evolves under OU with a different optimum per regime. Branches are painted by the trait value (viridis) — regime-A lineages are pulled toward the low optimum, regime-B lineages toward the high one, so the tips split into two clusters.

13.6 Pagel’s tree transforms

Pagel’s λ , κ and δ transform the tree’s branch and node lengths (Pagel 1999); run any trait model on the transformed tree to model departures from a strict clock.

```
simulate_traits(pagel_lambda(tree, 0.5), BrownianMotion(0.5), seed=1) # scale signal
pagel_delta(tree, 2.0) # node depths ^delta (>1 late, <1 early change)
pagel_kappa(tree, 0.0) # branch lengths ^kappa (0 = speciation: unit branches)
```

- λ scales internal (shared) depths while holding tip depths fixed: 1 is the original tree (full phylogenetic signal), 0 a star tree (independent tips).
- δ raises node depths to a power, root and tips fixed: values above 1 concentrate change late, below 1 early.
- κ raises each branch length to a power; 0 gives a speciation model, change per speciation event rather than per unit time.

13.7 Historical biogeography (DEC)

A species’ “trait” can be its *geographic range*, a subset of discrete areas. The Dispersal–Extinction–Cladogenesis model evolves the range by *dispersal* (gaining an area) and local *extinction* (losing one) along branches, plus a *cladogenetic* split of the ancestral range between daughters at each

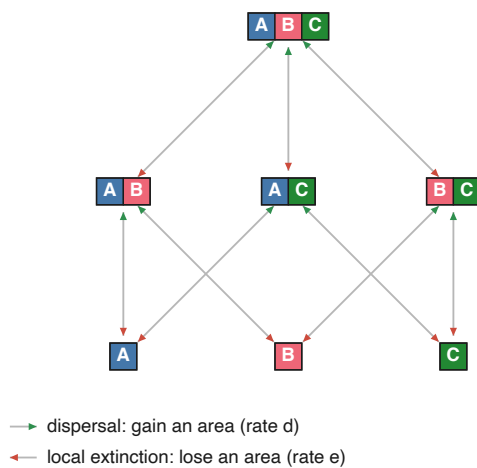
speciation, by narrow sympatry, subset sympatry, or vicariance (Ree and Smith 2008). Because of that node process, DEC has its own driver, `simulate_biogeography`.

```
dec = DEC(areas=["A", "B", "C"], dispersal=0.1, extinction=0.1, max_range_size=3)
res = simulate_biogeography(tree, dec, root_state={"A"}, seed=1)
```

```
res.labeled_values()           # {extant leaf: ('A', 'B') ...} - the observed ranges
res.ancestral_states()        # ancestral ranges at every internal node
res.changes()                  # anagenetic dispersal / extinction events along branches
```

The DEC model of geographic-range evolution

A along a branch: dispersal & extinction



B at a speciation: cladogenesis

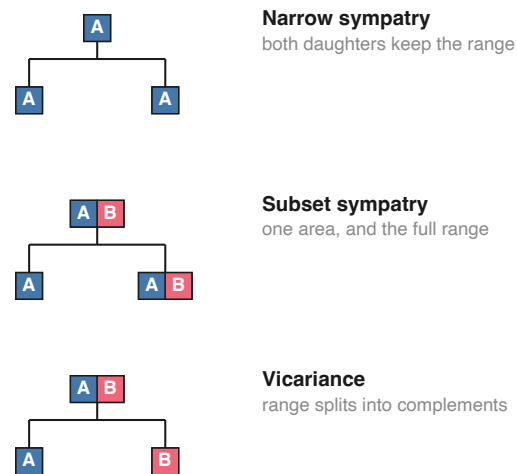


Figure 13.11. The DEC model of geographic-range evolution. **A**, along a branch a range gains areas by dispersal and loses them by local extinction — a Markov chain over the lattice of ranges (subsets of areas). **B**, at each speciation the ancestral range is split between the two daughters by narrow sympatry, subset sympatry, or vicariance.

`dispersal` may be a scalar or an area-by-area matrix, `extinction` a scalar or a per-area vector, and `max_range_size` caps how many areas a range may span.

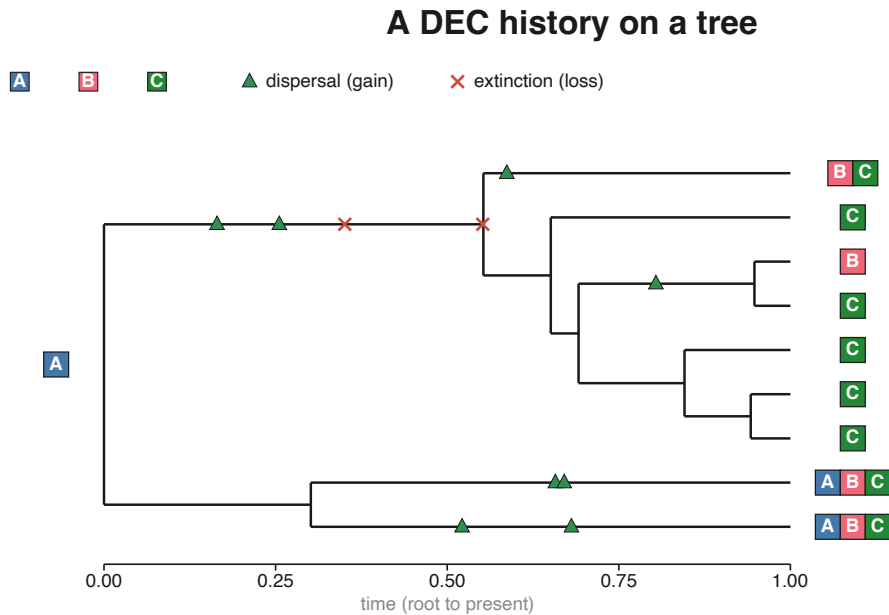


Figure 13.12. A DEC history simulated on a tree. The root begins in area A; along the branches the range gains areas by dispersal (green triangles) and loses them by local extinction (red crosses), and at each speciation the ancestral range is passed to the daughters. Each tip’s realized range is drawn as a row of area cells.

13.8 From the command line

The `zombi2 trait` command runs a single model on a tree you provide and writes the tip and ancestral values:

```
T=species_tree.nwk

zombi2 trait -t $T --model bm --sigma2 0.5 --seed 1 -o out/
zombi2 trait -t $T --model ou --alpha 2.0 --theta 10.0 -o out/
zombi2 trait -t $T --model eb --rate -0.8 -o out/
zombi2 trait -t $T --model mk --states 3 -o out/                # equal-rates Mk
zombi2 trait -t $T --model mk --ordered -o out/                # adjacent-only chain
zombi2 trait -t $T --model mk --q-matrix Q.tsv -o out/          # arbitrary Q
zombi2 trait -t $T --model threshold -o out/
zombi2 trait -t $T --model dec --areas A,B,C --dispersal 0.3 -o out/
```

For the Mk model, `--model mk` is equal-rates by default, `--ordered` gives the adjacent-only (tridiagonal) chain, and `--q-matrix FILE` reads an arbitrary Q. `--replicates N` repeats the simulation on the same tree.

Tip

To simulate a multi-optimum OU or any regime-dependent model, first paint the regimes with a discrete Mk run on the tree, then pass that `TraitResult` to `MultiOptimumOU`. The Python API is the way to compose the two stages; the CLI runs one model at a time.

13.9 Output

Continuous traits are written as their value, multivariate traits as {a,b,c}, and discrete traits as their state label. `to_tsv()` writes a `node<TAB>trait` table, and `to_newick()` a Newick with `[&trait=...]` on every node; pass `nodes="all"` to include the internal (ancestral) nodes.

Chapter 14

Coevolution

14.1 The idea

ZOMBI2 normally simulates in a **pipeline**: it builds a species tree, then evolves a trait along it, then evolves gene families along it. This works because the couplings between the three axes are *one-directional* and the joint distribution factorises,

$$P(\text{tree}) \cdot P(\text{trait} \mid \text{tree}) \cdot P(\text{genes} \mid \text{tree}, \text{trait}),$$

so each stage can be drawn on the frozen output of the previous one. The moment a trait or a genome feeds **back** into the process that generated the tree, this factorisation breaks: the tree, the trait and the gene content must be grown *together*. The **coevolve** mode is for these coupled scenarios.

There are three processes:

- **S** — species diversification (the birth–death process that grows the tree),
- **T** — a phenotypic trait (BM/OU/EB/Mk/threshold),
- **G** — gene-family content (the DTL process).

A **coupling is a directed edge driver → target**: the driver’s state modulates the target’s rates. A coevolution scenario is a **set of directed edges** on these three nodes. That single abstraction covers everything from state-dependent diversification to full three-way feedback, and it makes the one thing that matters — *direction* — explicit.

On the command line the coupling is one repeatable flag, `--couple driver:target`, where the order reads as the arrow (driver first):

```
zombi2 coevolve --couple traits:species ...    # T->S: trait sets speciation (SSE)
zombi2 coevolve --couple species:traits ...    # S->T: speciation drives the trait
# both arrows at once = ClaSSE:
zombi2 coevolve --couple traits:species --couple species:traits ...
```

So `--couple species:traits` and `--couple traits:species` are deliberately **different models**, and a bidirectional coupling is simply *both* edges. The `:` (rather than `->`) keeps the flag shell-safe.

The three processes give six directed edges, each a distinct model:

Edge (<code>--couple</code>)	Direction	Model
<code>traits:species</code>	$T \rightarrow S$	state-dependent diversification (SSE / ClaSSE)

Edge (<code>--couple</code>)	Direction	Model
<code>species:traits</code>	$S \rightarrow T$	cladogenetic trait jumps at speciation
<code>genes:species</code>	$G \rightarrow S$	key-innovation diversification
<code>species:genes</code>	$S \rightarrow G$	punctuational (cladogenetic) genome
<code>traits:genes</code>	$T \rightarrow G$	trait-linked gene families
<code>genes:traits</code>	$G \rightarrow T$	gene-conditioned trait

These six are the *building blocks*. They are not the whole story, because each node-pair's two edges can be switched on **together**, giving a **joint (bidirectional) model** — three more: **ClaSSE** (traits $\leftrightarrow$ species), **co-diversification** (species $\leftrightarrow$ genes) and **trait–gene feedback** (traits $\leftrightarrow$ genes), each documented in the “Both arrows” section under its pair. (And the named literature models are choices *within* an edge: BiSSE, MuSSE and QuaSSE are three flavours of the single `traits:species` edge, picked with `--sse-model`.) So there are more models than arrows.

Coevolve mode: directed edges and their joint models

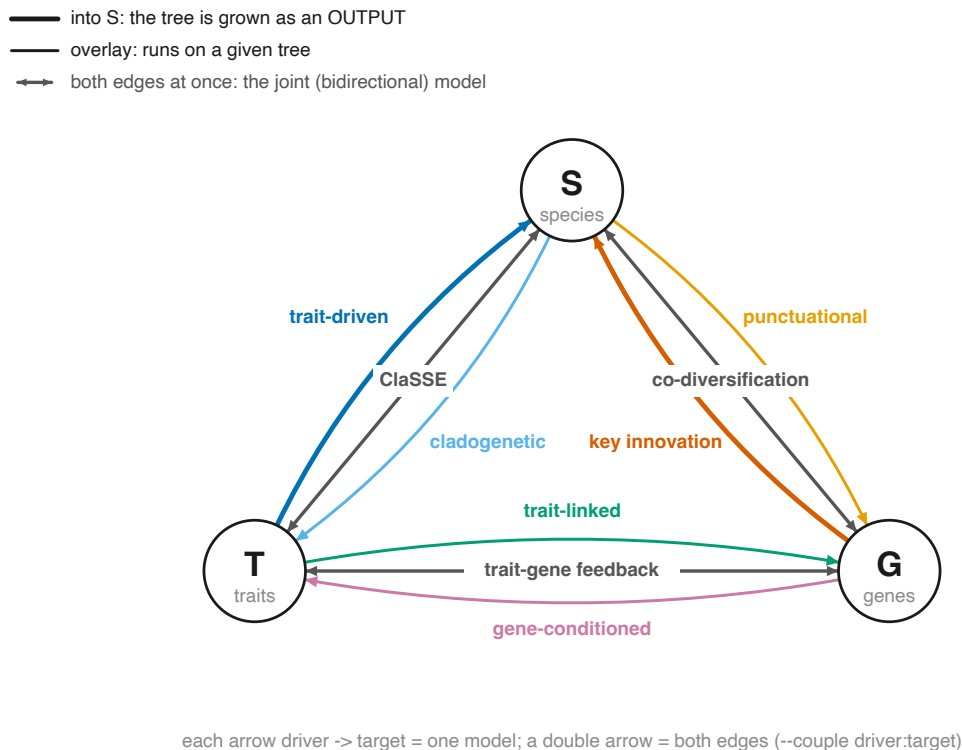


Figure 14.1. The couplings of `coevolve` mode. Each **directed** arrow driver → target is one model, coloured to match its label and selected with `--couple driver:target`; the two that point *into* S are drawn heavy, because an arrow into S makes the tree depend on the coupled state, so the tree becomes an output (the other four are overlays on a tree you supply). The straight **double-headed** arrow between each pair is that pair's *joint* model — both edges at once: ClaSSE (traits $\leftrightarrow$ species), co-diversification (species $\leftrightarrow$ genes) and trait–gene feedback (traits $\leftrightarrow$ genes).

The one rule that governs their difficulty is: **does any active edge point into S?** If no edge points into S, the tree is fixed — it is read from `-t/--tree` and every coupling is an *overlay* on a frozen tree. If an edge does point into S (`traits:species` or `genes:species`), the tree topology depends on the coupled state and cannot be drawn first: the tree becomes an **output**, and those runs are forward-only and take no `-t`.

Note the asymmetry: an arrow pointing *out* of S (`species:traits`, `species:genes`) does **not** trigger joint simulation. S drives the target but listens to nothing, so S can be drawn (or supplied via `-t`) first and the target overlaid on it — the tree stays an input. It is only an arrow *into* S that puts S downstream of its driver, breaks the pipeline factorisation, and forces the tree to be grown jointly as an output. So “touches S” is not the trigger; “points into S” is.

The rest of the chapter takes the three node-pairs in turn — species and traits, species and genes, traits and genes — documenting both directions of each, and the combined model where both arrows are on. The overlay edges all run on a species tree you provide; the examples build one with

```
from zombi2.species import BirthDeath, simulate_species_tree, prune
tree = simulate_species_tree(BirthDeath(1.0, 0.3), n_tips=60, age=6.0, seed=1)
```

14.2 Species and traits

14.2.1 `traits:species` — state-dependent diversification (SSE)

A discrete or continuous trait drives speciation and extinction, and the tree is grown *jointly* with the trait. Because the trait shapes the topology, this edge **produces** the tree (it takes no `-t`) and a stopping condition (`n_tips` or `age`) instead. The driver is `simulate_sse`:

```
from zombi2.coevolve import simulate_sse, BiSSE

# BiSSE: state 1 speciates three times faster, so it comes to dominate the standing tips
res = simulate_sse(BiSSE(lambda0=1, lambda1=3, mu0=0.2, mu1=0.2, q01=0.1, q10=0.1),
                  n_tips=200, seed=1)
res.tree          # the complete tree (extinct lineages kept; prune() for the reconstruct
res.labeled_values() # the trait at the extant tips
```

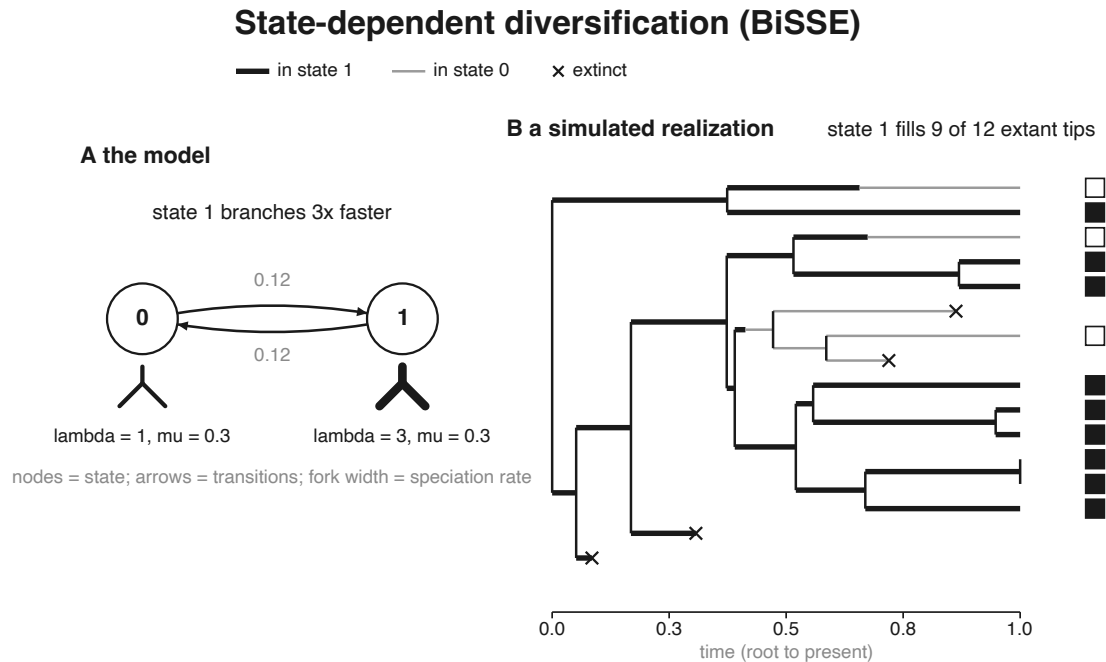


Figure 14.2. State-dependent diversification (BiSSE). **A**, the model: two states, the anagenetic transitions (arrow width = rate), and each state’s speciation rate drawn as a fork (width = λ) — state 1 branches three times faster. **B**, one `simulate_sse` realization: branches are heavy where the lineage is in state 1 and light in state 0, lineages that go extinct end in a cross, and the extant tips carry chips. The fast (state-1) lineages proliferate and fill most of the standing tips — the diversification signal, written into the shape of the tree itself.

BiSSE is the **binary** state-dependent birth–death process (Maddison et al. 2007); MuSSE the **k-state** variant (FitzJohn 2012); QuaSSE the **continuous-trait** variant, whose speciation and extinction are functions of the trait value (FitzJohn 2010); and HiSSE the **hidden-state** model (Beaulieu and O’Meara 2016), where diversification is driven by an unobserved class rather than the focal trait — the honest null against which a real association must be judged.

```
import numpy as np
from zombi2.coevolve import MuSSE, QuaSSE, HiSSE

MuSSE(birth=[1, 3], death=[0.2, 0.2], Q=np.array([[-0.1, 0.1], [0.1, -0.1]])) # k-state
QuaSSE(speciation=lambda x: 1 + 2 / (1 + np.exp(-x)), # continuous
        extinction=lambda x: 0.2, sigma2=0.5, rate_bound=5.0, x0=0.0)
HiSSE(classes=[BiSSE(0.5, 0.7, 0.2, 0.2, 0.1, 0.1), # hidden cla
              BiSSE(2.0, 3.0, 0.2, 0.2, 0.1, 0.1)], hidden_transition=0.1)
```

From the command line, `--sse-model` picks the flavour:

```
zombi2 coevolve --couple traits:species --sse-model bisse \
  --lambda0 1 --lambda1 3 --mu0 0.2 --mu1 0.2 --q01 0.1 --q10 0.1 \
  --tips 200 --seed 1 -o out/
```

It writes `species_tree.nwk` (the tree the trait’s rates shaped), `traits.tsv` (every node — tips and

ancestral states) and `trait_tree.nwk`. `--sse-model musse` takes `--birth/--death` vectors and a `--q-matrix` file; `--sse-model quasse` takes a sigmoidal speciation (`--spec-low/high/center/slope`) and Brownian `--diffusion`. **What it recovers:** the fast-speciating state accumulates lineages, so it dominates the standing tips — the diversification signal is written into the tree shape itself.

Continuous-trait diversification (QuaSSE)

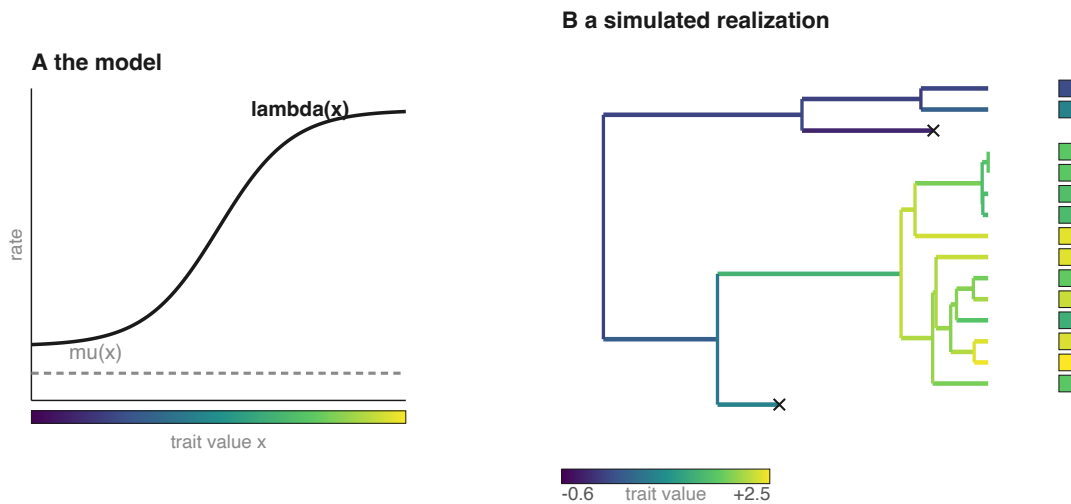


Figure 14.3. The continuous variant, QuaSSE. **A**, the model: the speciation rate is a rising function $\lambda(x)$ of the trait while extinction is flat, and the trait itself diffuses by Brownian motion (the axis is tinted with the viridis ramp used to paint the tree). **B**, one realization, each branch painted by its trait value: the high-value (yellow) lineages branch faster and proliferate, while the low-value (blue) lineages stay sparse and go extinct — the same “fast state fills the tips” signal as BiSSE, now on a continuous character.

14.2.2 species:traits — cladogenetic trait jumps

The reverse arrow makes the trait jump *at* each speciation rather than (or in addition to) drifting along the branches — speciational, or *cladogenetic*, evolution: sister species differ because something happened at their split. On its own this edge has no arrow into S, so it runs on a **given** tree, with a **Cladogenesis** kernel layered on an ordinary anagenetic trait model. Setting the anagenetic rates to zero gives *purely* speciational change:

```
import numpy as np
from zombi2 import Cladogenesis, simulate_traits, Mk
```

```
# a purely speciational binary trait: no within-branch change (zero-rate Mk), a jump at each speciation
res = simulate_traits(tree, Mk(np.zeros((2, 2))), cladogenesis=Cladogenesis(shift=0.4), seed=2024)
```

`Cladogenesis(shift=...)` is the per-daughter state-hop probability for a discrete trait; `Cladogenesis(jump_sigma2=...)` is the Gaussian jump variance for a continuous one. On the command line:

```
zombi2 coevolve --couple species:traits -t species_tree.nwk \
  --sse-model bisse --q01 0 --q10 0 --clado-shift 0.4 --seed 2 -o out/
```

What it recovers: change concentrated at the nodes — closely related tips can differ sharply while long unbranched stretches stay constant, the signature a purely-gradual model cannot produce.

Where the change happens: anagenetic vs cladogenetic

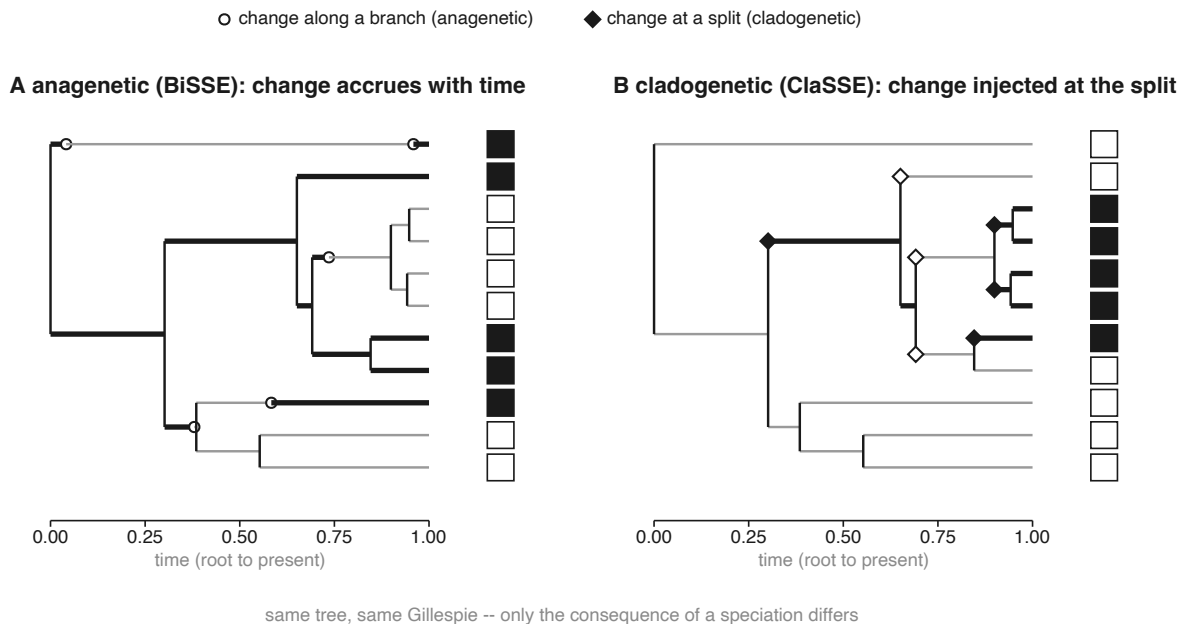


Figure 14.4. Where trait change happens: anagenetic (as in BiSSE) vs cladogenetic (the ClaSSE addition), drawn on **one shared tree** so that only the *location* of change differs. **A**, anagenetic — the trait changes *along* the branches (open circles); at a speciation the daughters inherit the parent, so the amount of change scales with elapsed time and sister tips are usually alike. **B**, cladogenetic — the trait changes *at* the splits (filled diamonds), each daughter drawn as part of the speciation event; change scales with the number of speciations, so sister tips can differ sharply while long unbranched lineages stay constant. Same Gillespie, same tree — only the consequence of a speciation event differs.

14.2.3 Both arrows: ClaSSE

Turn on **both** `traits:species` and `species:traits` and you get the full ClaSSE feedback: the trait shapes the tree *and* is kicked by its own branching. Because one arrow points into S, the tree is again an output:

```
res = simulate_sse(BiSSE(1, 3, 0.05, 0.05, 0.05, 0.05),
  n_tips=200, cladogenesis=Cladogenesis(shift=0.3), seed=3)
```

```
zombi2 coevolve --couple traits:species --couple species:traits \
  --lambda0 1 --lambda1 3 --q01 0.05 --q10 0.05 --clado-shift 0.3 \
  --tips 200 --seed 3 -o out/
```

14.3 Species and genes

14.3.1 `genes:species` — key-innovation diversification

Gene content can drive the tree: a small panel of binary **driver** (“key innovation”) families whose presence sets each lineage’s speciation and extinction rate. Drivers are gained de novo (origination) and — the interesting part — by **transfer**, which is *frequency-dependent*: a driver carried by more of the live population is donated more often, so it spreads as the tree grows. That feedback is why the tree and the gene content must grow together; this edge produces the tree. The neutral bulk genome does not affect diversification, so it is overlaid afterward on the finished tree with the ordinary **genomes** machinery (exact under independent families, not an approximation).

```
from zombi2.coevolve import simulate_gene_diversification, GeneDiversification

res = simulate_gene_diversification(
    GeneDiversification(n_drivers=2, lambda0=1.0, mu0=0.2, driver_speciation=1.2,
                        transfer=0.8, loss=0.3, root_drivers=1),
    n_tips=200, seed=1)
res.tree                # the tree the drivers shaped
res.driver_names()      # ['D0', 'D1']
res.tip_prevalence()    # fraction of extant tips carrying each driver
```

From the command line, the neutral genome overlay is a second, ordinary **genomes** call on the tree the first command wrote:

```
zombi2 coevolve --couple genes:species --drivers 2 --root-drivers 1 \
    --lambda0 1 --mu0 0.2 --driver-speciation 1.2 --driver-transfer 0.8 --driver-loss 0.3 \
    --tips 200 --seed 1 -o out/

# overlay the neutral genome on the resulting tree (the factorisation, made explicit):
zombi2 genomes -t out/species_tree.nwk --trans 1 --loss 0.5 --write profiles trees -o out/
```

coevolve writes `species_tree.nwk`, `drivers.tsv` (per-node driver presence) and `drivers_manifest.tsv` (the effect sizes and rates). `--driver-speciation/--driver-extinction` are the per-driver log-rate effects; `--driver-transfer/--driver-loss/--driver-origination` the driver dynamics. **What it recovers:** a successful key innovation both spreads across the tips (high `tip_prevalence`) and leaves the clades that carry it more speciose — a genomic cause of a diversification rate shift, rather than a trait one.

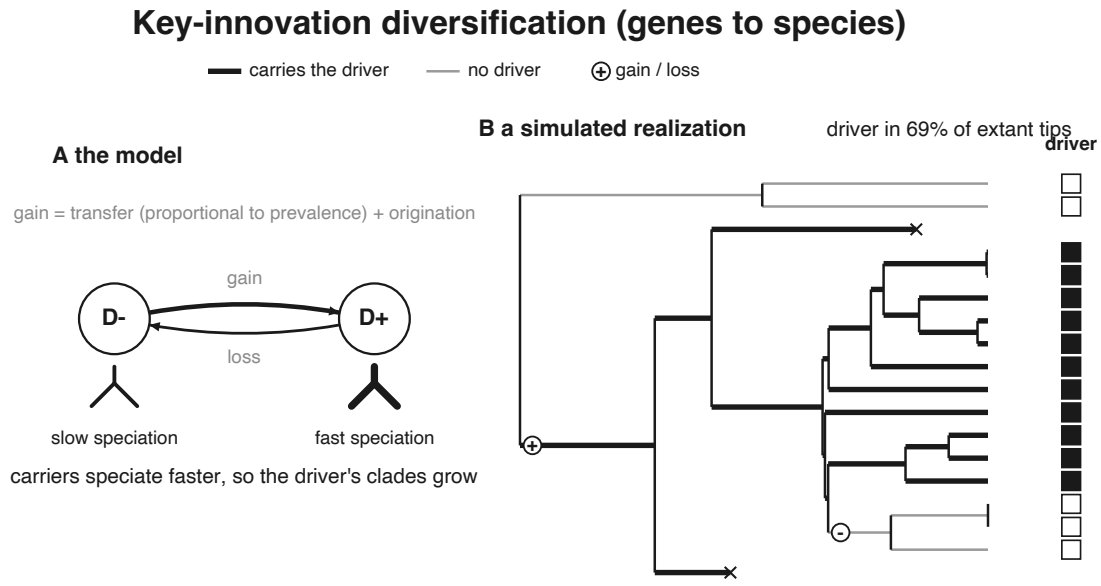


Figure 14.5. Key-innovation diversification. **A**, the model: a lineage carrying the driver (D+) speciates faster than one without it (D—, the fork width is the speciation rate); the driver is gained by origination and by frequency-dependent transfer, and lost. **B**, one realization: the driver *originates* on a single branch (the +), and from there the carrier (heavy) clade radiates into a speciose group while the non-carrier (light) lineages stay sparse. Here it reaches 69% of the extant tips — a genomic cause of a diversification-rate shift.

14.3.2 species:genes — punctuational genome

The reverse of **genes:species**, and the genomic twin of **species:traits**: here gene content does *not* affect diversification, so it is an overlay on a **given** tree. A genome is evolved down the tree with a **cladogenetic burst** of gene loss and gain at each speciation (a founder-effect upheaval), on top of the usual gradual along-branch change:

```
from zombi2.coevolve import simulate_cladogenetic_genome, CladogeneticGenome

# purely punctuational: gene content changes ONLY at speciations (no anagenetic loss/origination)
res = simulate_cladogenetic_genome(
    tree, CladogeneticGenome(initial_families=30, loss=0.0, origination=0.0,
                             cladogenetic_loss=0.15, cladogenetic_gain=3.0), seed=2)
res.genome_sizes()      # {node: family count}
res.profile_matrix()    # families × extant tips

zombi2 coevolve --couple species:genes -t species_tree.nwk \
  --genome-size 30 --clado-gene-loss 0.15 --clado-gene-gain 3 --seed 2 -o out/
```

`--clado-gene-loss` is the per-family drop probability at each speciation and `--clado-gene-gain` the mean number of families gained (Poisson); `--gene-loss`/`--gene-origination` add gradual along-branch change (both 0 = pure punctuation). It writes `Profiles.tsv/Presence.tsv` and

genome_sizes.tsv. **What it recovers:** the model’s signature is that **sister tips differ** — change is injected at their split, not spread evenly along the branches.

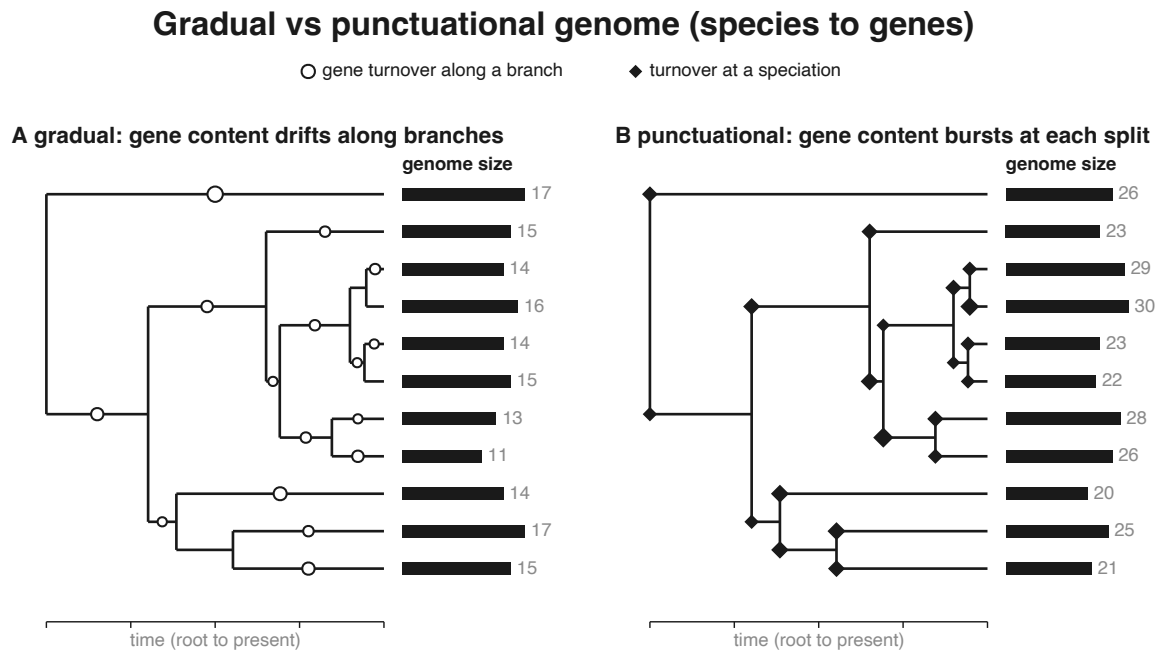


Figure 14.6. The genomic twin of the cladogenetic-trait figure: a genome evolved down **one shared tree** two ways. **A**, gradual — families are lost and gained *along* the branches (circles at the branch midpoints), so gene-content turnover scales with time and sister genomes stay similar. **B**, punctuational — gene content changes only in a *burst at each speciation* (diamonds at the nodes), so sister tips can differ sharply in size and content. The tip bars are the extant genome sizes. Same marker grammar as the trait figure: change on a branch vs change at a node.

14.3.3 Both arrows: co-diversification

Turn on **both** genes:species and species:genes and the *same* driver families both set the diversification rates *and* are reshuffled by a cladogenetic burst at every speciation — the genomic twin of ClaSSE. Because a burst can hand one daughter a key innovation and not its sister, speciation *itself* seeds the rate heterogeneity that then plays out along the branches. One arrow points into S, so the tree is again an output:

```
from zombi2.coevolve import GeneDiversification, simulate_co_diversification
```

```
res = simulate_co_diversification(
    GeneDiversification(3, lambda0=1.0, mu0=0.15, driver_speciation=1.0,
                        loss=0.0, origination=0.0, transfer=0.0, root_drivers=1,
                        cladogenetic_loss=0.15, cladogenetic_gain=0.2),
    n_tips=200, seed=5)
```

```
res.tip_prevalence() # the drivers still spread - the genes:species signal survives the bursts
```

```
zombi2 coevolve --couple genes:species --couple species:genes \
  --drivers 3 --lambda0 1 --mu0 0.15 --driver-speciation 1.0 \
  --driver-loss 0 --driver-origination 0 --driver-transfer 0 --root-drivers 1 \
```

```
--driver-clado-loss 0.15 --driver-clado-gain 0.2 --tips 200 --seed 5 -o out/
```

--driver-clado-loss/--driver-clado-gain are the per-driver drop/gain probabilities of the burst; with both 0 the `species:genes` arrow is off and this is plain `genes:species`. **What it recovers:** the diversification signal (a key innovation over-represented among the tips) *and* the punctuational signal (sisters differing where a burst split them) at once.

14.4 Traits and genes

14.4.1 traits:genes — trait-linked gene families

A lineage that becomes aerobic retains and acquires oxygen-using gene families; one that reverts sheds them. This edge simulates that link: a trait is evolved down the tree, then a panel of gene families is evolved whose **loss depends on the local trait value**. Writing the trait on a branch as s , a *present* family with coupling weight w_i is lost at rate

$$\text{loss}_i = \text{base_loss} \cdot \exp(-\text{effect_loss} \cdot w_i \cdot s),$$

so where the trait favours a responsive family it is retained, and where it does not it is purged faster than baseline; inert families ($w_i = 0$) always lose at `base_loss`. Gain is a trait-blind horizontal influx, and the trait-modulated loss then *selectively retains* it — the same retention mechanism as the gene-family coupling model, which is what writes the trait–gene association into the profiles. The driver takes the tree, a **pre-simulated trait**, and the coupling:

```
import numpy as np
from zombi2 import simulate_traits, BrownianMotion
from zombi2.coevolve import simulate_trait_linked_genomes, TraitGeneCoupling

trait = simulate_traits(tree, BrownianMotion(sigma2=1.0), seed=1)
weights = np.zeros(40); weights[::3] = 1.0           # every third family responds to the trait
res = simulate_trait_linked_genomes(
    tree, trait, TraitGeneCoupling(n_families=40, weights=weights,
                                   effect_loss=2.0, base_loss=1.0, transfer=0.5), seed=1)
res.profiles          # ProfileMatrix: families × extant species - the trait-linked data
res.trait              # the TraitResult the genes were conditioned on
res.genomes()          # promote to a full Genomes (gene trees, reconciliations)
```

The command builds the trait for you from `--trait-model` (any `zombi2` trait model) and selects the responsive families with `--responsive` (a count, a fraction, or an id list):

```
zombi2 coevolve --couple traits:genes -t species_tree.nwk \
  --trait-model bm --sigma2 1 --panel 40 --responsive 0.3 \
  --loss 0.5 --trans 1 --effect-loss 2 --seed 1 -o out/
```

It writes the usual gene-family output plus `traits.tsv/trait_tree.nwk` and `coupling.tsv` (the per-family weights, so the exact trait–gene linkage that generated the profiles is on record for downstream inference). **What it recovers:** responsive families are present where the trait favours them and absent where it does not, while inert families do not distinguish the states — the signal is entirely in the responsive panel, which is what an inference method should be able to pick out.

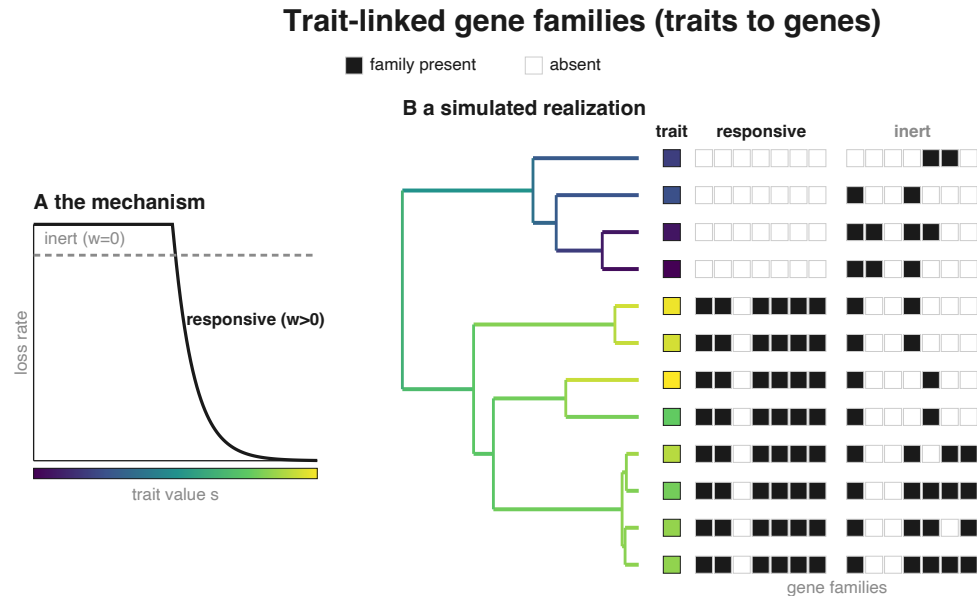


Figure 14.7. Trait-linked gene families. **A**, the mechanism: a responsive family’s loss rate falls with the trait ($\text{loss} = \text{base_loss} \cdot e^{-\text{effect_loss } w s}$), while an inert family loses at the flat baseline. **B**, one realization — the tree painted by the trait (viridis), a per-tip trait chip, then the gene-presence matrix. The responsive block is filled in the high-trait (yellow) tips and empty in the low-trait (blue) ones, tracking the trait almost perfectly; the inert block carries no such pattern. The signal lives entirely in the responsive families — the null is built in.

14.4.2 `genes:traits` — gene-conditioned trait

The reverse: here gene content conditions a **trait**. A binary *modifier* gene is gained and lost along the tree, and its presence sets a continuous trait’s **OU optimum** — a lineage that acquires the gene is pulled toward a new adaptive peak (`theta_present`), one that loses it drifts back toward `theta_absent`. “Gene presence enables a phenotypic shift.” Also an overlay on a given tree:

```
from zombi2.coevolve import simulate_gene_conditioned_trait, GeneConditionedTrait

res = simulate_gene_conditioned_trait(
    tree, GeneConditionedTrait(gene_gain=0.6, gene_loss=0.6, theta_absent=0.0,
                              theta_present=5.0, alpha=2.5, sigma2=0.4), seed=2)
res.gene_presence()      # {leaf: 0/1} - modifier presence at the tips
res.trait_values()       # {leaf: value} - carriers near 5, non-carriers near 0

zombi2 coevolve --couple genes:traits -t species_tree.nwk \
  --modifier-gain 0.6 --modifier-loss 0.6 --theta-absent 0 --theta-present 5 \
  --trait-alpha 2.5 --trait-sigma2 0.4 --seed 2 -o out/
```

`--modifier-gain/--modifier-loss` set the binary modifier’s dynamics (`--root-modifier` to start it present); `--theta-absent/--theta-present` are the two OU optima; `--trait-alpha` is the mean-reversion strength (0 = Brownian) and `--trait-sigma2` the diffusion. **What it recovers:** tips carrying the modifier sit near `theta_present`, those without near `theta_absent` — a

discrete genomic event reading out as a shift in a continuous phenotype.

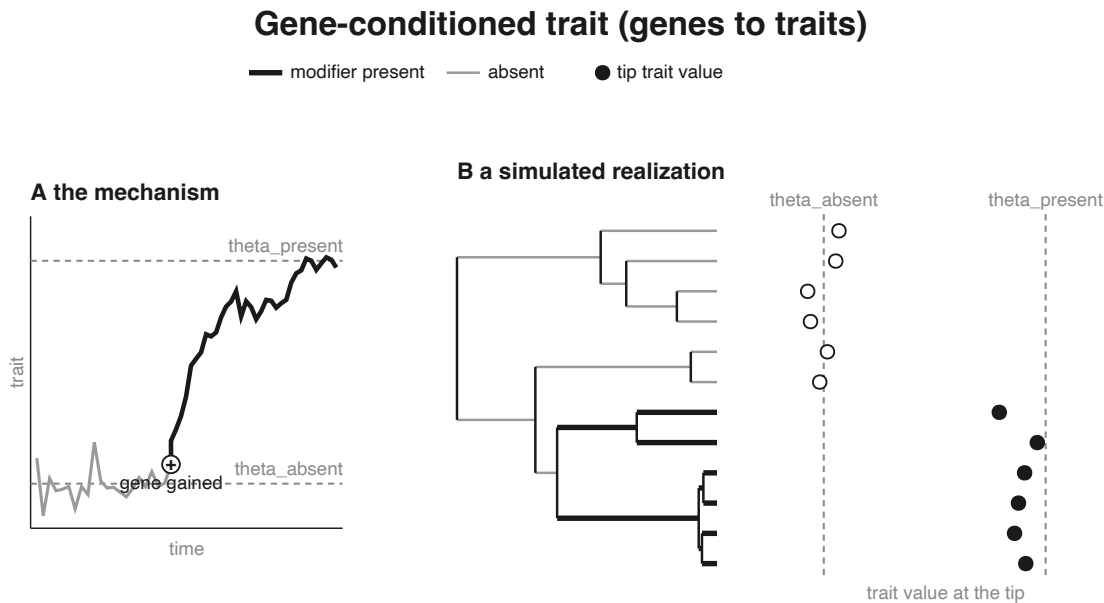


Figure 14.8. Gene-conditioned trait. **A**, the mechanism: one lineage’s trait sits near `theta_absent` while the modifier is absent (light), then the gene is gained (the +) and the trait climbs to the new OU optimum `theta_present` (heavy). **B**, one realization — the tree drawn heavy where the modifier is present, light where absent, and each tip’s trait value as a dot on a shared axis. Carriers (filled) sit at `theta_present`, non-carriers (open) at `theta_absent`: a discrete genomic event reading out as a shift in a continuous phenotype.

14.4.3 Both arrows: trait–gene feedback

Turn on **both** `traits:genes` and `genes:traits` and the trait and a coupled gene panel modulate *each other*: the trait sets the panel’s retention while the panel sets the trait’s optimum. Neither arrow points into S, so this stays an **overlay** on a given tree — but because each depends on the other’s *current* value, the two are integrated together along each branch rather than one after the other. The loop is self-reinforcing (carrying the panel pulls the trait up; a high trait keeps the panel), so a lineage settles into a panel-rich/high-trait or a panel-poor/low-trait regime, and the tips end up with the trait and the panel **correlated even though neither was imposed**:

```
from zombi2.coevolve import TraitGeneFeedback, simulate_trait_gene_feedback

res = simulate_trait_gene_feedback(
    tree, TraitGeneFeedback(n_families=24, effect_loss=1.5, base_loss=1.0, gain=1.0,
                           theta_low=-3.0, theta_high=3.0, alpha=1.0, sigma2=0.5), seed=2)
res.trait_gene_correlation()  # the emergent trait-gene association the feedback writes

zombi2 coevolve --couple traits:genes --couple genes:traits -t species_tree.nwk \
  --panel 24 --effect-loss 1.5 --loss 1.0 --trans 1.0 \
  --theta-absent -3 --theta-present 3 --trait-alpha 1 --trait-sigma2 0.5 --seed 2 -o out/
```

Here `--theta-absent/--theta-present` are the trait's optima at an empty/full panel and `--loss/--trans` the panel's base loss/gain. Setting `--effect-loss 0` recovers pure `genes:traits`, and setting `--theta-present` equal to `--theta-absent` recovers pure `traits:genes` — the joint model contains both single edges as limits. **What it recovers:** a trait-gene-content association that is *emergent* rather than built in; the decoupled control (`--effect-loss 0` with equal thetas) shows none.

14.5 A note on inference

Warning

State-dependent diversification models are notorious for high false-positive rates in *inference*: a neutral trait can look strongly diversification-linked (Maddison and FitzJohn 2015). ZOMBI2 is a **forward generator**, so it is not itself fooled — but the whole point of generating these scenarios is to test downstream inference, so it is worth generating the null (a zero-effect coupling, or the coupling simply absent) as a control. Every edge in this chapter offers one: the HiSSE hidden-state model for `traits:species`, and the inert families that `traits:genes` leaves untouched. Generate the null alongside the signal and check that a method can tell them apart.

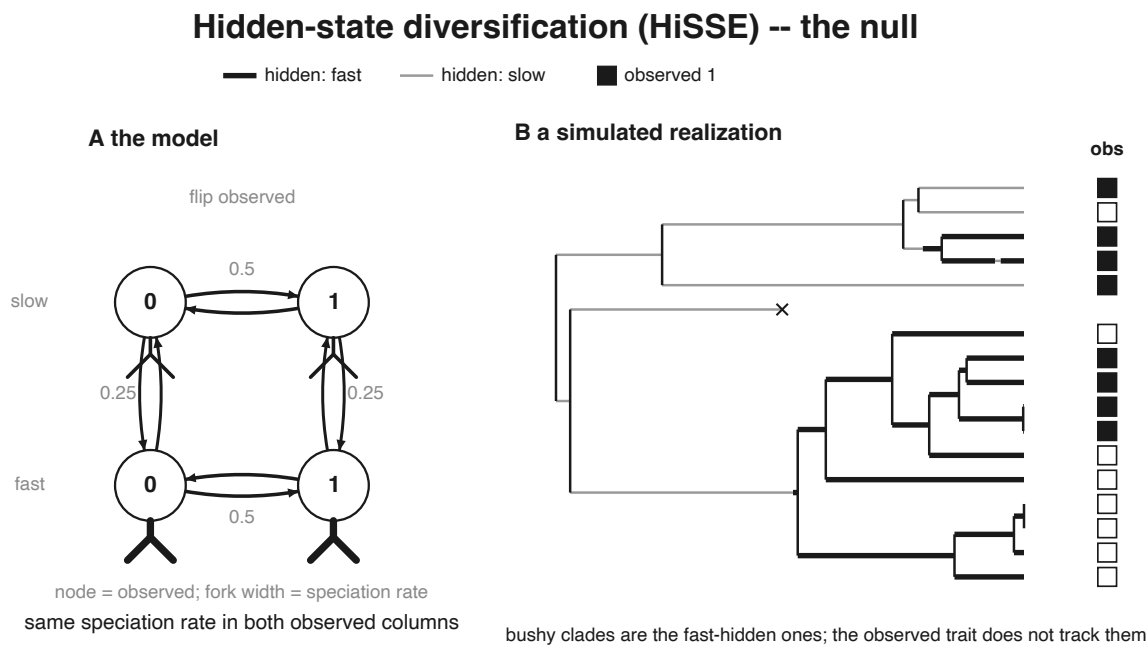


Figure 14.9. The `traits:species` null in action — HiSSE. **A**, the model: diversification is set by an *unobserved* class (a slow regime and a fast one; the fork width is the speciation rate), and it is the *same* in both observed columns, so the observed trait is neutral for diversification. **B**, one realization: branches heavy in the fast hidden class, light in the slow one. The bushy, speciose clades are the fast-hidden ones — but the observed tip states (the `obs` chips) are scattered across them and cannot explain the diversity. A raw BiSSE fit would wrongly read this as a trait effect; generating it is how you check that your inference does not.

Chapter 15

Sequence evolution

Species trees and gene trees in ZOMBI2 are **timetrees**: every branch length is an amount of *time*. Real sequence data does not measure time. What an aligner and a phylogenetic reconstruction see is the number of substitutions accumulated along each branch, and that number depends not only on how long a branch lasted but on how fast it was evolving. A slowly evolving genome and a fast one that lived for a quarter as long can look identical in an alignment.

Sequence evolution is the step that closes this gap. It overlays a substitution rate that varies across the tree and rescales every branch from time into substitutions per site, turning a **chronogram** (branch lengths in time) into a **phylogram** (branch lengths in substitutions). This chapter covers the shared gene $\times$ lineage clock and the `zombi2 sequence` command that applies it to a completed gene-family simulation.

15.1 The problem

A gene tree that comes out of a **genomes** run has branch lengths in time, inherited from the species timetree it was reconciled onto. To obtain the branch lengths you would actually *infer from sequences*, each branch must be multiplied by a rate. If that rate were a single constant everywhere, the tree would just be uniformly stretched: a **strict molecular clock**, and the relative branch proportions would be unchanged. Real lineages violate the strict clock. Rates drift over time, some clades evolve faster than others, and different gene families evolve at systematically different speeds even within the same lineage.

The model therefore needs two things at once: rate variation *across the tree* that is shared by all the genes living in a given lineage, and rate variation *across families* that is intrinsic to each gene. Both are multiplicative overlays on the timetree.

15.2 The model

Every gene family shares the same underlying lineage history. If a clade of the species tree is fast-evolving, it is fast for *all* the families whose gene branches pass through it. **SequenceEvolution** builds this in. It converts a whole `simulate_genomes` result — all the reconciled gene trees at once — from time into substitutions per site under a **gene $\times$ lineage** clock:

$$\text{rate}(\text{family } g, \text{ species branch } b) = R_b \cdot s_g$$

The rate factorises into two independent pieces.

- R_b — **the shared lineage clock**. A rate is drawn **once on the species tree** and **shared by every family**. It captures the fact that a whole region of the tree can be fast or slow regardless of which gene you look at.

- s_g — **the per-family speed**. Each family draws one intrinsic constant from a distribution, so some families are globally fast and others slow, independently of where in the tree they live.

A gene-tree branch that sits on species branch b over the time interval $[t_0, t_1]$ receives substitution length $s_g \cdot R_b \cdot (t_1 - t_0)$. Because reconciliation is exact, a gene branch that spans several species branches (after pruning away losses) simply sums the contributions of the pieces it crosses.

15.2.1 The shared lineage clock

The lineage clock can be **any** of ZOMBI2's relaxed molecular clocks — the full family (strict, uncorrelated lognormal, gamma and white noise, autocorrelated lognormal, and Cox–Ingersoll–Ross) is the subject of Chapter 16. Two are shown here; they are selected differently, and mutually exclusive: the autocorrelated lognormal via `branch_sigma`, and any other clock (including the discrete-bin model below) via `lineage=`.

Autocorrelated lognormal relaxed clock. The rate evolves down the species tree as a geometric random walk,

$$R_{\text{child}} = R_{\text{parent}} \cdot \exp(\mathcal{N}(0, \sigma\sqrt{\ell})),$$

where ℓ is the branch length in time and σ is the `branch_sigma` parameter. Because a child's rate is anchored to its parent's, nearby lineages have similar rates — the clock is *autocorrelated* (Thorne et al. 1998). Setting $\sigma = 0$ freezes the walk and recovers a strict clock, $R_b = 1$ everywhere.

Discrete-bin within-branch clock. The alternative is the discrete-bin, Markov-switching model implemented by `RateVariation`. An **ordered** set of rate **bins** (multipliers, some faster than 1, some slower) is laid down, and a continuous-time Markov process runs *along the phylogeny*, stepping only to an **adjacent bin**. Because the rate can change gradually within a branch, a single branch may be split into several **segments** in neighbouring bins; the substitution length of that branch is the sum of `segment_duration` $\times$ `bin_rate` over its segments. Unlike the lognormal clock it can vary the rate *within* a branch, not only at nodes. You pass a configured `RateVariation` — or any other clock from the family (Chapter 16) — as the `lineage` argument.

15.3 Usage from Python

`SequenceEvolution` takes a completed `simulate_genomes` result and returns a scaled object holding one phylogram per family, together with the rates it drew.

```
import zombi2 as z

tree = z.simulate_species_tree(z.BirthDeath(1.0, 0.3), n_tips=30, age=8.0, seed=1)
genomes = z.simulate_genomes(tree, duplication=0.2, transfer=0.1, loss=0.2,
                             origination=0.5, seed=1)

se = z.SequenceEvolution(branch_sigma=0.5,                                # shared lineage clock
                          family_speed=z.LogNormal(0.0, 0.4))             # per-family speed
phylo = se.scale(genomes, seed=2)

phylo.extant["1"]                # family 1's extant tree, branch lengths in substitutions/site
phylo.complete["1"]              # the complete tree (losses included)
```

```

phylo.family_speed["1"] # the drawn  $s_g$ 
phylo.branch_rate["i5"] # the shared  $R_b$  for species branch i5

```

To use the discrete-bin clock for the lineage part instead of the lognormal one, pass a `RateVariation` as `lineage=` (mutually exclusive with `branch_sigma`); any other clock from the family (Chapter 16) is passed the same way:

```

rv = z.RateVariation(bins=[0.25, 0.5, 1.0, 2.0, 4.0], switch_rate=1.0) # slow -> fast
phylo = z.SequenceEvolution(lineage=rv,
                             family_speed=z.LogNormal(0.0, 0.4)).scale(genomes, seed=2)

```

Bin order matters: the walk only moves between neighbours, so the bins must be given in increasing (or decreasing) order of rate.

15.4 Usage from the CLI

Sequence evolution is a **separate command**, `zombi2 sequence`, run on a prior `genomes` result. Decoupling the two means you can retune the clock without re-simulating gene content. The only requirement is that the `genomes` run wrote its event trace — pass `trace` to `--write`:

```

zombi2 genomes -t species_tree.nwk --dup 0.2 --trans 0.1 --loss 0.2 --orig 0.5 \
    --write trace profiles -o run/

# lognormal lineage clock + per-family speed
zombi2 sequence --genomes run/ --branch-speed 0.4 --family-speed 0.5 -o run/

# discrete-bin lineage clock instead of --branch-speed
zombi2 sequence --genomes run/ --branch-bins 0.25,0.5,1,2,4 --branch-switch-rate 1.0 \
    --family-speed 0.5 -o run/

```

Here `--branch-speed` sets the lognormal σ and `--family-speed` the spread of the per-family speed distribution. The `--branch-bins` form supplies the discrete-bin clock's ordered multipliers, with `--branch-switch-rate` the rate of stepping to a neighbouring bin.

`sequence` replays `run/Events_trace.tsv`, rebuilds the reconciled gene trees from the recorded events, and writes one phylogram per family to `run/gene_trees/<family>_extant_subst.nwk` (and `_complete_subst.nwk` for the trees that include losses). It also writes `gene_family_speeds.tsv` and `branch_rates.tsv`, recording the drawn s_g and R_b so a run is fully reproducible.

Note

`--branch-speed` or `--branch-bins` alone is a shared lineage clock; `--family-speed` alone is per-family speed; together they are the full gene $\times$ lineage model.

Tip

Because `sequence` reads only the event trace on disk, a single expensive `genomes` run can feed many sequence-evolution runs. Sweep the clock parameters — different `--branch-speed`, different bins, different `--family-speed` — reusing the same gene content each time.

15.5 Simulating sequence alignments

Rescaling turns each gene tree into a phylogram whose branch lengths *are* the expected number of substitutions per site. The natural next step is to draw an **actual alignment** down that tree — a sequence at every leaf, evolved base by base (or residue by residue) under a substitution model. Add `--subst-model` to the `sequence` command and it does exactly that, after the rescale step:

```
# DNA alignments (HKY85) along the rescaled gene trees
zombi2 sequence --genomes run/ --branch-speed 0.4 --family-speed 0.5 \
  --subst-model hky85 --seq-length 600 -o run/

# protein alignments under the LG model, with +Γ rate heterogeneity across sites
zombi2 sequence --genomes run/ --branch-speed 0.4 \
  --subst-model lg --seq-length 300 --gamma-shape 0.5 -o run/
```

DNA versus protein is **auto-detected** from the model name. One FASTA per gene family is written to `run/alignments/<family>.fasta`, containing the leaf sequences of that family’s extant tree; each record is headed by the same `<species>_<gene-id>` label the leaf carries in `gene_trees/<family>_extant_subst.nwk`, so alignment and tree line up one-to-one. Omit `--subst-model` and `sequence` behaves exactly as before — it only rescales the trees and writes no alignments.

15.5.1 Available models

A continuous-time Markov substitution model is a normalised rate matrix Q (one expected substitution per site per unit branch length) plus its stationary frequencies. The transition matrix over a branch of length t is $P(t) = e^{Qt}$; every model here is time-reversible, so e^{Qt} is computed by eigendecomposition of the symmetric matrix $B = \text{diag}(\sqrt{\pi}) Q \text{diag}(1/\sqrt{\pi})$.

Nucleotide (4 states, ACGT).

model	description	parameters
jc69	Jukes–Cantor: equal rates, equal base frequencies	—
k80	Kimura 2-parameter: transition/transversion ratio <code>--kappa</code>	<code>--kappa</code>
hky85	HKY85: transition bias with unequal base frequencies	<code>--kappa</code> , <code>--base-freqs</code>
gtr	general time-reversible: 6 exchangeabilities + frequencies	<code>--gtr-rates</code> , <code>--base-freqs</code>

Amino acid (20 states). Empirical exchangeability matrices, transcribed byte-for-byte from the reference PAML data files (Ziheng Yang), plus a parameter-free Poisson model.

model	description
poisson	equal exchangeabilities, uniform frequencies (protein F81)
lg	LG (Le and Gascuel 2008) — the modern default
wag	WAG (Whelan and Goldman 2001)
jtt	JTT (Jones et al. 1992)

model	description
dayhoff	Dayhoff (Dayhoff et al. 1978) (PAML values)

The protein models are empirical and take no parameters. `--gamma-shape ALPHA` adds discrete-Gamma across-site rate heterogeneity (+ Γ) for any model; `--seq-length N` sets the alignment length. To start each family from a chosen root sequence rather than a random draw from the stationary distribution, pass `--root-fasta FILE` — a FASTA keyed by family id, whose per-record length overrides `--seq-length` for that family.

Note

The rescaled branch lengths already carry the gene $\times$ lineage clock, so `--subst-model` needs no extra rate: one unit of branch length is one expected substitution per site by construction.

Chapter 16

Relaxed molecular clocks

Every tree ZOMBI2 produces is a **timetree**: branch lengths are amounts of *time*. Sequence evolution, though, does not accumulate time — it accumulates *substitutions*, and the number of substitutions along a branch is its duration multiplied by an evolutionary **rate**. If that rate were the same everywhere the two would be interchangeable, but real rates vary from branch to branch: some lineages evolve fast, some slow, and the pattern drifts over the tree. A **relaxed molecular clock** is the model of that variation. Applying one rescales every branch from time into expected substitutions per site and turns a **chronogram** (branch lengths in time) into a **phylogram** (branch lengths in substitutions).

Chapter 15 met one relaxed clock already — the shared lineage clock inside `SequenceEvolution`. This chapter steps back and presents the whole **family** of clocks as first-class models with one common interface, so you can rescale *any* tree (a species tree, a gene tree, anything read from Newick), compare models, and plug whichever you like into a sequence-evolution run.

16.1 The strict clock and what relaxing it means

The baseline is the **strict clock**: one rate on every branch. It multiplies the whole tree by a constant, so the phylogram is just the chronogram uniformly stretched — the relative branch proportions are untouched. `StrictClock(rate=1.0)` therefore reproduces the input tree exactly:

```
import zombi2 as z

tree = z.simulate_species_tree(z.BirthDeath(1.0, 0.3), n_tips=30, age=8.0, seed=1)

z.StrictClock(rate=1.0).scale(tree, seed=1).to_newick() == tree.to_newick()    # True
```

`scale` is the shared entry point of every clock. It takes a `Tree` and a seed and returns a `RateScaledTree` — the phylogram, plus the rates it drew. A **relaxed** clock is anything that lets that per-branch rate vary. The clocks differ only in *how* the rate is drawn, and they split into two kinds:

- **Uncorrelated** clocks draw an **independent** multiplier for each branch. A branch's rate tells you nothing about its neighbours' — rate is a property of the branch alone.
- **Autocorrelated** clocks anchor each branch's rate to its parent's, so the rate evolves *along* the tree and nearby lineages have similar rates.

16.2 Drawing a phylogram

Every clock is used the same way. Construct it, call `scale`, and read the result:

```

clock = z.UncorrelatedLogNormalClock(sigma=0.5)    # each branch an i.i.d. multiplier
phylogram = clock.scale(tree, seed=2)

phylogram.to_newick()    # the rescaled tree; branch lengths in substitutions/site
phylogram.branch_rate    # {node: the rate multiplier applied to that node's branch}
phylogram.branch_lengths    # {node: the substitution length of that node's branch}

```

The result is a valid tree in its own right; `to_newick()` writes it out, and it round-trips through `read_newick` like any other. Because a clock is seeded, the same clock and seed always give the same phylogram:

```

a = clock.scale(tree, seed=7).to_newick()
b = clock.scale(tree, seed=7).to_newick()
assert a == b

```

A clock works on a **gene** tree just as well as a species tree — anything with time-valued branch lengths. Load a reconciled gene tree with `read_newick` and scale it directly.

16.3 The uncorrelated clocks

These are the PhyloBayes-style relaxed clocks (Drummond et al. 2006). Each branch draws its rate independently, from a distribution centred (in the first two cases) on mean rate 1, so on average the tree’s total length is preserved.

Uncorrelated lognormal — the workhorse. Each branch draws its rate as $\exp(\mathcal{N}(-\sigma^2/2, \sigma))$, a lognormal with mean 1. The single parameter **sigma** is the amount of heterogeneity: **sigma** = 0 is the strict clock, larger values spread the rates wider.

```

z.UncorrelatedLogNormalClock(sigma=0.5).scale(tree, seed=2)

```

Uncorrelated gamma — the same idea with a gamma distribution: $\text{rate} \sim \text{Gamma}(\text{shape}, 1/\text{shape})$, mean 1, variance $1/\text{shape}$. Here a *large* shape means *less* variation (rates concentrate near 1); a small shape spreads them out.

```

z.UncorrelatedGammaClock(shape=3.0).scale(tree, seed=2)

```

White noise — an uncorrelated clock whose variance depends on branch *length*. The rate is the integral of a white-noise process over the branch, so it is gamma-distributed with mean 1 and variance $\sigma^2/\Delta t$ inversely proportional to the branch duration Δt . Long branches average the noise away and sit near rate 1; short branches are highly variable. This branch-length dependence is what sets it apart from the plain gamma clock above.

```

z.WhiteNoiseClock(sigma=0.5).scale(tree, seed=2)

```

Note

The uncorrelated clocks are memoryless across branches. Two branches meeting at a node can have very different rates — there is no smoothing. If you expect rates to change gradually down the tree, reach for an autocorrelated clock instead.

16.4 The autocorrelated clocks

Here a branch inherits its rate from its parent, so the rate is smooth along the tree.

Autocorrelated lognormal — a geometric random walk down the tree, $R_{\text{child}} = R_{\text{parent}} \cdot \exp(\mathcal{N}(0, \sigma\sqrt{\ell}))$, where ℓ is the branch length in time (Thorne et al. 1998). A child’s rate is centred on its parent’s, and the variance grows with elapsed time. `sigma = 0` freezes the walk into a strict clock at `root_rate`. This is exactly the clock `SequenceEvolution`’s `branch_sigma` selects (see Chapter 15), now available as a model in its own right.

```
z.AutocorrelatedLogNormalClock(sigma=0.3).scale(tree, seed=2)
```

Cox–Ingersoll–Ross (CIR) — a mean-reverting diffusion of the rate itself (Lepage et al. 2007). The instantaneous rate follows

$$dr = \theta(\mu - r)dt + \sigma\sqrt{r}dW,$$

which stays strictly positive and is pulled back toward the long-run mean μ (the `mean` argument) at speed θ (`theta`); σ (`sigma`) is the volatility. Unlike the lognormal walk the rate also varies *within* a branch — the path is simulated by sub-stepping, so a long branch is split into several rate segments — and, because it reverts to the mean, the tree’s total length stays close to the mean times the total time rather than drifting.

```
z.CIRClock(theta=1.0, sigma=0.4, mean=1.0).scale(tree, seed=2)
```

Discrete-bin (GTDB) — the model of Chapter 15’s `RateVariation`, also an autocorrelated clock: an ordered set of rate bins with a nearest-neighbour Markov walk between them along the tree. It is included here for completeness and shares the same interface.

```
z.RateVariation(bins=[0.25, 0.5, 1.0, 2.0, 4.0], switch_rate=1.0).scale(tree, seed=2)
```

The whole family, at a glance:

clock	class	kind	main parameter
strict	<code>StrictClock</code>	—	<code>rate</code>
uncorrelated lognormal	<code>UncorrelatedLogNormalClock</code>	uncorrelated	<code>sigma</code>
uncorrelated gamma	<code>UncorrelatedGammaClock</code>	uncorrelated	<code>shape</code>
white noise	<code>WhiteNoiseClock</code>	uncorrelated	<code>sigma</code>
autocorrelated lognormal	<code>AutocorrelatedLogNormalClock</code>	autocorrelated	<code>sigma</code>
Cox–Ingersoll– Ross	<code>CIRClock</code>	autocorrelated	<code>theta, sigma</code>
discrete-bin (GTDB)	<code>RateVariation</code>	autocorrelated	<code>bins, switch_rate</code>

They all live in the `zombi2.clocks` namespace (and at the top level as `z.StrictClock`, and so on), sharing the `Clock` interface: `scale(tree, seed=...)` returns a `RateScaledTree`.

16.5 Feeding a sequence-evolution run

The clocks are the **shared lineage clock** of Chapter 15’s gene $\times$ lineage model. `SequenceEvolution` took `branch_sigma` (the autocorrelated lognormal) or a `RateVariation`; it now takes *any* clock via `lineage=`, so you can drive a genome simulation’s gene trees with any model in the family:

```
genomes = z.simulate_genomes(tree, duplication=0.2, transfer=0.1, loss=0.2,
                             origination=0.5, seed=1)

se = z.SequenceEvolution(lineage=z.UncorrelatedGammaClock(shape=3.0), # shared lineage clock
                          family_speed=z.LogNormal(0.0, 0.4))         # per-family speed
phylo = se.scale(genomes, seed=2)
```

Everything Chapter 15 says about the lineage clock still holds: it is drawn once and shared by every family, then multiplied by each family’s own speed. Passing `branch_sigma` remains a shorthand for `lineage=AutocorrelatedLogNormalClock` at the same drift σ .

16.6 Usage from the CLI

The `zombi2 sequence` command grows a `--clock` selector for the family. As in Chapter 15 it runs on a prior `genomes` result whose event trace was written (`--write trace`):

```
zombi2 genomes -t species_tree.nwk --dup 0.2 --trans 0.1 --loss 0.2 --orig 0.5 \
  --write trace profiles -o run/

# an uncorrelated lognormal lineage clock, plus per-family speed
zombi2 sequence --genomes run/ --clock uncorrelated-lognormal --clock-sigma 0.5 \
  --family-speed 0.5 -o run/

# a mean-reverting CIR clock instead
zombi2 sequence --genomes run/ --clock cir --clock-theta 1.0 --clock-sigma 0.4 -o run/
```

`--clock` chooses the model — `strict`, `autocorrelated-lognormal`, `uncorrelated-lognormal`, `uncorrelated-gamma`, `white-noise`, `cir`, or `discrete-bin`. Its parameter is supplied by `--clock-sigma` (the spread of the lognormal, white-noise and CIR clocks), `--clock-shape` (the gamma shape), `--clock-theta` (the CIR mean-reversion speed), and `--clock-mean` (the target/strict/root rate, default 1); the discrete-bin clock reads its bins from `--branch-bins`. The output is exactly as in Chapter 15 — one phylogram per family under `run/gene_trees/`, plus `gene_family_speeds.tsv` and `branch_rates.tsv` recording the drawn rates.

The historical flags still work: `--branch-speed SIGMA` is the autocorrelated lognormal clock and `--branch-bins R1,R2,...` the discrete-bin one, so old command lines are unchanged.

Tip

Because `sequence` only replays the event trace on disk, one expensive `genomes` run feeds any number of clock experiments. Sweep `--clock` and its parameters — `strict`, then a couple of uncorrelated models, then CIR — reusing the same gene content, and compare the phylograms.

Note

The rescaled branch lengths already carry the clock, so adding `--subst-model` (Chapter 15) to simulate an alignment needs no extra rate: one unit of the rescaled branch length is one expected substitution per site by construction.

References

- Andréoletti, Jérémy, Antoine Zwaans, Rachel C. M. Warnock, et al. 2022. “The Occurrence Birth–Death Process for Combined-Evidence Analysis in Macroevolution and Epidemiology.” *Systematic Biology* 71 (6): 1440–52. <https://doi.org/10.1093/sysbio/syac037>.
- Beaulieu, Jeremy M., Dwueng-Chwuan Jhwueng, Carl Boettiger, and Brian C. O’Meara. 2012. “Modeling Stabilizing Selection: Expanding the Ornstein–Uhlenbeck Model of Adaptive Evolution.” *Evolution* 66 (8): 2369–83. <https://doi.org/10.1111/j.1558-5646.2012.01619.x>.
- Beaulieu, Jeremy M., and Brian C. O’Meara. 2016. “Detecting Hidden Diversification Shifts in Models of Trait-Dependent Speciation and Extinction.” *Systematic Biology* 65 (4): 583–601. <https://doi.org/10.1093/sysbio/syw022>.
- Beaulieu, Jeremy M., Brian C. O’Meara, and Michael J. Donoghue. 2013. “Identifying Hidden Rate Changes in the Evolution of a Binary Morphological Character.” *Systematic Biology* 62 (5): 725–37. <https://doi.org/10.1093/sysbio/syt034>.
- Butler, Marguerite A., and Aaron A. King. 2004. “Phylogenetic Comparative Analysis: A Modeling Approach for Adaptive Evolution.” *The American Naturalist* 164 (6): 683–95. <https://doi.org/10.1086/426002>.
- Clavel, Julien, Gilles Escarguel, and Gildas Merceron. 2015. “mvMORPH: an R package for fitting multivariate evolutionary models to morphometric data.” *Methods in Ecology and Evolution* 6 (11): 1311–19. <https://doi.org/10.1111/2041-210X.12420>.
- Croce, Giancarlo, Thomas Gueudré, Maria Virginia Ruiz Cuevas, et al. 2019. “A multi-scale co-evolutionary approach to predict interactions between protein domains.” *PLOS Computational Biology* 15 (10): e1006891. <https://doi.org/10.1371/journal.pcbi.1006891>.
- Davín, Adrián A., Théo Tricou, Eric Tannier, Damien M. de Vienne, and Gergely J. Szöllösi. 2020. “Zombi: a phylogenetic simulator of trees, genomes and sequences that accounts for dead lineages.” *Bioinformatics* 36 (4): 1286–88. <https://doi.org/10.1093/bioinformatics/btz710>.
- Dayhoff, Margaret O., Robert M. Schwartz, and Bruce C. Orcutt. 1978. “A model of evolutionary change in proteins.” *Atlas of Protein Sequence and Structure* 5 (Suppl. 3): 345–52.
- Drummond, Alexei J., Simon Y. W. Ho, Matthew J. Phillips, and Andrew Rambaut. 2006. “Relaxed Phylogenetics and Dating with Confidence.” *PLoS Biology* 4 (5): e88. <https://doi.org/10.1371/journal.pbio.0040088>.
- Etienne, Rampal S., Bart Haegeman, Tanja Stadler, et al. 2012. “Diversity-dependence brings molecular phylogenies closer to agreement with the fossil record.” *Proceedings of the Royal Society B: Biological Sciences* 279 (1732): 1300–1309. <https://doi.org/10.1098/rspb.2011.1439>.

- Felsenstein, Joseph. 1985. “Phylogenies and the Comparative Method.” *The American Naturalist* 125 (1): 1–15. <https://doi.org/10.1086/284325>.
- Felsenstein, Joseph. 2012. “A Comparative Method for Both Discrete and Continuous Characters Using the Threshold Model.” *The American Naturalist* 179 (2): 145–56. <https://doi.org/10.1086/663681>.
- FitzJohn, Richard G. 2010. “Quantitative Traits and Diversification.” *Systematic Biology* 59 (6): 619–33. <https://doi.org/10.1093/sysbio/syq053>.
- FitzJohn, Richard G. 2012. “Diversitree: comparative phylogenetic analyses of diversification in R.” *Methods in Ecology and Evolution* 3 (6): 1084–92. <https://doi.org/10.1111/j.2041-210X.2012.00234.x>.
- Fukunaga, Tsukasa, and Wataru Iwasaki. 2022. “Inverse Potts model improves accuracy of phylogenetic profiling.” *Bioinformatics* 38 (7): 1794–800. <https://doi.org/10.1093/bioinformatics/btac034>.
- Gavryushkina, Alexandra, David Welch, Tanja Stadler, and Alexei J. Drummond. 2014. “Bayesian Inference of Sampled Ancestor Trees for Epidemiology and Fossil Calibration.” *PLoS Computational Biology* 10 (12): e1003919. <https://doi.org/10.1371/journal.pcbi.1003919>.
- Hansen, Thomas F. 1997. “Stabilizing Selection and the Comparative Analysis of Adaptation.” *Evolution* 51 (5): 1341–51. <https://doi.org/10.1111/j.1558-5646.1997.tb01457.x>.
- Harmon, Luke J., Jonathan B. Losos, T. Jonathan Davies, et al. 2010. “Early Bursts of Body Size and Shape Evolution Are Rare in Comparative Data.” *Evolution* 64 (8): 2385–96. <https://doi.org/10.1111/j.1558-5646.2010.01025.x>.
- Hartmann, Klaas, Dennis Wong, and Tanja Stadler. 2010. “Sampling Trees from Evolutionary Models.” *Systematic Biology* 59 (4): 465–76. <https://doi.org/10.1093/sysbio/syq026>.
- Heath, Tracy A., John P. Huelsenbeck, and Tanja Stadler. 2014. “The fossilized birth–death process for coherent calibration of divergence-time estimates.” *Proceedings of the National Academy of Sciences* 111 (29): E2957–66. <https://doi.org/10.1073/pnas.1319091111>.
- Höhna, Sebastian, Michael R. May, and Brian R. Moore. 2016. “TESS: an R package for efficiently simulating phylogenetic trees and performing Bayesian inference of lineage diversification rates.” *Bioinformatics* 32 (5): 789–91. <https://doi.org/10.1093/bioinformatics/btv651>.
- Huelsenbeck, John P., Rasmus Nielsen, and Jonathan P. Bollback. 2003. “Stochastic Mapping of Morphological Characters.” *Systematic Biology* 52 (2): 131–58. <https://doi.org/10.1080/10635150390192780>.
- Jones, David T., William R. Taylor, and Janet M. Thornton. 1992. “The rapid generation of mutation data matrices from protein sequences.” *Computer Applications in the Biosciences (CABIOS)* 8 (3): 275–82. <https://doi.org/10.1093/bioinformatics/8.3.275>.

- Lambert, Amaury, and Tanja Stadler. 2013. “Birth–death models and coalescent point processes: The shape and probability of reconstructed phylogenies.” *Theoretical Population Biology* 90: 113–28. <https://doi.org/10.1016/j.tpb.2013.10.002>.
- Le, Si Quang, and Olivier Gascuel. 2008. “An improved general amino acid replacement matrix.” *Molecular Biology and Evolution* 25 (7): 1307–20. <https://doi.org/10.1093/molbev/msn067>.
- Lepage, Thomas, David Bryant, Hervé Philippe, and Nicolas Lartillot. 2007. “A General Comparison of Relaxed Molecular Clock Models.” *Molecular Biology and Evolution* 24 (12): 2669–80. <https://doi.org/10.1093/molbev/msm193>.
- Lewis, Paul O. 2001. “A Likelihood Approach to Estimating Phylogeny from Discrete Morphological Character Data.” *Systematic Biology* 50 (6): 913–25. <https://doi.org/10.1080/106351501753462876>.
- Louca, Stilianos, and Matthew W. Pennell. 2020. “Extant timetrees are consistent with a myriad of diversification histories.” *Nature* 580 (7804): 502–5. <https://doi.org/10.1038/s41586-020-2176-1>.
- Maddison, Wayne P., and Richard G. FitzJohn. 2015. “The Unsolved Challenge to Phylogenetic Correlation Tests for Categorical Characters.” *Systematic Biology* 64 (1): 127–36. <https://doi.org/10.1093/sysbio/syu070>.
- Maddison, Wayne P., Peter E. Midford, and Sarah P. Otto. 2007. “Estimating a Binary Character’s Effect on Speciation and Extinction.” *Systematic Biology* 56 (5): 701–10. <https://doi.org/10.1080/10635150701607033>.
- Maliet, Odile, Florian Hartig, and Hélène Morlon. 2019. “A model with many small shifts for estimating species-specific diversification rates.” *Nature Ecology & Evolution* 3 (7): 1086–92. <https://doi.org/10.1038/s41559-019-0908-0>.
- Nee, Sean, Robert M. May, and Paul H. Harvey. 1994. “The reconstructed evolutionary process.” *Philosophical Transactions of the Royal Society of London. Series B: Biological Sciences* 344 (1309): 305–11. <https://doi.org/10.1098/rstb.1994.0068>.
- Nielsen, Rasmus. 2002. “Mapping Mutations on Phylogenies.” *Systematic Biology* 51 (5): 729–39. <https://doi.org/10.1080/10635150290102393>.
- Pagel, Mark. 1994. “Detecting correlated evolution on phylogenies: a general method for the comparative analysis of discrete characters.” *Proceedings of the Royal Society of London. Series B: Biological Sciences* 255 (1342): 37–45. <https://doi.org/10.1098/rspb.1994.0006>.
- Pagel, Mark. 1999. “Inferring the historical patterns of biological evolution.” *Nature* 401 (6756): 877–84. <https://doi.org/10.1038/44766>.
- Pellegrini, Matteo, Edward M. Marcotte, Michael J. Thompson, David Eisenberg, and Todd O. Yeates. 1999. “Assigning protein functions by comparative genome analysis: Protein phylogenetic profiles.” *Proceedings of the National Academy of Sciences* 96 (8): 4285–88.

<https://doi.org/10.1073/pnas.96.8.4285>.

- Rabosky, Daniel L., and Irby J. Lovette. 2008. “Density-dependent diversification in North American wood warblers.” *Proceedings of the Royal Society B: Biological Sciences* 275 (1649): 2363–71. <https://doi.org/10.1098/rspb.2008.0630>.
- Ree, Richard H., and Stephen A. Smith. 2008. “Maximum Likelihood Inference of Geographic Range Evolution by Dispersal, Local Extinction, and Cladogenesis.” *Systematic Biology* 57 (1): 4–14. <https://doi.org/10.1080/10635150701883881>.
- Stadler, Tanja. 2009. “On incomplete sampling under birth–death models and connections to the sampling-based coalescent.” *Journal of Theoretical Biology* 261 (1): 58–66. <https://doi.org/10.1016/j.jtbi.2009.07.018>.
- Stadler, Tanja. 2011. “Mammalian phylogeny reveals recent diversification rate shifts.” *Proceedings of the National Academy of Sciences* 108 (15): 6187–92. <https://doi.org/10.1073/pnas.1016876108>.
- Stadler, Tanja, Denise Kühnert, Sebastian Bonhoeffer, and Alexei J. Drummond. 2013. “Birth–death skyline plot reveals temporal changes of epidemic spread in HIV and hepatitis C virus (HCV).” *Proceedings of the National Academy of Sciences* 110 (1): 228–33. <https://doi.org/10.1073/pnas.1207965110>.
- Szöllősi, Gergely J., Eric Tannier, Nicolas Lartillot, and Vincent Daubin. 2013. “Lateral Gene Transfer from the Dead.” *Systematic Biology* 62 (3): 386–97. <https://doi.org/10.1093/sysbio/syt003>.
- Thorne, Jeffrey L., Hirohisa Kishino, and Ian S. Painter. 1998. “Estimating the rate of evolution of the rate of molecular evolution.” *Molecular Biology and Evolution* 15 (12): 1647–57. <https://doi.org/10.1093/oxfordjournals.molbev.a025892>.
- Whelan, Simon, and Nick Goldman. 2001. “A general empirical model of protein evolution derived from multiple protein families using a maximum-likelihood approach.” *Molecular Biology and Evolution* 18 (5): 691–99. <https://doi.org/10.1093/oxfordjournals.molbev.a003851>.